

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 13.09.2026, 14:53:25
Файлов исходного кода: 92
Суммарный объём кода: 1.36 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs-bfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sorting]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler-cycle]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракка (KMP) [id: knuth-morris-pratt]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts

25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts

27. src/data/tickets/ticket14_20.ts

28. src/data/tickets/ticket21_24.ts

29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx

31. src/components/BellmanFordViz.tsx

32. src/components/BfsViz.tsx

33. src/components/ChapterImage.tsx

34. src/components/ChapterNav.tsx

35. src/components/ChapterView.tsx

36. src/components/ColoringViz.tsx

37. src/components/ComplexityClassesViz.tsx

38. src/components/ComponentsViz.tsx

39. src/components/DfsBridgesSimulator.tsx

40. src/components/DijkstraViz.tsx

41. src/components/DPVisualizer.tsx

42. src/components/EulerSimulator.tsx

43. src/components/ExpressQuiz.tsx

44. src/components/FloydViz.tsx

45. src/components/GraphTraversalViz.tsx

46. src/components/HeapViz.tsx

47. src/components/hints/demoKit.tsx

48. src/components/hints/demosExtra.tsx

49. src/components/hints/demosGraph.tsx

50. src/components/hints/demosStruct.tsx

51. src/components/hints/MiniDemo.tsx

52. src/components/hints/SimulatorModal.tsx

53. src/components/hints/TermPopover.tsx

54. src/components/JohnsonViz.tsx

55. src/components/KosarajuViz.tsx

56. src/components/KruskalSimulator.tsx

57. src/components/MnemonicCards.tsx

```
88. scripts/export/common.mjs
89. scripts/export/guide-pdf-entry.tsx
90. scripts/export/render-pages.mjs
```

```
# CI / автоматизация
```

```
91. .github/workflows/deploy-pages.yml
92. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/92: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

` .github/workflows/deploy-pages.yml `

Полное содержимое после изменений

```yaml

```
 VITE_BASE: /algv0/
 run: npm run build

- name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
```

```
+
+   - name: Upload Pages artifact
+     uses: actions/upload-pages-artifact@v5
+     with:
+       path: ./dist
+
+ deploy:
+   name: Publish to GitHub Pages
+   needs: build
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+....
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (artifacts)
#
|
|→ full_code_all_page.pdf (artifacts)
```

# Готовые TXT и PDF коммитятся обратно в ветку (их отдаёт скрипт)

# промежуточные PNG сохраняются как artifacts запуска.

on:

- ```
node-version: "24"
cache: npm
```
- name: Install ImageMagick + DejaVu fonts
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommen
 # На Ubuntu политика ImageMagick иногда запре
 # разрешаем то, что нужно пайплайну (text -> p
 for policy in /etc/ImageMagick-6/policy.xml /
 if [-f "\$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
done
convert -version
 - name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${GITHUB_EVENT_INPUTS.densi
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"

Patch относительно `main`

```diff

diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml  
index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB\_STEP\_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

```

 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width,
+ <link rel="icon" type="image/svg+xml" href="%BASE_
 <title> Дискретка для тупых – Интерактивный гид по
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen

```

```
`public/favicon.svg`
```

```
Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
  <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
````
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
````
```

```
`src/App.tsx`
```



```

 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

```

```
const progress = useMemo(() => ((index + 1) / chapter
```

```

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />
 </>
) : (
 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>
 <main id="main-content" className="flex-1 m-8" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Университет"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 </div>
 </main>
 </div>
)}
 </div>
)
)

```

```
 <SimulatorModal vizId={modalVizId} onClose={() =>
 <footer className="p-8 text-center text-slate-600"
 © 2026 Universal Educational Guide. Интерактивные
 </footer>
 </div>
);
}
```

### Patch относительно `main`

```
````diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
  const [activeChapterId, setActiveChapterId] = useState<string>();
  const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
  </>
-  ) : (
+  ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
```

```

return (
  <header className="bg-slate-900 border-b border-slate-800">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="flex items-center gap-3 w-full">
        <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
          <Sparkles className="w-6 h-6" />
        </div>
        <div className="min-w-0 flex-1">
          <h1 className="text-xl font-extrabold text-white">
            Универсальное пособие
          </h1>
          <p className="text-xs text-indigo-400 font-medium">
            Подготовка к экзаменам: Алгоритмы, Структуры
          </p>
        </div>
      </div>
    </div>

    <div className="flex items-center w-full lg:w-auto">
      <button
        id="tab-guide-btn"
        onClick={() => setActiveTab('guide')}
        className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
          activeTab === 'guide'
            ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
            : 'text-slate-400 hover:text-white'
        }`}
      >
        <BookOpen className="w-4 h-4" /> Учебное пособие
      </button>
      <button
        id="tab-simulator-btn"
        onClick={() => setActiveTab('simulator')}
        className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
          activeTab === 'simulator'
            ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
            : 'text-slate-400 hover:text-white'
        }`}
      >
        <Code className="w-4 h-4" /> Симулятор
      </button>
    </div>
  </div>
)

```

Универсальное пособие • полный исходный код

```

        className="flex items-center justify-center
        :bg-sky-600/20 border border-sky-500/30 tra
        title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center
    er:bg-amber-600/20 border border-amber-500/
    title="Скачать всё пособие одним автономным
    >
    {busy ? <Loader2 className="w-4 h-4 animate
    </button>
</div>
</div>
</header>
);
};
....
```

Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
```

```
 rel="noreferrer"

`src/components/PythonCompiler.tsx`

Полное содержимое после изменений

```tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в браузер
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL);
  }
}
```

```

        await runtime.runPythonAsync(code);

        const text = [...stdout, ...stderr].join("");
        setOutput(text || "Готово: программа ничего не вывела");
    } catch (error) {
        const captured = [...stdout, ...stderr].join("");
        setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
        setRuntimeMessage("Не удалось выполнить код – подробности в консоли");
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть результат");
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >

```

```

        onKeyDown={({event}) => {
          if ((event.ctrlKey || event.metaKey) &&
            event.preventDefault();
            void runCode();
          }
        })
        spellCheck={false}
        aria-label="Код Python"
        className="block w-full min-h-[360px] res
        -none focus:ring-2 focus:ring-inset focus:r
        placeholder="Напишите Python-код..."
      />

<div className="border-t border-slate-800 b
  <label className="block px-4 pt-3 text-[1
    Ввод для input() • по одной строке на к
  </label>
  <textarea
    id="python-stdin"
    value={stdin}
    onChange={({event}) => setStdin(event.tar
    spellCheck={false}
    rows={2}
    placeholder={'Например:\nАлиса'}
    className="block w-full resize-y bg-tra
    eholder:text-slate-700"
  />
</div>
</section>

<section className="rounded-2xl border border
  <div className="flex items-center justify-b
    <div className="flex items-center gap-2 t
      <Terminal className="w-4 h-4 text-emera
    </div>
    <span className={`inline-flex items-cente
      {runtimeReady && <CheckCircle2 classNam
      {runtimeReady ? "готов" : "не загружен"

```

```

-----
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, PL
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/py
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/py
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {

```



```

+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };
+
+ return (
+   <main className="max-w-screen-2xl mx-auto px-4 sm:p
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:i
+         <div>
+           <div className="inline-flex items-center g
+             <Terminal className="w-4 h-4" /> Боковая
+           </div>
+           <h2 className="text-2xl sm:text-3xl font-e
+           <p className="text-slate-400 mt-2 max-w-2x
+             Пишите небольшой код и запускайте его пр
+             который переводит CPython в WebAssembly.
+           </p>
+         </div>
+         <a
+           href="https://pyodide.org/"
+           target="_blank"
+           rel="noreferrer"
+           className="inline-flex items-center gap-1.5
+         >
+           Как это работает <ExternalLink className="
+         </a>
+       </div>
+
+       <div className="grid xl:grid-cols-[minmax(0,1.
+         <section className="rounded-2xl border borde

```

```

+         aria-label="Код Python"
+         className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+         placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+         Ввод для input() • по одной строке на
+     </label>
+     <textarea
+         id="python-stdin"
+         value={stdin}
+         onChange={(event) => setStdin(event.ta
+         spellCheck={false}
+         rows={2}
+         placeholder={'Например:\nАлиса'}
+         className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+     />
+     </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+     <div className="flex items-center justify-l
+     <div className="flex items-center gap-2
+     <Terminal className="w-4 h-4 text-emera
+     </div>
+     <span className={`inline-flex items-centr
+         {runtimeReady && <CheckCircle2 classNa
+         {runtimeReady ? "готов" : "не загружен
+     </span>
+     </div>
+     <pre className="min-h-[360px] max-h-[560px
px] leading-6 text-slate-300">
+         {output}
+     </pre>
+     <div className="border-t border-slate-800

```

Универсальное пособие • полный исходный код

```
-----  
// https://vite.dev/config/  
export default defineConfig({  
  // На GitHub Pages приложение живёт в /algov0/, локаль  
  // VITE_BASE можно переопределить, если репозиторий б  
  base: process.env.VITE_BASE || "/",  
  plugins: [react(), tailwindcss(), viteSingleFile()],  
  resolve: {  
    alias: {  
      "@": path.resolve(__dirname, "src"),  
    },  
  },  
  server: {  
    host: "0.0.0.0",  
    port: 5173,  
    strictPort: true,  
    // предпросмотр может открываться через внешний про  
    allowedHosts: true,  
  },  
  preview: {  
    host: "0.0.0.0",  
    port: 4173,  
    strictPort: true,  
    allowedHosts: true,  
  },  
});  
\\
```

Patch относительно `main`

```
```diff  
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
```

**\*\*строго\*\*** придерживаться готовых шаблонов разметки,

**##** Правила для малых страниц (Билетов / Глав)

1. **\*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML
2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна иметь объект `{ "id": "...", "content": ... }` для вставки в ``src/data/content.ts``.
3. **\*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``)
  - Обязательная сетка аналогий (2 колонки: неформальный текст и формальный текст)
  - "Душные" детали (код, формулы, сложные доказательства)
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы и не используй классы в файлах ``src/data/content.ts``. Не используй чистый markdown.

---

ФАЙЛ 4/92: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРOK: 14 | РАЗМЕР: 600 строк

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/img/icon.svg" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

---

ФАЙЛ 5/92: `metadata.json`

```
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 "svg-to-pdfkit": "^0.1.8",
 "tailwind-merge": "3.4.0"
 },
 "devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.24.0",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и структурам данных"
}
```

---

ФАЙЛ 7/92: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 125 | РАЗМЕР: 1.2 КБ

---

# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с симуляцией алгоритмов и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
```

Аудит контента

```
```bash
node scripts/audit-coverage.mjs # какие темы без
node scripts/audit-coverage.mjs --md # markdown-таблицы
```
```

Актуальный отчёт: [``TOPICS_AUDIT.md``](./TOPICS_AUDIT.md)

Как устроен интерфейс

- * **Темы 1–24** — канонический список билетов; одна страница (``src/data/content.ts``, таблица ``CANON`` задаёт точные темы)
- * **Python-компилятор** — боковая панель (кнопка «Python-отладчик». Список «переменных темы» для каждой страницы (``src/data/vizSync.ts``; значения из выполняемого Python-кода, объекты демонстрации (матрицы, графы, деревья) напрямую, эталонный код, инициализация подставляет заранее заданные значения)
- * **Содержание** — на десктопе боковая панель с поиском, на телефоне — липкая строка «Содержание · N из M» и вкладки
- * **Переходы между темами** — кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре — «←» / «→»
- * **Всплывающие подсказки по терминам** — текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симуляторы — ``src/data/simulators.ts``, Словарь — ``src/data/glossary.ts``, мини-анимации — ``src/data/mini.ts``)
- * **Реестр интерактивов** — ``src/components/vizRegistry.ts`` и для панели под главой, и для модалки из подсказки.

Структура

...

```
src/
  App.tsx          — оболочка, привязка глав к визуализации
  components/      — интерактивные симуляторы (Действия)
  data/            — контент пособия (главы/билеты)
```

Универсальное пособие • полный исходный код

Экспорт перехватывает основные структурные компоненты HTML:

1. Заголовки `

` и `` заменяются на `##` и `###`
2. Элементы `

.analogy-badge` получают выделение маркера
3. Теги `` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `- ` получают маркер `-`.
- 5. Результат извлекается в виде текста и чистится от лишнего.

ФАЙЛ 8/92: TOPICS_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.mjs`

> Критерии:

> * **Аналогия** — в HTML главы есть блок «Аналогия ...»

> * **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |
|-------|----|------------|
|-------|----|------------|

| | | |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

| | | |
|--------------------|-----------|-------------------------------|
| Алгоритмы на карте | `alg-map` | Страница-заглушка (800x400px) |
|--------------------|-----------|-------------------------------|

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

```
-----
| 3. Декартово дерево (Trear) | `trear` | Нет ни визуализации |
| 8. Графы. Компоненты связности | `graph-components` |
| 24. Классы сложности, сведение задач | `complexity-classes` |
-----
```

Сводная таблица по всем главам

| # | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|----|---|----------|---------|-----|----------|
| 2 | 2. Двумерная разреженная матрица (Sparse Table) | | | | |
| 3 | 3. Декартово дерево (Trear) | 2 | – | – | – |
| 4 | 4. Splay-дерево | 2 | | – | – |
| 5 | 5. Двумерная матрица префиксных сумм | 1 | | | – |
| – | Планарность: K5, K3,3 и формула Эйлера | – | | | – |
| 6 | 6. Графы. Представление, DFS, BFS | 4 | | | – |
| 8 | 8. Графы. Компоненты связности | 1 | – | – | – |
| 9 | 9. Графы. Топологическая сортировка | 1 | | | – |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | | | – |
| 12 | 12. Графы. Точки сочленения | 1 | | | – |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | | | – |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | | | – |
| 16 | 16. Графы. Кратчайший путь. Флойд | – | | | – |
| 17 | 17. Графы. Алгоритм Джонсона | – | | | – |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | | | – |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | | | – |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | | | – |
| 21 | 21. Префикс-функция (π), КМП | 4 | | | – |
| 22 | 22. Z-функция | 4 | | | – |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | | | – |
| 24 | 24. Классы сложности, сведение задач | 4 | – | – | – |
| – | Обзор: Кратчайшие пути и Графы | – | | | – |
| – | Алгоритмы на карте | – | – | – | – |
| – | Эйлер: Путь ≠ Цикл | – | | | – |
| – | Мосты в графах: код + визуализация | 1 | | | – |


```
    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "vite.config.ts"]
}
```

ФАЙЛ 10/92: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 36 | РАЗМЕР: 1

```
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algov0/, локальн
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
```

```

/** Боковая (на мобильном – нижняя) панель компилятора
const [compilerOpen, setCompilerOpen] = useState(false);
/** Ширина панели компилятора (desktop), растягивается
const [compilerWidth, setCompilerWidth] = useState<number>(
  const cached = typeof window !== "undefined" ? Number(window.innerWidth) : 0;
  return Number.isFinite(cached) && cached >= 380 ? cached : 380;
));
useEffect(() => {
  window.localStorage.setItem("compilerWidth", String(compilerWidth));
}, [compilerWidth]);
/** Высота терминала: null оставляет адаптивную высоту
const [compilerHeight, setCompilerHeight] = useState<number>(
  const cached = typeof window !== "undefined" ? Number(window.innerHeight) : 0;
  return Number.isFinite(cached) && cached >= 260 ? cached : 260;
));
useEffect(() => {
  if (compilerHeight !== null) window.localStorage.setItem("compilerHeight", String(compilerHeight));
}, [compilerHeight]);
/** Содержание свернуто (по умолчанию сворачиваем сам контент)
const [sidebarCollapsed, setSidebarCollapsed] = useState<boolean>(
  const cached = typeof window !== "undefined" ? window.localStorage.getItem("sidebarCollapsed") : null;
  if (cached !== null) return cached === "1";
  return typeof window !== "undefined" && window.innerWidth < 380;
));
const toggleSidebar = useCallback(() => {
  setSidebarCollapsed((c) => {
    window.localStorage.setItem("sidebarCollapsed", c ? "1" : null);
    return !c;
  });
}, []);

// Десктопный отступ контента = ширине панели компилятора
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.innerWidth < 380 ? compilerWidth : 0;
  };
  apply();
}, [compilerOpen, compilerWidth]);

```

```

window.addEventListener("keydown", onKey);
return () => window.removeEventListener("keydown",
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter

return (
  <div className="min-h-screen bg-slate-950 text-slate-
    <Navbar
      compilerOpen={compilerOpen}
      onToggleCompiler={() => setCompilerOpen((o) =>
    />

    {/* При открытой панели контент сдвигается: снизу
    <div ref={contentRef} className={compilerOpen ? "
      <
        <MobileToc
          selectedChapterId={activeChapterId}
          setSelectedChapterId={goTo}
          currentTitle={activeChapter.title}
          index={index}
          total={chapters.length}
        />

        <div className="flex flex-col lg:flex-row m
          {/* Сайдбар только на десктопе – на мобил
          <div
            className={`hidden lg:block shrink-0 bo
            tion-all duration-200 ${sidebarCollapsed ?
          >
            {!sidebarCollapsed && <Sidebar selected
          </div>
          {/* Вкладка-переключатель содержания у ле
          <button
            type="button"
            onClick={toggleSidebar}
            className="hidden lg:flex items-center
            l-0 border-slate-700 bg-slate-900/90 px-1 p

```

```
        />
      </main>
    </div>
  </>

  <footer className="p-8 text-center text-slate-600">
    © 2026 Universal Educational Guide. Интерактивное
  </footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
  compilerOpen && (
    <PythonCompiler
      chapterId={activeChapter.id}
      chapterTitle={activeChapter.title}
      width={compilerWidth}
      height={compilerHeight}
      onWidthChange={setCompilerWidth}
      onHeightChange={setCompilerHeight}
      onOpenGuide={() => {
        document.getElementById("chapter-viz")?.scrollIntoView()
      }}
      onClose={() => setCompilerOpen(false)}
    />
  )
  </div>
  );
}
```

ФАЙЛ 12/92: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 KB

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";
```

```

        const parent = (node as Text).parentElement;
        if (!parent) return NodeFilter.FILTER_REJECT;
        if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
        if (!node.nodeValue || node.nodeValue.trim().length === 0)
            return NodeFilter.FILTER_ACCEPT;
    },
    });

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
}

for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
        const surface = m[1].toLowerCase();
        const id = labelToId.get(surface);
        if (!id) continue;

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE)
            continue;
        perTerm.set(id, usedTerm + 1);
        perSurface.set(surface, usedSurface + 1);

        if (m.index > last) frag.appendChild(document.createTextNode(
            m[0].replace(/<span>|</span>/g, "")));
        last = m.index + m[0].length;
    }

    document.body.appendChild(frag);
}

```

```
// страховка: контент перерисовали, а подсказок в нём
useEffect(() => {
  const el = ref.current;
  if (!el) return;
  if (!el.querySelector(".term-hint")) run();
});
}
```

ФАЙЛ 13/92: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
@import "tailwindcss";
```

```
@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
    box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
  }
  .no-scrollbar::-webkit-scrollbar {
    display: none;
  }
  .no-scrollbar {
    scrollbar-width: none;
  }
}
```

```
@keyframes pulse-gold {
```

```
    animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
    cursor: help;
    border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
    color: #c7d2fe;
    border-radius: 3px;
    padding: 0 1px;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
    background: rgba(99, 102, 241, 0.22);
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
```

```
.chapter-body :where(.text-3xl) {
  font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
  font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
  font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
  from {
    width: 0%;
  }
  to {
    width: 100%;
  }
}
```



```
declare module '*.jpg' {
  const src: string;
  export default src;
}
declare module '*.jpg?url' {
  const src: string;
  export default src;
}
declare module '*.jpeg' {
  const src: string;
  export default src;
}
declare module '*.png' {
  const src: string;
  export default src;
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 17/92: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 817 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных",
    category: "Продвинутые структуры",
    content: `
```

```

<div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; padding: 10px; margin: 10px 0;">
  <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a2b3c; color: white; margin: 0;">
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C2 - (1 << ky)][R2 - (1 << kx) + 1][ky]);
int minimum = min(ans1, ans2);</pre>
  </div>
</details>
</div>
</div>
</section>`,
  }, {
    id: "treap",
    title: "3. Декартово дерево (Treap)",
    type: "html",
    description: "Дерево поиска по ключу и Куча по приоритету",
    category: "Продвинутые структуры",
    content: `
<section id="treap" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">3</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Декартово дерево (Treap)</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">Определение</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Декартово дерево — это бинарное дерево поиска, в котором каждый узел содержит пару (ключ, приоритет). Узлы упорядочены по ключу в левом поддереве и по приоритету в правом поддереве.</p>
        <p class="text-xl font-mono text-white">Пара (ключ, приоритет) хранится в узле. Ключ — это значение, по которому осуществляется поиск. Приоритет — это значение, которое определяет порядок вставки и удаления. Вставка и удаление выполняются с помощью операций вращения и перемещения. Декартово дерево можно использовать для хранения и поиска элементов. Оно поддерживает операции вставки, удаления и поиска за время  $O(\log n)$ .</p>
      </div>
      <p class="text-slate-300 text-sm mb-4">Построение декартова дерева можно выполнить за время  $O(n \log n)$  с помощью алгоритма, который платит  $\approx n \cdot \log n$  сравнений, counting по максимуму. Каждая точка вставляется спуском от корня:  $x \leq \text{уз. лев}$  — влево, иначе — вправо. Если уз. лев — единственный. Весь процесс — в симуляторе под названием «соединить сам»</b> — кликаешь две точки,

```

быстрее $n \cdot \log n$.

`<p class="text-slate-400 text-sm mb-1">`

`<p class="text-slate-400 text-sm">` • «Уб
аются merge за $O(h)$.

`</div>`

`<p class="text-slate-300 text-sm mt-4"><b c`
`"bg-slate-950 px-2 py-0.5 rounded text-xs f`
оение — `<code class="bg-slate-950 px-2 py-0`
`code>` (сортировка + n спусков). Худший случ
`text-xs font-mono text-rose-300 border bord`
`ed text-xs font-mono text-rose-300 border b`

`<p class="text-slate-500 text-xs mt-3 leadin`
и останется набор букв, как treap без y ост
но кричат по «!»: корень самый громкий, род
; равные x — идут влево по уставу.

`<details class="bg-slate-900/40 rounded-lg l`
`<summary class="p-4 font-bold text-slate`
`-b border-slate-700/50">`

Код: split (Скрыто)

`</summary>`

`<div class="p-5 text-sm text-slate-300`

`<pre class="bg-slate-950 p-4 rounde`
`pair<Node*, Node*> split(Node* t, int x) {`
`if (!t) return {nullptr, nullptr};`
`if (t->val <= x) {`
`auto [L, R] = split(t->right, x);`
`t->right = L;`
`return {t, R};`
`} else {`
`auto [L, R] = split(t->left, x);`
`t->left = R;`
`return {L, t};`
`}`
`</pre>`

`</div>`

```

{
  id: "splay-tree",
  title: "4. Splay-дерево",
  type: "html",
  description: "Самонастраивающееся дерево: поднимаем",
  category: "Продвинутые структуры",
  content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ротов поднимает x в корень. Единственный ин
      </div>

      <div class="grid grid-cols-1 md:grid-cols-3">
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-white font-bold">мож
          <p class="text-slate-400 text-xs mt">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-emerald-300 font-bold">
          <p class="text-slate-400 text-xs mt">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-white font-bold">0(n
          <p class="text-slate-400 text-xs mt">
        </div>
      </div>
    </div>
  </div>
`

```

омент «отстёжки», а врезка «Механика одного

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Смотрим
<div class="overflow-x-auto -mx-2 px-2">
  <table class="w-full text-xs sm:text-sm">
    <thead>
      <tr class="text-left text-slate-300">
        <th class="py-2 pr-3 font-bold">Имя
        <th class="py-2 pr-3 font-bold">Возраст
        <th class="py-2 pr-3 font-bold">Пол
        <th class="py-2 font-bold">Детали
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-500">
        <td class="py-3 pr-3 font-bold">Иван
        <td class="py-3 pr-3">деда
        <td class="py-3 pr-3">сам х
        <td class="py-3">х — корень
      </tr>
      <tr class="border-b border-slate-500">
        <td class="py-3 pr-3 font-bold">Петр
        <td class="py-3 pr-3">х и п
        <td class="py-3 pr-3"><span class="text-slate-500">
        <td class="py-3">ветка укор
      </tr>
      <tr>
        <td class="py-3 pr-3 font-bold">Алексей
        <td class="py-3 pr-3">х и п
        <td class="py-3 pr-3"><span class="text-slate-500">
        <td class="py-3">р и г стал
      </tr>
    </tbody>
  </table>
</div>
<p class="text-slate-300 text-sm mt-3">И по
у. Zig случается только последним.</p>
```

а под «горячие» ключи, дешёвые split/merge.
во меняется даже при чтении (плохо для много

```
<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #333; padding: 10px; margin: 10px 0;">  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50">
```

Код: rotate + splay + find (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300">
  <pre class="bg-slate-950 p-4 rounded" style="border: 1px solid #333; margin: 0 10px; font-family: monospace; font-size: 0.9em;"># узел: { key, left, right, parent }
```

```
def rotate_up(x):                                # x встаёт на место p
    p = x.parent
    g = p.parent
    if p.left is x:                               # правый поворот
        p.left = x.right
        if x.right: x.right.parent = p
        x.right = p
    else:                                         # левый (зеркально)
        p.right = x.left
        if x.left: x.left.parent = p
        x.left = p
    p.parent = x
    x.parent = g
    if g:
        if g.left is p: g.left = x
        else: g.right = x
```

```
def splay(x):                                    # поднять x в корень
    while x.parent:
        p, g = x.parent, x.parent.parent
        if g is None:                            # ZIG
            rotate_up(x)
        elif (g.left is p) == (p.left is x):    # ZIG-ZIG
            rotate_up(p)
            rotate_up(x)
        else:                                    # ZIG-ZAG: НИЖНЯЯ
```

```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2" >
  <div class="bg-slate-800 p-6 rounded-lg" >
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md" >
      ✖ Аналогия 1: Вырезание прямоугольника
    </div>
    <p class="text-slate-300 text-sm mb-2" >
      Из него маленький целевой прямоугольник в центре.
      Его верхний угол отрезается дважды! Поэтому
    </p>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-mt-12" >
  <div class="flex items-center mb-6 flex-wrap gap-3" >
    <span class="bg-blue-600 text-white px-4 py-1 rounded" >
      Теория
    </span>
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400" >
      7. Графы. Планарные. Покраска
    </h2>
  </div>
  <div class="space-y-8" >
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600" >
      <h3 class="text-xl font-bold text-blue-400" >
        Теорема о 4 красках
      </h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4" >
        <p class="text-lg text-blue-300 italic" >
          Любой планарный граф можно раскрасить 4 цветами.
        </p>
        <p class="text-xl font-mono text-white" >
          Например, граф можно раскрасить 4 цветами.
        </p>
      </div>
      <div class="grid grid-cols-1 gap-6" >
        <div class="bg-slate-800 p-6 rounded-lg" >

```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
        еская сортировка – это расписание, которое
        >
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
        Ограничения: Циклы
        </div>
        <p class="text-slate-300 text-sm mb-6">
        получить опыт, нужна работа". Алгоритм выяв
        </div>
        </div>
        </div>
        </div>
    </section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
        category: "Графы. Продвинутое",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-white">10. Графы. Компоненты сильной связности (SCC)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Макро-вершина – это компонент сильной связности (SCC).
        <p class="text-xl font-mono text-white">Макро-вершины инвертированы в порядке top-sort.</p>
      </div>
    </div>
  </div>
</section>`
    }
  ],
  meta: {
    title: "10. Графы. Компоненты сильной связности (SCC)",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
    category: "Графы. Продвинутое",
    id: "scc-kosaraju",
  },
}
```



```

        rder border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
        </div>
        <p class="text-slate-300 text-sm mb-4">
            Этот мост, экономикой обоих кусков придет
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        description: "Вершины, удаление которых убивает связность",
        category: "Графы. Продвинутые",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">12. Графы. Точки сочленения
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
            <h3 class="text-xl font-bold text-rose-400">Точки сочленения
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">Вершины, удаление которых убивает связность
                <p class="text-xl font-mono text-white">
                    теи > 1).</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        Аналогия 1: Центральный роутер
                    </div>
                    <p class="text-slate-300 text-sm mb-4">

```

```

    </div>
</section>`,
  },
  {
    id: "pathfinding-accel",
    title: "17. Графы. Ускорения поиска",
    type: "html",
    description: "",
    category: "Графы. Пути",
    content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">17. Графы. Пути
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Ускорения поиска
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Вспомогательная таблица
        <p class="text-xl font-mono text-white">
          d-длина.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Копаем туннель под землей.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Если копая туннель в 2 РАЗА меньше работы (геометрически: площадь
            </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },

```

```

    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          которое торчит из посещенных в непосещенных
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
            order border-emerald-500 shadow-md">
            ✦ Аналогия: Заражение вирусом
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ли вирус). Мы стартуем из одного города, и
            Сеть растет только из одного связного куска
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-boruvka",
    title: "20. Графы. Остовное дерево. Борувка",
    type: "html",
    description: "",
    category: "Графы. Остовные",
  },

```

```

    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
        <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-blue-300 italic">
            <p class="text-xl font-mono text-white">
i], который одновременно является её суффиксом.
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
            <!-- Аналогия 1 -->
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
                    Аналогия 1: Бумеранг (Префикс)
                </div>
                <p class="text-slate-300 text-sm mb-4">
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
мя перепроверок!</p>
            </div>
        </div>
        <details class="bg-slate-900/40 rounded-lg">
            <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>

```

```

        <summary class="p-4 font-bold text-slate-700 border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
            </div>
        </details>
    </div>
</div>
</section>`,
    },
    {
        id: "complexity-classes",
        title: "24. Классы сложности, сведение задач",
        type: "html",
        description: "",
        category: "Теория сложности",
        content: `
<section id="complexity-classes" class="mb-12 scroll-mt">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-purple-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-purple-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-purple-300 italic">
                <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
            </div>
        </div>
    </div>
    </section>
        `
    }
}
    
```

```

    content: string;
  }

export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1
  <h2 class="text-3xl font-bold text-white">He by
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400

    <div class="bg-slate-900 p-4 rounded-lg mb-
      <p class="text-lg text-blue-300 italic
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
        <p><strong class="text-white">Эйлер
        <div class="p-3 bg-slate-950 rounde
          <p class="text-sm text-blue-300
          <p><strong class="text-white">П
          <p><strong class="text-white">Ц
        </div>
      </div>
    </div>
  </div>
  <div>
    <p class="text-slate-300 text-sm">
      <strong>Билет 13 (Различие пути и цикла
      <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
      <br>• <span class="text-yellow-400 font
финиш).

```

```

<div class="p-5 text-sm text-slate-300 space-y-4">
  <p class="text-blue-400">Теорема Эйлера
  <p>
    Связный граф содержит эйлеров цикл
    <br>Эйлеров путь (не цикл)  $\Rightarrow$  ровно 2 вершины с нечётной степенью
  </p>
  <p class="text-rose-400">Почему Гамильтон?
  <p>
    Задача коммивояжёра (TSP) – это взвешенный граф.
    Доказано, что любой полиномиальный алгоритм решает задачу.
    В общем, не парься. Просто запомни.
  </p>
</div>
</details>
</div>
</section>`,
},
{
  id: "bridges-code",
  title: "Мосты в графах: код + визуализация",
  type: "html",
  content: `<section id="bridges-code" class="mb-20 space-y-4">
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-2 rounded">Код
  <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">Мосты в графах
  </div>

  <div class="bg-slate-900 p-4 rounded-lg mb-6">
    <p class="text-lg text-emerald-300 italic">Мосты в графах – это графы, в которых
    <div class="text-left font-mono text-sm">
      <p><strong class="text-white">tin[v] – количество мостов,
      <p><strong class="text-white">low[v] – количество мостов,
      <div class="p-3 bg-slate-950 rounded">
        <span class="text-rose-400 font`

```

</div>

```
<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50" data-bbox="200 143 1000 212">
  <summary class="p-4 font-bold text-slate-400" data-bbox="270 170 1000 212">
    order-slate-700/50">
```

Полный код на C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2" data-bbox="268 261 1000 331">
  <p>Классическая реализация – <code class="text-slate-400 font-mono">
    оходов.</p>
```

```
    <div class="bg-slate-950 p-4 rounded-lg" data-bbox="335 356 1000 403">
      <pre class="text-xs font-mono text-slate-400" data-bbox="405 383 1000 403">
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежных
```

```
vector<bool> visited;
```

```
vector<int> tin, low;
```

```
int timer;
```

```
void dfs(int v, int p = -1) {
```

```
    visited[v] = true;
```

```
    tin[v] = low[v] = timer++; // устанавливаем
```

```
    for (int to : adj[v]) {
```

```
        if (to == p) continue; // не идём назад
```

```
        if (visited[to]) {
```

```
            // обратное ребро: обновляем low
```

```
            low[v] = min(low[v], tin[to]);
```

```
        } else {
```

```
            // ребро дерева DFS
```

```
            dfs(to, v); // рекурсивный вызов
```

```
            low[v] = min(low[v], low[to]);
```



```
<h3 class="text-xl font-bold text-blue-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
<p class="text-lg text-blue-300 italic">
```

```
<p class="text-3xl font-mono text-white">
```

```
<div class="text-sm text-slate-400 space-x-4">
```

```
<p><span class="text-blue-400 font-mono">
```

```
<p><span class="text-emerald-400 font-mono">
```

```
<p><span class="text-rose-400 font-mono">
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
<strong>Следствие:</strong> Для планарного графа  $G$  с  $n$  вершинами и  $m$  ребрами:  
ode>.
```

```
Если ребер больше  $\frac{n(n-1)}{2}$  – граф гарантированно планарен.
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
```

```
<div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
```

```
<div class="absolute -top-3 left-4 bg-slate-900 p-2 rounded border border-slate-800 order-green-500 shadow-md">
```

```
К5: Полный граф на 5 вершинах
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

```
У К5: <code class="text-white">V=5, E=10 > 9</code> – БАХ, непланарен!
```

```
Все 5 вершин соединены со всеми. На рисунке – не так.
```

```
Теорема Куратовского: К5 – эталон непланарности.
```

```
Если внутри графа спрятан К5 – забудем про планарность.
```

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
```

```
<div class="absolute -top-3 left-4 bg-rose-500 p-2 rounded border-rose-500 shadow-md">
```

```
К3,3: 3 дома и 3 колодца
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-fuchsia-500 shadow-md">
```

Аналогия: карта на стене

```
</div>
```

```
<p class="text-slate-300 text-sm mb"
  Красим карту регионов: соседние
  Регион = вершина, общая граница
  Для плоской карты (планарный граф)
  теорема о четырёх красках (Аппеля)
```

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
er border-amber-500 shadow-md">
```

Что нужно знать на экзамен

```
</div>
```

```
<ul class="text-slate-300 text-sm sp"
  <li><strong class="text-white"></li>
вободный у соседей. Дал  $\leq \Delta + 1$  цветов ( $\Delta$  – макс. ст. верш. в графе)
  <li><strong class="text-white"></li>
тит 2 цветов, хоть он и непланарен.</li>
  <li><strong class="text-white"></li>
= n,  $\chi$ (цикла чётной длины) = 2,  $\chi$ (цикла нечётной длины) = 3
  <li><strong class="text-white"></li>
ый и не нечётный цикл,  $\chi(G) \leq \Delta$ .</li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-400 text-xs mt-4">
```

Почему раскраска «сложная»: проверить р-раскрасимость – NP-полная задача.

Даже «раскрасима ли карта в 3 цвета?» – NP-полная задача.

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соединены
      её перекресток (удаляют вершину) – районы будут
      <strong>г>хрупкость</strong> системы.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
    <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
      Телеком: Центральный свитч
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У тебя несколько компьютеров в одной
      абелем (мостом). Сами свитчи – это точки со
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400 font-bold">Особ
    <p>
      Для стартовой вершины обхода правил
      то выше неё прыгать некуда). Поэтому для не
      у него <strong>больше 1 независимого ребенка
    </p>
  </div>
</details>

```



```
});
```

```
// Дубликаты: берём последнюю (самую свежую) версию стр.  
const dedup = new Map<string, Chapter>();  
merged.forEach((c) => {  
  if (c.id === "alg-map") return; // страница-заглушка  
  if (stale.has(c.id)) return;  
  dedup.set(c.id, c);  
});
```

```
// Нормализуем названия по каноническому списку тем.  
const withCanon = Array.from(dedup.values()).map((c) => {  
  const canon = CANON[c.id];  
  if (canon) return { ...c, title: canon.title, category: canon.category };  
  return c;  
});
```

```
// Сначала вводная страница, затем билеты 1–24 по порядку  
const numbered = withCanon.filter((c) => CANON[c.id]);  
numbered.sort((a, b) => CANON[a.id].num - CANON[b.id].num);  
const extras = withCanon  
  .filter((c) => !CANON[c.id])  
  .sort((a, b) => a.title.localeCompare(b.title, "ru"))
```

```
export const chapters: Chapter[] = [...extras, ...numbered];
```

ФАЙЛ 20/92: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 184 | РАЗМЕР: 10.5 КБ

```
/**
```

```
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет  
 * под sys.settrace и на каждой строке записывает фрейм  
 * как панель Variables в обычном отладчике.  
 */
```

```

-----
    return `import sys as __sys, json as __json

__src = ${srcLiteral}
__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
    pass

def __safe(v):
    try:
        r = repr(v)
    except Exception:
        r = "<...>"
    return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
    """Небольшой JSON-снимок: UI использует его без парсера
    if v is None or isinstance(v, (bool, int, str)):
        return v
    if isinstance(v, float):
        return v if v == v and v not in (float("inf"), float("nan")) else None
    if depth >= 4:
        return __safe(v)
    limit = 80 if depth < 2 else 32
    if isinstance(v, (list, tuple)):
        return [__snapshot(x, depth + 1) for x in v]
    if isinstance(v, (set, frozenset)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, dict):
        # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
        # предел 80 незаметно обрезал уровни 4x4 и 8x8
        dict_limit = 192 if depth == 0 else limit
        return {str(k): __snapshot(x, depth + 1) for k, x in v.items()}
    # deque, heap-подобные и другие итерируемые учебные

```

```

        if event == "return" and frame.f_code.co_filename ==
            loc, values = __capture(frame)
            __final.update(loc)
            __final_values.update(values)
        return __tracer

__err = ""
try:
    __sys.settrace(__tracer)
    exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
    pass
except Exception:
    import traceback as __tb
    __err = __tb.format_exc(limit=3)
finally:
    __sys.settrace(None)

__OUT = __json.dumps({
    "steps": __steps,
    "finalLocals": __final,
    "finalValues": __final_values,
    "error": __err,
    "truncated": __was_truncated,
})
__OUT =
\;
}

/**
 * Какие переменные изменились на этом шаге относительно предыдущего
 * они подсвечиваются (как amber-подсветка изменений в коде)
 * Новые переменные тоже считаются изменёнными.
 */
export function changedVars(
    prev: Record<string, string> | null | undefined,
    cur: Record<string, string>
): Record<string, boolean> {

```

```
-----
* в подчёркнутый термин: при наведении (или тапе на мо
* с объяснением на пальцах и мини-визуализацией.
*/
export interface GlossaryEntry {
  id: string;
  /** Варианты написания в тексте (регистр не важен). */
  labels: string[];
  title: string;
  /** Объяснение «на пальцах» – 1–2 фразы. */
  short: string;
  /** Строгая формулировка / что важно не забыть. */
  formal?: string;
  complexity?: string;
  /** Какая мини-анимация показывается в карточке. */
  demo?: DemoKind;
  /** id полного симулятора (см. vizRegistry) – кнопка
  simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
  {
    id: "bfs",
    labels: ["BFS", "обход в ширину", "поиск в ширину",
    title: "BFS – обход в ширину",
    short:
      "Волна от источника: сначала все соседи, потом со
    formal:
      "Очередь (FIFO). Кладём старт, достаём вершину –
        у рёбер.",
    complexity: "O(V + E)",
    demo: "bfs",
    simulator: "queue-bfs",
  },
  {
    id: "dfs",
    labels: ["DFS", "обход в глубину", "поиск в глубину",
    title: "DFS – обход в глубину",
    short:
```



```

    demo: "stack",
    simulator: "stack-dfs",
},
{
    id: "queue",
    labels: ["очередь", "очереди", "очередью", "FIFO"],
    title: "Очередь (FIFO)",
    short: "Очередь в столовой: кто первым встал, тот поел",
    formal: "push_back / pop_front за  $O(1)$ . Основа BFS",
    demo: "queue",
    simulator: "queue-bfs",
},
{
    id: "dsu",
    labels: ["DSU", "СНМ", "система непересекающихся множеств"],
    title: "DSU — система непересекающихся множеств",
    short:
        "«Кто твой староста?» Каждый элемент знает своего старосту",
    formal:
        "find со сжатием путей + union по рангу/размеру. Обратная функция Аккермана",
    complexity: "почти  $O(1)$  — обратная функция Аккермана",
    demo: "dsu",
    simulator: "mst-kruskal",
},
{
    id: "trie",
    labels: ["бор", "бора", "боре", "бору", "бором", "борт"],
    title: "Бор (trie, префиксное дерево)",
    short:
        "Дерево, где путь от корня по буквам и есть слово",
    formal: "Из корня по символам; в вершине хранится ф. слово",
    complexity: "поиск слова —  $O(|\text{слова}|)$ ",
    demo: "trie",
    simulator: "aho-corasick",
},
{
    id: "bfs-on-trie",
    labels: [

```

```

        циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
    ,
    formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "префикс"],
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от начала до i»
    formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1].",
    complexity: "препроцессинг O(n), запрос O(1)",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "sparse-table",
    labels: [
        "разреженная таблица", "разреженная таблица", "sparse-table",
        "разреженная", "двоичные подъёмы", "двоичный подъём",
    ],
    title: "Разреженная таблица (метод двоичных подъёмов)",
    short:
        "Заранее считаем ответ для всех отрезков длины 1,

```

```

    },
    {
      id: "mst",
      labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
      ],
      title: "Минимальное остовное дерево (MST)",
      short:
        "Связать все точки проводами так, чтобы суммарная
      formal: "V-1 ребро, никаких циклов. Краскал (жадно и
        но).",
      demo: "mst",
      simulator: "mst-kruskal",
    },
    {
      id: "amortized",
      labels: [
        "амортизированная", "амортизированный", "амортизи
        "амортизированное", "амортизированного", "амортизи
      ],
      title: "Амортизированная сложность",
      short:
        "Средняя цена операции «по абонементу»: одна опер
      formal: "Метод потенциалов: дорогая операция оплачи
        ассив.",
      demo: "amortized",
    },
    {
      id: "np",
      labels: [
        "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
        "класс P", "полином", "полиномиальное",
      ],
      title: "Класс NP и NP-полнота",
      short:
        "NP — «решение проверить легко, найти трудно» (су
        ые в NP.",
    }
  ]
}

```



```

    formal:
      "Связный граф: эйлеров цикл   все степени чётные;
simulator: "euler-path-vs-cycle",
},
{
  id: "planarity",
  labels: ["планарный", "планарность", "планарного",
  title: "Планарность",
  short:
    "Граф планарен, если его можно нарисовать на листе
    ,
  formal:
    " $V - E + F = 2$  для связного планарного графа. Кур
    ,
  simulator: "planarity-euler-formula",
},
{
  id: "splay",
  labels: [
    "splay", "splay-дерево", "AVL", "AVL-дерево", "вс
  ],
  title: "Splay и повороты",
  short:
    "Каждый раз, когда к узлу обратились, его «вытягив
  formal:
    "Три случая: zig (родитель – корень), zig-zig (уз
    рацию.",
  demo: "amortized",
  simulator: "splay-tree",
},
{
  id: "adjacency",
  labels: ["матрица смежности", "список смежности", "
  title: "Способы хранить граф",
  short:
    "Матрица смежности – таблица «есть ли ребро», быс
    й, экономно для разреженных графов.",
  formal: "Матрица: проверка  $O(1)$ , память  $O(V^2)$ . Спис

```

```

    formal:
        "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядк
          ами.",
    complexity: "O(V³) времени, O(V²) памяти",
    demo: "floyd",
    simulator: "floyd",
},
{
    id: "prim",
    labels: ["Прима", "Прим", "Prim"],
    title: "Алгоритм Прима",
    short:
        "Остов растёт как плесень из одной точки: на кажды
    formal:
        "Множество посещённых + куча рёбер на границе. До
          усок.",
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-prim",
},
{
    id: "boruvka",
    labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
    title: "Алгоритм Борувки",
    short:
        "Все компоненты одновременно шлют «гонца» к ближай
    formal:
        "Фаза: для каждой компоненты ищем минимальное исх
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-boruvka",
},
{
    id: "kruskal",
    labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
    title: "Алгоритм Краскала",
    short:
        "Сортируем все рёбра по весу и берём подряд те, ч

```

```

    labels: ["сжатие координат", "сжатия координат", "сж",
    title: "Сжатие координат",
    short:
        "Координаты бывают гигантские и дробные, а нам ну
        рядковым номером.",
    formal:
        "sort + unique + binary search: значение → индекс
        ",
    complexity: "O(n log n)",
    demo: "coord-compress",
},
{
    id: "treap",
    labels: ["Treap", "декартово дерево", "декартова де",
    title: "Treap = Tree + Heap",
    short:
        "Одно дерево живёт по двум правилам сразу: по ключу
        ча. Случайность и держит его сбалансированным.",
    formal:
        "Базис — split(t, x) (разрезать по ключу) и merge
        я через них. Ожидаемая высота O(log n).",
    complexity: "ожидаемо O(log n)",
    demo: "heap",
},
{
    id: "bst",
    labels: ["бинарное дерево поиска", "дерево поиска",
    title: "Бинарное дерево поиска",
    short:
        "Слева всё меньше корня, справа — всё больше. Пои
    formal:
        "Без балансировки дерево может вырождаться в списо
    complexity: "O(высоты): от O(log n) до O(n)",
    demo: "segment-tree",
},
{
    id: "inclusion-exclusion",
    labels: [

```

```

    labels: ["строки", "строке", "строку", "строка", "с
    title: "Строка как массив символов",
    short:
        "Строковые алгоритмы держатся на трёх вопросах: г
        (Z-функция) и где встречаются слова словаря (бо
    formal:
        "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функци
    demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворо
    title: "Поворот (rotation) в дереве",
    short:
        "Локальная перевеска двух узлов: ребёнок встаёт н
        дившемуся месту. Порядок ключей при этом не лом
    formal:
        "Правый поворот вокруг p:  $x = p.left$ ;  $p.left = x$ .
        g и Zig-Zag.",
    complexity: "O(1)",
    demo: "zig",
    simulator: "splay-rotations",
},
{
    id: "zig",
    labels: ["Zig", "Zag", "зиг"],
    title: "Zig — одиночный поворот",
    short:
        "Самый простой случай: родитель x уже сидит в корн
        канчивается, если глубина была нечётной.",
    formal: "Применяется ровно один раз за всю операцию
    demo: "zig",
    simulator: "splay-rotations",
},
{
    id: "zig-zig",
    labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],

```



```

export const graphChapters: Chapter[] = [
  {
    id: "intro",
    title: "Обзор: Кратчайшие пути и Графы",
    type: "html",
    category: "Графы",
    content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1">
  <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
    <p class="text-slate-300 text-sm mb-4">Пред
    оезда). Алгоритмы поиска кратчайшего пути п
    <div class="mt-8 mb-4">
      <div id="slot-mnemonic-cards"></div>
    </div>
  </div>
</div>
</section>`
  },
  {
    id: "dijkstra",
    title: "Билет 14. Дейкстра",
    type: "html",
    category: "Графы",
    content: `<section id="dijkstra-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1">
  <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">

```

```

        <div id="slot-dijkstra-viz" class="mt-8"></div>
    </div>
</div>
</section>`
},
{
  id: "bellman-ford",
  title: "Билет 15. Форд-Беллман",
  type: "html",
  category: "Графы",
  content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Форд-Беллман
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-rose-300 italic">Параноик — это человек, который
      <p class="text-xl font-mono text-white">
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4
          order border-emerald-500 shadow-md">
          ♂ Аналогия 1: Параноик
        </div>
        <p class="text-slate-300 text-sm mb-4">Параноик — это человек, который
        берет ВСЕ возможные дороги V-1 раз подряд.
        <div id="slot-chapter-image-bellman"
        </div>

```

```

        <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px; margin-bottom: 10px;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
                Аналогия 1: Автомагистрали
            </div>
            <p class="text-slate-300 text-sm mb-4">
                Сколько времени долететь из Москвы в Париж, если мы сидим на самолете?
            </p>
            <div id="slot-chapter-image-floyd" class="h-40">
                <img alt="Placeholder for Floyd's algorithm diagram" data-bbox="100 260 400 350" style="width: 100%; height: 100%; object-fit: cover;"/>
            </div>

            <!-- Аналогия 2 -->
            <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px; margin-bottom: 10px;">
                <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
                    Время работы
                </div>
                <p class="text-slate-300 text-sm mb-4">
                    Три вложенных цикла. Сложность  $O(V^3)$ . Если  $V=1000$ , то  $10^9$  операций.
                </p>
            </div>

            <div id="slot-floyd-viz" class="mt-8"></div>
        </div>
    </div>
</section>`
},
{
    id: "aho-corasick",
    title: "Билет 23. Алгоритм Ахо-Корасик",
    type: "html",
    category: "Строки",
    content: `<section id="aho-corasick" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
        Алгоритм
    </span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
        Алгоритм Ахо-Корасик
    </h2>
</div>

<div class="space-y-8">
            `

```

```

pen:border-slate-700/50">
    Внутренняя структура автомата и код BFS (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
    <p>Каждая вершина имеет таблицу переходов <code>go</code> по
        ссылке <code>term_link</code> (ближайшая терминальная
    <div class="bg-black/50 p-4 rounded-lg font-mono">
struct Node {
    map<char, int> go;          // Предвычисленные переходы
    int link = 0;              // Суффиксная ссылка: куда перейти по
    int term_link = 0;         // Терминальная ссылка: мат. слово
    bool is_terminal = false;
    vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные ссылки
void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {
        nodes[child].link = 0;
        q.push(child);
    }

    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto const& [ch, child] : nodes[v].trie) {
            // Суффиксная ссылка ребёнка – это переход по
            nodes[child].link = nodes[nodes[v].link].go[ch];

            // Если суффиксная ссылка сама является словом
            // Иначе наследуем terminal_link
            if (nodes[nodes[child].link].is_terminal) {
                nodes[child].term_link = nodes[nodes[child].link].term_link;
            } else {
                nodes[child].term_link = nodes[nodes[child].link].term_link;
            }
        }
    }
}

```

```
export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
  explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
  "euler-path-vs-cycle": [
    {
      question: "В связном графе ровно 2 вершины с нечётными степенями. Какое утверждение верно?",
      options: [
        "Есть эйлеров цикл (вернусь домой!)",
        "Есть эйлеров путь (из одной нечётной в другую)",
        "Ни пути, ни цикла нет — это полный провал"
      ],
      correctIndex: 1,
      explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных вершин больше 2 — эйлеров цикла нет, и эйлерова пути тоже нет."
    },
    {
      question: "Почему гамильтонов цикл — это NP-полная задача?",
      options: [
        "Потому что Гамильтон — болезнь, и лечить её тяжело",
        "Потому что для эйлерова цикла есть простой критерий",
        "Потому что Эйлер был умнее Гамильтона"
      ],
      correctIndex: 1,
      explanation: "Для эйлерова цикла работает критерий нечётности степеней. Для гамильтонова цикла нет простого критерия. Это NP-полная задача, которую не известно, можно ли решить за полиномиальное время."
    },
    {
      question: "Куда ведёт эйлеров путь, если нечётных вершин ровно 2?",
      options: [
        "Всё ещё можно пройти по всем рёбрам ровно раз",
        "Всё ещё можно пройти по всем рёбрам ровно раз",
        "Всё ещё можно пройти по всем рёбрам ровно раз"
      ],
      correctIndex: 0,
      explanation: "Если нечётных вершин ровно 2, то существует эйлеров путь, который проходит по всем рёбрам ровно раз."
    }
  ]
```

```

        explanation: "DFS обходит каждую вершину и каждое
    }
],
"planarity-euler-formula": [
    {
        question: "Планарный граф имеет  $V=6$ ,  $E=12$ . Возможн
        options: [
            "Да, если это двудольный граф",
            "Нет, нарушается неравенство  $E \leq 3V - 6$ ",
            "Да, если нет треугольных граней"
        ],
        correctIndex: 1,
        explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$  – на грани пр
            Так что такой граф непланарен."
    },
    {
        question: "Сколько граней у планарного графа с  $V=7$ 
        options: [
            "3 грани",
            "5 граней",
            "10 граней"
        ],
        correctIndex: 1,
        explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$ . Не
    },
    {
        question: "Почему  $K_5$  непланарен?",
        options: [
            "Потому что  $10 > 3*5 - 6 = 9$  – нарушение",
            "Потому что Эйлер был против",
            "Потому что 5 вершин – несчастливое число"
        ],
        correctIndex: 0,
        explanation: "У  $K_5$ :  $V=5$ ,  $E=10$ ,  $3V - 6 = 9$ .  $10 > 9$ 
    }
]
};

```

```

chapterId: string,
snapshot: Omit<VizStateEventDetail, "chapterId">
) {
  if (typeof window === "undefined") return;
  const detail: VizStateEventDetail = { chapterId, ...snapshot };
  latestByChapter.set(chapterId, detail);
  window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>(
    "viz-state-event", { detail }
  ));
}

/** Вернуть демонстрацию в ручной режим при закрытии/перезагрузке */
export function clearVizState(chapterId: string) {
  if (typeof window === "undefined") return;
  latestByChapter.delete(chapterId);
  window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>(
    "viz-state-event", { detail: { chapterId } }
  ));
}

/** Текущий снимок компилятора для прямой отрисовки масива */
export function useVizRuntime(): VizStateEventDetail | null {
  const chapterId = useContext(VizChapterContext);
  const [runtime, setRuntime] = useState<VizStateEventDetail>(
    chapterId ? latestByChapter.get(chapterId) ?? null : null
  );

  useEffect(() => {
    setRuntime(chapterId ? latestByChapter.get(chapterId) : null);
    if (!chapterId) return;
    const handler = (e: Event) => {
      const detail = (e as CustomEvent<VizStateEventDetail>).detail;
      if (detail?.chapterId === chapterId) {
        setRuntime(detail.variables || detail.line !== null ? detail : null);
      }
    };
    window.addEventListener(CHANNEL, handler);
    return () => window.removeEventListener(CHANNEL, handler);
  }, [chapterId]);

  return runtime;
}

```

Универсальное пособие • полный исходный код

```
-----  
/** Утилиты без небезопасных cast-ов для визуализаторов  
export const vizNumber = {value: DebugValue | undefined  
    typeof value === "number" && Number.isFinite(value) ?  
  
export const vizString = {value: DebugValue | undefined  
    typeof value === "string" ? value : null;  
  
export const vizArray = {value: DebugValue | undefined  
    Array.isArray(value) ? value : null;  
  
export const vizRecord = {value: DebugValue | undefined  
    value !== null && typeof value === "object" && !Array  
        ? {value as Record<string, DebugValue>  
          : null;  
  
/* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)  
  
const DEMO_CHANNEL = "algo:viz-demo";  
const latestDemoByChapter = new Map<string, string>();  
  
export interface VizDemoEventDetail {  
    chapterId: string;  
    /** base-запись страницы – demoId пустой; подписанные  
    demoId: string;  
}  
  
/** Демонстрация сообщает, какая её вкладка сейчас открыта  
export function emitVizDemo(chapterId: string, demoId: string) {  
    if (typeof window === "undefined") return;  
    latestDemoByChapter.set(chapterId, demoId);  
    window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {  
        detail: {chapterId, demoId},  
    }));  
}  
  
/**  
 * Подписка: панель компилятора следует за активной вкладкой  
 * Сразу отдаём последнее значение: компилятор часто отстает  
 * как пользователь переключился с 1D на 2D.  
 */
```



```

vizTitle?: string;
/** Чему соответствует один шаг визуализации (для тул
stepNote?: string;
/** Переменные синхронизации. Пусто – на странице нет
variables: SyncVariable[];
/** Полный референсный Python-код, который кнопка-гла
code?: string;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
  "segment-trees": {
    vizTitle: "Дерево отрезков",
    stepNote: "Шаг = спуск по вершине v: либо читаем tree
    code: `n = 8                                # длина массива
tree = [0] * (2 * n)
i = 0                                           # лист: tree[n + i]

v, l, r = 1, 0, n - 1                        # вершина и её отрезок
m = (l + r) // 2                              # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
    variables: [
      { name: "n", role: "длина исходного массива; листы
      { name: "i", role: "индекс элемента массива; его
      { name: "v", role: "текущая вершина дерева отрезков
      { name: "l, r", role: "границы отрезка, за который
      { name: "m", role: "середина отрезка m = (l + r)
    ],
  },
  treap: {
    vizTitle: "Treap: плоскость → дерево → Split/Merge/
    stepNote: "Шаг = вставка очередной пары (x; y) или
    code: `pairs = [(45, 8), (3, 6), (6, 5), (2, 4), (9
pairs.sort(key=lambda p: -p[1])              # сортируем по y –
x, y = 6, 5                                  # подсвеченная пар
i = 0                                          # индекс шага: сор

def insert(root, x, y):                      # вставка спуском

```

```

    ],
},
"sparse-table": {
    vizTitle: "Разреженная таблица (1D и 2D)",
    stepNote: "Шаг = одна ячейка  $st[i][j] = \min(st[i][j - 1], st[i + (1 \ll (j - 1))][j])$ ",
    code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
    st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j])
    variables: [
        { name: "n", role: "длина массива / сторона квадрата", range: "1..1000", type: "int" },
        { name: "k", role: "уровень таблицы: ячейка хранит минимум из k предыдущих", range: "1..LOG", type: "int" },
        { name: "i", role: "начало блока в строке уровня", range: "0..n - (1 << j)", type: "int" },
        { name: "j", role: "номер уровня при построении (1..LOG)", range: "1..LOG", type: "int" },
        { name: "r, c", role: "строка и столбец ячейки в таблице", range: "0..n - 1", type: "int" },
        { name: "kx, ky", role: "уровни по вертикали и горизонтали", range: "1..LOG", type: "int" }
    ],
},
"prefix-sums-2d": {
    vizTitle: "Префиксные суммы (1D и 2D)",
    stepNote: "Шаг = заполнение очередного  $P[i] = P[i - 1] + a[i - 1]$ ",
    code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
    P.append(P[i - 1] + a[i - 1])
print("P =", P)`
    variables: [
        { name: "n", role: "число строк матрицы", range: "1..1000", type: "int" },
        { name: "m", role: "число столбцов матрицы", range: "1..1000", type: "int" },
        { name: "i", role: "индекс текущей строки (таблица строится по строкам)", range: "0..n - 1", type: "int" },
    ],
},

```

```

-----
parent = (i - 1) // 2      # родитель
kids = (2 * i + 1, 2 * i + 2)  # дети
print(f"i={i} parent={parent} kids={kids}")`,
    variables: [
        { name: "n", role: "текущее число элементов в куче"},
        { name: "i", role: "индекс элемента, который сейчас обрабатываем"},
        { name: "(i-1)//2", role: "индекс родителя вершины"},
        { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка"},
    ],
},
"stack-dfs": {
    vizTitle: "Стек и обход в глубину",
    stepNote: "Шаг = push новой вершины на стек или pop",
    code: `st = [] # стек = рекурсия
st.append("A") # push – зашли в вершину
st.append("B")
top = st.pop() # pop – возврат на уровень
print("pop →", top, "стек:", st)`,
    variables: [
        { name: "top", role: "вершина стека – то, откуда мы вышли"},
        { name: "v", role: "текущая вершина графа, которую мы обрабатываем"},
    ],
},
"queue-bfs": {
    vizTitle: "Очередь и обход в ширину",
    stepNote: "Шаг = dequeue вершины из головы очереди",
    code: `from collections import deque

q = deque(["A"])
q.append("B") # enqueue соседа
v = q.popleft() # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
    variables: [
        { name: "front", role: "голова очереди – вершина, которую мы обрабатываем"},
        { name: "v", role: "вершина, извлечённая из очереди"},
        { name: "u", role: "сосед вершины v, которого кладем в очередь"},
    ],
},

```

```

        for to in adj[v]:
            if to not in comp:
                comp[to] = comp[v]
                q.append(to)
print(comp, "компонент:", count)` ,
variables: [
    { name: "v", role: "текущая вершина обхода (метка",
    { name: "comp", role: "словарь или список: номер",
    { name: "q", role: "текущая очередь BFS", range:
    { name: "count", role: "сколько раз пришлось запу
],
},
"top-sort": {
    vizTitle: "Топологическая сортировка",
    stepNote: "Шаг = выход рекурсии из вершины v: она д
    code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2],
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:          # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)            # выход из v!

for v in adj:
    if v not in used:
        dfs(v)
print(order[::-1])            # разворот — ответ`,
variables: [
    { name: "v", role: "текущая вершина DFS", range:
    { name: "to", role: "вершина, куда ведёт ребро из",
    { name: "i", role: "перебор стартовых вершин во в
    { name: "order", role: "список выхода из рекурсии",
],
},
"scc-kosaraju": {
    vizTitle: "Компоненты сильной связности (Косарайю)"

```

```

        { name: "tin[v]", role: "время входа в вершину (п",
        { name: "low[v]", role: "минимальный tin, достижи",
        { name: "i", role: "перебор стартовых вершин для",
    ],
},
"bridges-code": {
    vizTitle: "Мосты: DFS + tin/low",
    stepNote: "Шаг = строка псевдокода слева; ребро (v,
    code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3                                # как на демо

v, to = "B", "G"                        # шаг: возврат из G
is_bridge = low[to] > tin[v]             # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
    variables: [
        { name: "v", role: "текущая вершина DFS", range:
        { name: "to", role: "сосед; ребро (v, to) проверя",
        { name: "tin[v]", role: "время входа в вершину –",
        { name: "low[v]", role: "минимум tin, куда можно",
        { name: "timer", role: "глобальный счётчик времени",
    ],
},
"euler-path-vs-cycle": {
    vizTitle: "Эйлеров путь и цикл",
    stepNote: "Шаг = один клик по следующей вершине мар",
    code: `edges = [{"A", "B"}, {"A", "C"}, {"B", "C"}, {"
path = ["A", "B", "C"]                  # маршрут: рисуется на граф
deg = {v: 0 for v in "ABCDE"}
for u, v in edges:                       # степени вершин
    deg[u] = deg.get(u, 0) + 1
    deg[v] = deg.get(v, 0) + 1

odd = []
for v in deg:
    if deg[v] % 2:
        odd.append(v)
print(deg, "нечётных:", len(odd))
# 0 нечётных → цикл · 2 → путь · иначе – нельзя`,

```

```

dist = {"A": 0}
prev = {}
done = set()
pq = [(0, "A")]
while pq:
    du, u = heapq.heappop(pq)
    if u in done:
        continue
    done.add(u)
    print(f"обрабатываем {u} (dist={du})")
    for v, w in g[u]:
        if v in done:
            continue
        print(f"смотрим ребро {u}→{v} (w={w})")
        if du + w < dist.get(v, float("inf")):
            dist[v] = du + w
            prev[v] = u
            heapq.heappush(pq, (dist[v], v))
print("итог:", dist),
variables: [
    { name: "u", role: "вершина с минимальным dist среди"},
    { name: "v", role: "сосед вершины u, до которого"},
    { name: "w", role: "вес ребра (u, v)", range: "по"},
    { name: "dist[v]", role: "лучшее известное расстояние"},
],
},
"bellman-ford": {
    vizTitle: "Форд–Беллман",
    stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
    code: `n, m = 7, 10 # вершин, рёбер
dist = {"S": 0}

for i in range(1, n): # итерация i = 1 ... n-1
    u, v, w = "S", "A", 5 # шаг: очередное ребро
    if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
        dist[v] = dist[u] + w # релаксация
    if i == 1:
        print(f"итерация {i}: {dist}")

```

```

-----
h = {"A": 0, "B": -3, "C": -1, "D": 1, "E": 2, "F": 4}

u, v, w = "A", "B", -3      # ребро с отрицательным вес
w_new = w + h[u] - h[v]      #  $w' = w + h[u] - h[v] \geq 0$ 
print(f"w'({u}→{v}) = {w_new}")
# после Дейкстры: ответ = d[v] - h[start] + h[v]`,
    variables: [
        { name: "h[v]", role: "потенциал вершины – кратчайшие расстояния на-Форда" },
        { name: "u, v", role: "ребро (u, v) перевзвешиваемое" },
        { name: "d[v]", role: "расстояние Дейкстры в новой вершине" },
    ],
},
"mst-kruskal": {
    vizTitle: "Краскал и DSU",
    stepNote: "Шаг = следующее ребро из отсортированных",
    code: `edges = [(3, "B", "C"), (1, "A", "B"), (2, "A", "C"),
edges.sort()                # по весу

parent = {}                  # DSU: корень вершины
for v in "ABC":
    parent[v] = v
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

mst = []
for w, u, v in edges:        # шаг: очередное ребро
    if find(u) != find(v):    # разные компоненты → берём
        parent[find(u)] = find(v)
        mst.append((w, u, v))
print(mst)`,
    variables: [
        { name: "n", role: "число вершин; в остов войдёт n-1 вершина" },
        { name: "m", role: "число рёбер, сортируем по весу" },
        { name: "i", role: "ребро сортируем по весу w; ра" },
    ],
},

```

```

    ],
},
"string-kmp": {
    vizTitle: "Префикс-функция (КМП)",
    stepNote: "Шаг = обработка символа s[i]: откаты по
    code: `s = "aabaabaaa"
n = len(s)
i, j = 1, 0
pi = [None] * n                # пустые клетки: значения п
pi[0] = 0

for i in range(1, n):          # шаг: считаем pi[i]
    j = pi[i - 1]               # длина совпадения
    while j > 0 and s[i] != s[j]:
        j = pi[j - 1]          # откат назад
    if s[i] == s[j]:
        j += 1
    pi[i] = j
print(pi)`,
    variables: [
        { name: "n", role: "длина строки s, для которой с
        { name: "i", role: "индекс текущего символа – счи
        { name: "j", role: "длина текущего наилучшего сов
        { name: "π[i]", role: "префикс-функция: длина наи
        ица" },
    ],
},
"string-z-func": {
    vizTitle: "Z-функция",
    stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l
    code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0               # правый Z-блок [l, r]
z = [None] * n                 # пустые клетки: значения п
z[0] = 0

for i in range(1, n):          # шаг: вычисляем z[i]
    if i <= r:

```



```

# сведение A ≤p B: стрелка между задачами
reductions = [{"3-SAT", "Vertex Cover"}]
for a, b in reductions:
    print(f"{a} ≤p {b}")`,
    variables: [
        { name: "cls", role: "какой класс подсветить: P /"},
        { name: "task", role: "какую задачу подсветить: 3"},
        { name: "reductions", role: "список пар задач – п"},
    ],
},
intro: {
    // мнемодкарточки без шагов алгоритма
    variables: [],
},
"alg-map": {
    variables: [],
},

/* — Дополнительные демонстрации внутри страниц (вкл

"sparse-table#2d-build": {
    vizTitle: "Разреженная таблица 2D: построение",
    stepNote: "Шаг = новый уровень k: блоки 2^k×2^k доп",
    code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67,
        88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [5
        1, 42, 85, 14]]
r, c, k = 0, 0, 0
st2 = {} # пустой каркас виден до

# --- тело алгоритма ---
# База не отдельный уровень вычислений: блок 1×1 – это
# Записываем её одним шагом, а не проигрываем 64 одноти
# Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(((r, c, 0, 0), A[r][c]) for r in range(4)

for k in range(1, 4):
    print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
    size = 9 - (1 << k)

```

```

-----
    S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1];

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
    variables: [
      { name: "i, j", role: "текущая ячейка таблицы S (
      { name: "S[i][j]", role: "сумма прямоугольника (0
        },
      { name: "r1, c1, r2, c2", role: "углы запроса; от
    ],
  },
};

/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null,
  (demoId && chapterId && PAGE_SYNC[`${chapterId}#${demo
  (chapterId && PAGE_SYNC[chapterId]) ||
  { variables: [] }];

/**
 * Стартовое содержимое редактора для страницы.
 *
 * Формат: короткий комментарий-шапка + исполняемый реф
 * Визуализация читает из его трассы знакомые имена (i,
 * но точно так же примет пользовательский код с теми же
 */
export function buildSyncTemplate(chapterId: string, ch
  const sync = getPageSync(chapterId, demoId);

  const head = sync.vizTitle
    ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .
    : `# «${chapterTitle}»\n# демо на странице нет – св

  if (!sync.code) return head;
  return `${head}\n${sync.code}\n`;
}

```

ФАЙЛ 26/92: src/data/tickets/ticket1_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

```
import { Chapter } from '../..types';
```

```
export const tickets1to5: Chapter[] = [
```

```
{
```

```
  id: "segment-trees",
```

```
  title: "1. Дерево отрезков с операциями на отрезках
```

```
  type: "html",
```

```
  description: "Дерево отрезков – структура данных дл
```

```
  category: "Оптимизации и Продвинутое структуры",
```

```
  content: `
```

```
<section id="segment-trees" class="mb-12 scroll-mt-10">
```

```
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
    <span class="bg-indigo-600 text-white px-4 py-1">
```

```
    <h2 class="text-2xl sm:text-3xl font-bold text-v
```

```
  </div>
```

```
  <div class="space-y-8">
```

```
    <div id="intro" class="bg-slate-700/50 p-6 rounded
```

```
      <h3 class="text-xl font-bold text-blue-400">
```

```
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
        <p class="text-lg text-blue-300 italic">
```

```
        <p class="text-xl font-mono text-white">
```

```
        promise (lazy).</p>
```

```
      </div>
```

```
      <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
        <div class="bg-slate-800 p-6 rounded-lg">
```

```
          <div class="absolute -top-3 left-4 l
```

```
          rder border-emerald-500 shadow-md">
```

```
            Аналогия 1: Корпоративная пре
```

```
          </div>
```

```

    },
    {
      id: "sparse-table",
      title: "2. Двумерная разреженная матрица (Sparse Table)",
      type: "html",
      description: "Разреженная таблица для идемпотентных операций",
      category: "Продвинутые структуры",
      content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-blue-300 italic">
              <p class="text-xl font-mono text-white">
                ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                  Аналогия 1: Школьные линейки
                </div>
                <p class="text-slate-300 text-sm mb-4">
                  зок длины 7. Ты берешь две заготовленные заготовки
                  зок длины 7 (перекрывая друг друга по 1 единицу)
                  ничего складывать, просто пересекаем куски.
                </div>
                <div class="bg-slate-900 p-6 rounded-lg">
                  <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                    Аналогия 2: Ступени
                  </div>

```

```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Удар катаной (Sp
    </div>
    <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
    </div>
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Аналогия 2: Слияние капель (M
    </div>
    <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300"
    <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
  if (!t) return {nullptr, nullptr};
  if (t->val <= x) {
    auto [L, R] = split(t->right, x);
    t->right = L;
    return {t, R};
  } else {
    auto [L, R] = split(t->left, x);
    t->left = R;
    return {L, t};
  }
}

```

```

        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Аналогия 2: Балансировка чере
        </div>
        <p class="text-slate-300 text-sm mb">
и становиться кривым как бамбук. Но как тол
        </p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg">
    <summary class="p-4 font-bold text-slate-
    -b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
// Zig, Zag
function rotate(x) {
    let p = x.parent;
    if (p.left === x) { // Right rotate
        p.left = x.right;
        x.right = p;
    } else { // Left rotate
        p.right = x.left;
        x.left = p;
    }
}</pre>
        </div>
    </details>

    <div class="bg-indigo-950/60 p-6 rounded-xl">
        <h4 class="text-lg font-bold text-indigo">

        <div class="space-y-4 text-sm text-slate">
            <div>
                <strong class="text-white block">
корень?</strong>
                <p>Оно поднимет узел, на котором
```

```
    <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
      <h3 class="text-xl font-bold text-emerald-400">
        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-emerald-300 italic">
            <p class="text-xl font-mono text-white">
              , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
            </div>
          <div class="grid grid-cols-1 xl:grid-cols-2">
            <div class="bg-slate-800 p-6 rounded-lg">
              <div class="absolute -top-3 left-4 l
              rder border-emerald-500 shadow-md">
                ≈ Аналогия 1: Вырезание прямоу
              </div>
              <p class="text-slate-300 text-sm mb-4">
                з него маленький целевой прямоугольник в цел
                ый верхний угол отрезается дважды! Поэтому
              </div>
            </div>
          </div>
        </div>
      </div>
    </section>`
  }
];
```

ФАЙЛ 27/92: src/data/tickets/ticket14_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

```
import { Chapter } from '../types';
```

```
export const tickets14to20: Chapter[] = [
```

```
  {
    id: "dijkstra",
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
    type: "html",
    category: "Графы. Пути",
```

```

{
  id: "bellman-ford",
  title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
  type: "html",
  category: "Графы. Пути",
  content: `
<section id="bellman-ford-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15. Графы. Поиск кратчайшего пути. Форд-Беллман
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-rose-400">
      <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана находит кратчайшие пути от заданного источника до всех остальных вершин графа. Он работает на графах с отрицательными весами ребер, но не содержит отрицательных циклов. Алгоритм работает за время  $O(V^2)$ , где  $V$  — количество вершин в графе.
        <p class="text-xl font-mono text-white">
          1. Инициализация: для каждой вершины  $v$  установить  $d[v] = \infty$ , кроме источника  $s$ , для которого  $d[s] = 0$ .
          2. Для каждой вершины  $u$  в графе:
            для каждой вершины  $v$  в графе:
              если  $d[u] + w(u, v) < d[v]$ , то  $d[v] = d[u] + w(u, v)$ 
          3. После завершения всех итераций  $d[v]$  будет содержать длину кратчайшего пути от  $s$  до  $v$ .
        </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            ♂ Аналогия 1: Параноик
          </div>
          <p class="text-slate-300 text-sm mb-4">Параноик считает, что все его соседи замышляют против него заговор. Поэтому он проверяет ВСЕ возможные дороги V-1 раз подряд. Это приводит к времени работы  $O(V^2)$ .
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            ♂ Отрицательный цикл
          </div>
          <p class="text-slate-300 text-sm mb-4">Если в графе есть отрицательный цикл, то алгоритм Форда-Беллмана не сможет найти кратчайшие пути, так как они будут уменьшаться бесконечно. В этом случае алгоритм выдаст ошибку.
        </div>
      </div>
    </div>
    <div id="slot-bellman-viz" class="mt-8"></div>
  </div>
</div>
`

```



```

        <li>...</li>
        <li><b>Шаг k=N:</b> Ты проверил все
    /li>
  </ul>
</div>
</div>
</div>
</section>`
  },
  {
    id: "johnson-algo",
    title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-amber-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-6">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6 mb-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
er border-amber-500 shadow-md">
            ⚡ Суть перевзвешивания
          </div>
          <p class="text-slate-300 text-sm mb-6">
            Дейкстра (Билет 14) быстрая, но
            ает очень медленно. Мы хотим сделать <b>Джон
            е пути остались теми же.
          </p>
          <ol class="text-sm text-slate-300 s

```

```

-----
        Аналогия: Прокладка интернет
        </div>
        <p class="text-slate-300 text-sm mb-6">
с самых дешевых дорог в стране". Он строит
        единую сеть.</p>
        </div>
    </div>
</div>
</section>`
    },
    {
        id: "mst-prima",
        title: "19. Графы. Остовное дерево. Прима",
        type: "html",
        category: "Графы. Остовные",
        content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-emerald-400">
                    <div class="bg-slate-900 p-4 rounded-lg mb-6">
                        <p class="text-lg text-emerald-300 italic">
                            <p class="text-xl font-mono text-white">
                                которое торчит из посещенных в непосещенных
                            </div>
                        <div class="grid grid-cols-1 gap-6">
                            <div class="bg-slate-800 p-6 rounded-lg">
                                <div class="absolute -top-3 left-4 border
                                rder border-emerald-500 shadow-md">
                                    ✦ Аналогия: Заражение вирусом
                                </div>
                                <p class="text-slate-300 text-sm mb-6">
ли вирус). Мы стартуем из одного города, и

```

```
    </div>
</section>`
  }
};
```

ФАЙЛ 28/92: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 262 | РАЗМ

```
import { Chapter } from '../..types';
```

```
export const tickets21to24: Chapter[] = [
  {
```

```
    id: "string-kmp",
    title: "21. Префикс-функция (π) — Взгляд назад (КМП)",
    type: "html",
    category: "Строки",
    content: `
```

```
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
        сом, оканчивающимся в позиции i.</p>
    </div>
```

```
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4">
```

```

        <li><b>abca</b>: 0! Начало ("a"
</code></li>
        <li><b>abcab</b>: Начало ("ab")
/!i>
        <li><b>abcabc</b>: Начало ("abc
e></li>
        <li><b>abcabcd</b>: "d" всё исп
"abca" vs "abcd"). Конец "abcd" не совпада
</ul>
    </div>
</details>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`
    },
    {
        id: "string-z-func",
        title: "22. Z-функция — Сдвиг и наложение",
        type: "html",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">

```

```

        </p>
    </div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Пример вычисления (Скрыто)

```

    </summary>
    <div class="p-5 text-sm text-slate-300
        <p>Тот же пример: <b>abcabcd</b></p>
        <ul class="list-disc pl-5 space-y-2
            <li><code>Z[0]</code> – не счита
            <li><code>Z[1]</code> (<b>bcabcd
[1] = 0</code></li>
            <li><code>Z[2]</code> (<b>cabcd
</li>
            <li><code>Z[3]</code> (<b>abcd<
? Нет, в начале 'a', тут 'd'). Совпало 3 бу
            <li><code>Z[4], Z[5], Z[6]</code>
        </ul>
    </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Код C++ (Скрыто)

```

    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
int l = 0, r = 0;
for (int i = 1; i &lt; n; i++) {
    if (i &lt;= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] &lt; n && s[z[i]] == s[i + z[i]]) z
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
    </div>
</details>

```

```

    <div class="bg-slate-900 p-6 rounded-lg border-2"
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
      </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border b
      <summary class="p-4 font-bold text-slate-400 ou
        r-slate-700/50">
        Код (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300 space-y-
        <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch)
  }
  return t[v].link;
}

int go(int v, char c) {
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v),
  }
  return t[v].go[c];
}</pre>
    </div>
  </details>
</div>
</section>`
},

```

```

        аче В, то В – это <b>NP-Полная</b> (сосредо
        </div>
    </div>
</div>
</section>`
    }
};
```

ФАЙЛ 29/92: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМ

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
    {
        id: "graph-dfs-bfs",
        title: "6. Графы. Представление, DFS, BFS",
        type: "html",
        description: "Представление графов и базовые обходы",
        category: "Графы. База",
        content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">6
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-slate-300">Представление
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-slate-300 italic">Графы — это абстрактная модель
        <p class="text-xl font-mono text-white">
          (V + E).</p>
      </div>
    </div>
  </div>
</section>`
    }
];
```

```
<li><b>Где используется:</b></li>
<li>Топологическая сортировка (l
<li>Поиск мостов и точек сочленен
<li>Сильно связанные компоненты К
<li>Поиск циклов и просто провер
</ul>
</div>

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
  BFS (Поиск в ширину) = Волна
</div>
<p class="text-slate-300 text-sm mb-4">
  <b>Что делает:</b> Концентрические волны
  Работает через Очередь (FIFO). Продвигается
  </p>
<ul class="text-xs text-emerald-300">
  <li><b>Где используется:</b></li>
  <li>Кратчайший путь в Невзвешенном графе
  <li>Алгоритм Ахо-Корасик (сборка)
  <li>Алгоритм Форда-Фалкерсона /
  <li>Двунаправленный поиск для у
  </ul>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50">
      Код обходов C++ (Основа)
  </summary>
  <div class="grid grid-cols-1 md:grid-cols-2">
    <pre class="bg-slate-950 p-3 rounded">
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;
```



```

        <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
            <h3 class="text-xl font-bold text-blue-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-blue-300 italic">
                        <p class="text-xl font-mono text-white">
                            ый граф можно раскрасить 4 цветами.</p>
                </div>
                <div class="grid grid-cols-1 gap-6">
                    <div class="bg-slate-800 p-6 rounded-lg">
                        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                            Аналогия 1: Карта мира
                        </div>
                        <p class="text-slate-300 text-sm mb-4">
                            . Границы не пересекаются в одной точке по-
                            ы не сливались цветами, Всегда можно всего
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "graph-components",
        title: "8. Графы. Компоненты связности",
        type: "html",
        description: "Нахождение кусков графа",
        category: "Графы. База",
        content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
        </div>
    </div>
    <div class="space-y-8">

```

```
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
перед.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
        </div>
        <p class="text-slate-300 text-sm mb
еская сортировка – это расписание, которое
>
    </div>
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
            Ограничения: Циклы
        </div>
        <p class="text-slate-300 text-sm mb
получить опыт, нужна работа". Алгоритм выяв
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Душный мод: Код на C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <p class="text-indigo-400 font-bold">
        <div class="bg-slate-950 p-4 rounde
            <pre class="text-xs font-mono t
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-emerald-300 italic">
    <div class="text-left font-mono text-sm">
      <p><strong class="text-white">Компоненты
и <i>ориентированного</i> графа, в котором
    </div>
  </div>
```

```
<p class="text-slate-300 text-sm mb-4">
  <strong>«Это просто цикл?» – Нет!</strong>
</p>
<ul class="list-disc list-inside text-sm text-slate-300">
  <li>Если у тебя есть цикл <code>A → B → A</code>
  <li>Если ты добавишь вершину <strong>Г</strong>
  <li>Правило: <b>Если два цикла имеют хотя бы
  </ul>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия: Интриги в офисе
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Сильная связность – это "клуб сильных"
      ни в одной SCC. Добавить курьера Гошу, который
      йти наружу (к директору), но обратно в клуб
    </p>
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg">
  <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
    Связь с Билетом 12
  </div>
  <p class="text-slate-300 text-sm mb-4">
```

```

        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-rose-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">
                <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
                        Аналогия 1: Единственный мост
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        description: "Вершины, удаление которых убивает свя
category: "Графы. Продвинутое",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>

```

```

        <div class="absolute -top-3 left-4 bg-emerald-500 border-emerald-500 shadow-md">
            Телеком: Центральный свитч
        </div>
        <p class="text-slate-300 text-sm mb-4">
            У тебя несколько компьютеров в одной
            абелем (мостом). Сами свитчи – это точки со
        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border-1" >
    <summary class="p-4 font-bold text-slate-400" >
        Душный мод: Код и корень DFS (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-2">
        <p class="text-blue-400 font-bold">Особый
        <p>
            Для стартовой вершины обхода правил
            то выше неё прыгать некуда). Поэтому для не
            у него <strong>больше 1 независимого ребенка
        </p>
        <div class="bg-slate-950 p-4 rounded-lg">
            <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)

```

```
-----
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
            order border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрывая
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Если сходится нечетное число линий, значит, из
            а везде должно быть четное число (зашел-вышел)
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
];
```

=====

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

=====

```
// Adjacency list
const adj: number[][] = Array.from(
  { length: Object.keys(nodes).length },
  () => [],
);
edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  useVizStepSync(currentStepIndex, setCurrentStepIndex,
    const v = vizNumber(vars.v);
    if (v === null) return null;
    return steps.map((step, index) => ({ step, index }));
  });
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();
    const currentTin = Array(nodes.length).fill(-1);
    const currentLow = Array(nodes.length).fill(-1);

    const recordStep = (desc: string, u: number | null)
```

```

        if (currentLow[to] >= currentTin[v] && p !==
            ap.add(v);
            recordStep(
                `Найдена точка сочленения: вершина ${v}
                v,
            );
        }
    }
}

if (p === -1 && children > 1) {
    ap.add(v);
    recordStep(
        `Корень дерева DFS ${v} имеет >1 детей. Это
        v,
    );
} else if (p === -1) {
    recordStep(
        `Корень дерева DFS ${v} имеет <=1 ребенка, он
        v,
    );
}
};

for (let i = 0; i < nodes.length; i++) {
    if (!visited.has(i)) {
        dfs(i);
    }
}

recordStep("Алгоритм завершен. Найдены все точки сочленения");
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
    desc: "",
    visited: new Set(),
    ap: new Set(),

```



```

        <Pause className="w-4 h-4 ml-1" />
      ) : {
        <Play className="w-4 h-4 ml-1" />
      }}
      {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
      onClick={reset}
      className="flex items-center justify-center
      ors"
    >
      <RotateCcw className="w-4 h-4" />
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 g
  <div className="lg:col-span-2 relative h-96 lg:l
    <svg className="w-full h-full" viewBox="0 0 8
      { /* Edges */ }
      {edges.map((edge, i) => {
        const u = nodes.find((n) => n.id === edge
        const v = nodes.find((n) => n.id === edge

      const isBridge =
        (edge.u === 3 && edge.v === 4) ||
        (edge.u === 4 && edge.v === 3);

      return (
        <line
          key={"edge-" + i}
          x1={u.x}
          y1={u.y}
          x2={v.x}
          y2={v.y}
          stroke={isBridge ? "#f43f5e" : "#475568"}
          strokeWidth={isBridge ? "4" : "2"}

```

```

<text
  x={node.x}
  y={node.y}
  dy=".3em"
  textAnchor="middle"
  fill="white"
  fontSize="14px"
  fontWeight="bold"
  className="select-none pointer-events"
>
  {node.label}
</text>

```

```

{isVisited && (
  <>
    <text
      x={node.x}
      y={node.y - r - 15}
      textAnchor="middle"
      fill="#94a3b8"
      fontSize="10px"
    >
      tin:{" "}
      {currentStep.tin[node.id] >= 0
        ? currentStep.tin[node.id]
        : "-"}
    </text>
    <text
      x={node.x}
      y={node.y + r + 20}
      textAnchor="middle"
      fill="#94a3b8"
      fontSize="10px"
    >
      low:{" "}
      {currentStep.low[node.id] >= 0
        ? currentStep.low[node.id]
        : "-"}
    </text>

```

```

        </div>
      </div>

      <div>
        <h5 className="text-xs text-slate-500 font-medium">
          ЛОГ ДЕЙСТВИЙ:
        </h5>
        <div className="space-y-2">
          {steps
            .slice(0, currentStepIndex + 1)
            .reverse()
            .map((step, idx) => {
              <div
                key={idx}
                className={`text-xs p-2 rounded $
xt-slate-500`} `}
              >
                {step.desc}
              </div>
            })}
        </div>
      </div>
    </div>
    <div className="mt-4 shrink-0">
      <div className="w-full bg-slate-800 h-2 rounded">
        <div
          className="bg-rose-500 h-full transition"
          style={{
            width: `${(currentStepIndex / Math.max(
              steps.length, 1
            )) * 100}%`
          }}
        />
      </div>
    </div>
  </div>
</div>
</div>
</div>
);
};

```

```

const getEdges = (cycle: boolean): Edge[] => {
  const defaultEdges = [
    { u: 'S', v: 'A', w: 4 },
    { u: 'S', v: 'B', w: 5 },
    { u: 'B', v: 'C', w: -2 },
    { u: 'A', v: 'C', w: 1 },
    { u: 'C', v: 'D', w: 3 },
    { u: 'C', v: 'E', w: 4 },
    { u: 'D', v: 'F', w: 2 },
    { u: 'E', v: 'F', w: 1 },
  ];
  if (cycle) {
    return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
  } else {
    return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
  }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
  const list: { [key: string]: { v: string; w: number } } = {};
  nodes.forEach(n => (list[n.id] = []));
  ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
  return list;
};

const adjList = getAdjList(edges);

const codeLines = [
  { line: 1, text: 'function bellman_ford(graph, start)' },
  { line: 2, text: '  dist = {v: ∞}; dist[start] = 0;' },
  { line: 3, text: '  for i from 1 to V - 1: // |V| - 1 iterations' },
  { line: 4, text: '    for (u, v, weight) in E:' },
  { line: 5, text: '      if dist[u] + weight < dist[v]' },
  { line: 6, text: '        dist[v] = dist[u] + weight;' },
  { line: 7, text: '  for (u, v, weight) in E: // Π' },
  { line: 8, text: '    if dist[u] + weight < dist[v]' }
];

```

```

        });
    }

    if (!anyUpdate && iter < vCount - 1) {
        simulationSteps.push({
            iteration: iter, checkingEdge: null, distance:
            log: ` Ранний выход после итерации ${iter}.
            activeCodeLine: 10
        });
        break;
    }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
    let cycleFound = false;
    for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
            cycleFound = true;
            simulationSteps.push({
                iteration: vCount, checkingEdge: { u: edge.u, v: edge.v, w: edge.w },
                distances: { ...dist }, previous: { ...prev },
                log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v} с весом ${edge.w}.
                activeCodeLine: 9,
            });
            break;
        }
    }

    if (!cycleFound) {
        simulationSteps.push({
            iteration: vCount, checkingEdge: null,
            distances: { ...dist }, previous: { ...prev },
            log: ` Проверка завершена. Ни одно ребро не вызвало цикла.
            activeCodeLine: 10,
        });
    }
}
}

```

```

    <div className="p-2.5 bg-blue-600 text-white"
      <span className="text-2xl"> </span>
    </div>
    <div>
      <h3 className="text-2xl font-bold text-white">
        <p className="text-sm text-blue-300">Полный
      </p>
    </div>
  </div>

  <div className="flex items-center gap-3">
    <div className="bg-slate-900 p-1 rounded-lg b
      <button onClick={() => setHasCycle(false)}
        bg-blue-600 text-white' : 'text-slate-400'
      <button onClick={() => setHasCycle(true)} c
        r gap-1 ${hasCycle ? 'bg-rose-600 text-white
    </div>
    <button onClick={() => { setIsPlaying(false);
      g-slate-600 text-white px-3 py-2 rounded-lg
    <button onClick={() => setIsPlaying(!isPlaying
      sition-colors text-white ${isPlaying ? 'bg-
    <button onClick={() => { setIsPlaying(false);
      ; }} className="flex items-center gap-1 bg-l
      s">⏮ar    </button>
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 m
    {/* Граф */}
    <div className="w-full lg:w-2/3 bg-blue-950/20
      y-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-blue
        ld shadow-sm">
        Итерация {currentStep.iteration} (⏮ar {curr
      </div>

      {currentStep.hasNegativeCycle && (
        <div className="absolute top-3 right-3 bg-r
          animate-pulse shadow-lg shadow-rose-600/50

```

```

    });
  }}}

  {nodes.map((node) => {
    const dist = currentStep.distances[node.id];
    const isCheckingNode = currentStep.checkingNode ===
      node.id);

    return (
      <g key={node.id} className="transition-...
        <circle cx={node.x} cy={node.y} r={isC
        ckingNode ? '#7dd3fc' : '#64748b'} strokeWid
        <text x={node.x} y={node.y + 5} fill=
        <text x={node.x} y={node.y - 28} fill=
        ight="bold" textAnchor="middle" className="
        <text x={node.x} y={node.y + 35} fill=
        split(' ')[0]}</text>
      </g>
    );
  }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
  <p className="font-semibold text-amber-400 m
  <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
  <h4 className="text-lg font-bold text-emerald
  <div className="space-y-2 text-sm font-mono t
    {Object.keys(adjList).map(u => {
      <div key={u} className="flex flex-wrap g
      -colors">
        <span className="font-bold text-white
        {adjList[u].map((edge, idx) => {
          const isChk = currentStep.checkingE
```

```
-----
    <h4 className="text-lg font-bold text-slate-600">
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map(item => {
        <div key={item.line} className={`py-1.5 px-4
e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-600'}`>
          <span className="text-slate-600 w-5 flex-shrink-0">{item.line}</span>
          <span className="whitespace-pre flex-grow-1">{item.code}</span>
        </div>
      })}
    </div>
  </div>
</div>
</div>
</div>
);
}
```

ФАЙЛ 32/92: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
-----
import { useState, useEffect, useCallback } from 'react';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';
```

```
const NODES = [
  { id: 0, label: '0', x: 300, y: 50 },
  { id: 1, label: '1', x: 150, y: 150 },
  { id: 2, label: '2', x: 450, y: 150 },
  { id: 3, label: '3', x: 75, y: 250 },
  { id: 4, label: '4', x: 225, y: 250 },
  { id: 5, label: '5', x: 375, y: 250 },
  { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
  { from: 0, to: 1 }, { from: 0, to: 2 },
  { from: 1, to: 3 }, { from: 1, to: 4 },
  { from: 2, to: 5 }, { from: 2, to: 6 },
  { from: 3, to: 4 }, { from: 4, to: 5 },
  { from: 5, to: 6 },
];
```



```
});
```

```
while (queue.length > 0) {
  steps.push({
    activeLine: 4,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: `Очередь не пуста, элементов: ${queue.length}`
  });
```

```
const curr = queue.shift()!;
steps.push({
  activeLine: 5,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: curr,
  logs: `Достаем из очереди (${curr}).`
});
```

```
const neighbors = ADJ[curr];
if (neighbors.length > 0) {
  for (const n of neighbors) {
    if (!visited.has(n)) {
      steps.push({
        activeLine: 7,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Рассматриваем соседа: ${n}. Он еще не`
      });
```

```
      visited.add(n);
      queue.push(n);
```

```
      steps.push({
        activeLine: 9,
        queue: [...queue],
```

```

    }, []));

    // @ts-expect-error зарезервировано под кнопку «назад»
    const handlePrev = useCallback(() => {
      setStepIdx(prev => Math.max(prev - 1, 0));
    }, []);

    useEffect(() => {
      if (!isPlaying) return;
      if (stepIdx >= STEPS.length - 1) {
        setIsPlaying(false);
        return;
      }
      const timer = setTimeout(handleNext, 1200);
      return () => clearTimeout(timer);
    }, [isPlaying, stepIdx, handleNext]);

    return (
      <div className="bg-slate-900 border-2 border-slate-700 p-4">
        <div className="flex flex-col md:flex-row items-center justify-between">
          <div>
            <h3 className="text-2xl font-bold text-white">
              <span className="text-blue-400"> Анонс </span> Албума
            </h3>
            <p className="text-slate-400 text-sm mt-1">Видео-презентация
            </p>
          </div>
          <div className="flex bg-slate-800 p-1.5 rounded">
            <button
              onClick={() => { setIsPlaying(false); setStepIdx(prev => prev - 1); }}
              className="p-2 text-slate-400 hover:text-white"
              title="C6poc"
            >
              <RotateCcw className="w-5 h-5" />
            </button>
            <button
              onClick={() => setIsPlaying(p => !p)}
              className="p-2 text-blue-400 hover:text-blue-300"
              title={isPlaying ? 'Пауза' : 'Воспроизведение'}
            >
              <RotateCcw className="w-5 h-5" />
            </button>
          </div>
        </div>
      </div>
    );
  }
}

```

```

const isInQueue = step.queue.includes(n.id)
const isVisited = step.visited.includes(n.id)

let fill = '#1e293b'; // slate-800
let stroke = '#475569'; // slate-600
let scale = 1;
void scale;

if (isCurrent) {
  fill = '#3b82f6'; // blue-500
  stroke = '#60a5fa'; // blue-400
  scale = 1.3;
} else if (isInQueue) {
  fill = '#059669'; // emerald-600
  stroke = '#34d399'; // emerald-400
  scale = 1.1;
} else if (isVisited) {
  fill = '#0f172a'; // slate-900
  stroke = '#3b82f6'; // blue-500
}

return (
  <g key={n.id} className="transition-tran
    {isCurrent && {
      <circle cx={n.x} cy={n.y} r="30" fi
    }}
    <circle
      cx={n.x} cy={n.y}
      r="20"
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-colors duration
  />
  <text
    x={n.x} y={n.y}
    textAnchor="middle"
    dy=".3em"

```

```

        });
      })}
    </div>
  </div>

  <div className="bg-slate-800/80 border border-
    <div className="text-sm font-bold text-eme
      <span>Состояние Очереди (Queue):</span>
      <span className="text-xs text-slate-500
    </div>
    <div className="flex items-center gap-2 mi
      {step.queue.length === 0 ? {
        <span className="text-slate-500 text-x
      } : {
        step.queue.map(({qId, i}) => {
          <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
          {qId}
          </span>
        })
      }}
    </div>
    <div className="mt-2 text-sm text-amber-30
ms-center justify-center">
      "{step.logs}"
    </div>
  </div>
</div>
</div>
</div>
);
}

```

```

        <p className={`text-center text-xs mt-2 font-bold`>
    </div>
    );
}

```

ФАЙЛ 34/92: `src/components/ChapterNav.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";

interface Props {
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Codecademy
 * Компактный вариант живёт в шапке главы, крупный — по центру
 */
export const ChapterNav: React.FC<Props> = ({ prev, next, onGo, compact }) => {
  if (compact) {
    return (
      <div className="flex items-center gap-2 text-xs">
        <button
          disabled={!prev}
          onClick={() => prev && onGo(prev.id)}
          title={prev ? prev.title : "Это первая тема"}
          className="flex items-center gap-1 px-2.5 py-1.5 rounded-md
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
          <ChevronLeft />
        
        <button
          disabled={!next}
          onClick={() => next && onGo(next.id)}
          title={next ? next.title : "Это последняя тема"}
          className="flex items-center gap-1 px-2.5 py-1.5 rounded-md
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
          <ChevronRight />
        
      
    );
  }
  return (
    <div className="flex items-center justify-center gap-4 text-sm">
      <button
        disabled={!prev}
        onClick={() => prev && onGo(prev.id)}
        title={prev ? prev.title : "Это первая тема"}
        className="flex items-center gap-1 px-3 py-2 rounded-md
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
        <ChevronLeft />
      
      <div>
        <span>Глава {index + 1}</span>
        <span> из <span>{total}</span>
      
      <button
        disabled={!next}
        onClick={() => next && onGo(next.id)}
        title={next ? next.title : "Это последняя тема"}
        className="flex items-center gap-1 px-3 py-2 rounded-md
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
        <ChevronRight />
      
    
  );
};
```

```

    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg
          transition-all sm:col-start-2"
      >
        <div className="flex items-center justify-end
          text-indigo-400 mb-1">
          Следующая тема <ChevronRight className="w-3
        </div>
        <div className="text-sm font-bold text-slate-
      </button>
    )}
  </nav>
);
};

```

ФАЙЛ 35/92: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, Loader2, Maximize2, MousePointer } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml";

```

```

interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
}

```

```

        await downloadChapterHtml(chapter);
        setSaving("done");
        window.setTimeout(() => setSaving("idle"), 2500);
    } catch {
        setSaving("idle");
    }
}, [chapter]));

const open = useCallback(
    (el: HTMLElement) => {
        const termId = el.dataset.termId;
        if (!termId) return;
        cancelHide();
        setAnchor({ termId, rect: el.getBoundingClientRect() });
    },
    [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
    if (isTouch()) return;
    const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
    if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
    if (isTouch()) return;
    const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
    if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
    const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
    if (!el) return;
    e.preventDefault();
    if (anchor && anchor.termId === el.dataset.termId)
        else open(el);
};

```

```

<div
  ref={contentRef}
  className="prose prose-invert max-w-none"
  onMouseOver={handlePointerOver}
  onMouseOut={handlePointerOut}
  onClick={handleClick}
  onFocus={handleFocus}
  dangerouslySetInnerHTML={{ __html: chapter.content }}
/>

<p className="mt-6 flex items-start gap-2 text-
  ed-lg px-3 py-2">
  <MousePointerClick className="w-4 h-4 shrink-0" />
  <span>
    Подчёркнутые термины – интерактивные: наведи
    объяснение на пальцах, мини-анимацию и кнопки
  </span>
</p>

{viz && (
  <section id="chapter-viz" className="mt-8 scroll-mt-16">
    <div className="flex items-center justify-between">
      <h3 className="flex items-center gap-2 text-indigo-500">
        <Sparkles className="w-4 h-4 text-indigo-500" />
        Демонстрация: {viz.title}
      </h3>
      <div className="flex items-center gap-2">
        <button
          onClick={() => onOpenSimulator(chapter.title)}
          className="flex items-center gap-1.5 text-sm font-medium
            slate-300 hover:text-white hover:border-indigo-500"
        >
          <Maximize2 className="w-3.5 h-3.5" />
        </button>
        {onOpenCompiler && (
          <button
            onClick={onOpenCompiler}

```



```
import { useMemo, useState } from "react";
import { Palette, RotateCcw, Wand2 } from "lucide-react";
import { useVizRuntime, vizNumber, vizRecord, vizArray,

/**
 * Тема 7 «Графы. Планарные. Покраска».
 * Жадная раскраска планарного графа: кликни цвет, потом
 * Компилятор читает v (текущая вершина), color[] (массив
 */

interface V {
  id: number;
  label: string;
  x: number;
  y: number;
}

// Планарный граф (триангуляция): 7 вершин, все рёбра б
const NODES: V[] = [
  { id: 0, label: "A", x: 210, y: 40 },
  { id: 1, label: "B", x: 350, y: 110 },
  { id: 2, label: "C", x: 320, y: 250 },
  { id: 3, label: "D", x: 110, y: 260 },
  { id: 4, label: "E", x: 70, y: 110 },
  { id: 5, label: "F", x: 210, y: 150 },
  { id: 6, label: "G", x: 205, y: 285 },
];

const EDGES: [number, number][] = [
  [0, 1], [1, 2], [2, 3], [3, 4], [4, 0],
  [0, 5], [1, 5], [2, 5], [3, 5], [4, 5],
  [2, 6], [3, 6], [5, 6],
];

const ADJ: number[][] = (() => {
  const adj: number[][] = NODES.map(() => []);
```

```
const liveColorArr = vizArray(vars?.color) ?? vizArray()
const liveK = vizNumber(vars?.k);
```

```
const liveColors: number[] | null = (() => {
  if (liveColorArr) {
    const arr = liveColorArr.map((c) => vizNumber(c))
    if (arr.every((c) => c === null || c >= -1)) {
      const padded = [...arr.map((c) => c ?? -1)];
      while (padded.length < NODES.length) padded.push(-1);
      return padded.slice(0, NODES.length);
    }
    return null;
  }
  if (liveColorRec) {
    return NODES.map((n) => {
      const byLabel = liveColorRec[n.label];
      const byId = liveColorRec[String(n.id)];
      const raw = byLabel !== undefined ? byLabel : byId;
      const c = typeof raw === "string" ? COLORS.find((v) => v === raw) : raw;
      return c === null || c === undefined ? -1 : c;
    });
  }
  return null;
})();
```

```
const compilerLinked = liveColors !== null || liveVLabel !== null
const shownColors = liveColors ?? colors;
```

```
const liveCurrent = (() => {
  if (liveVLabel) {
    const i = NODES.findIndex((n) => n.label === liveVLabel)
    if (i >= 0) return i;
  }
  if (liveVNum !== null && liveVNum >= 0 && liveVNum < NODES.length) {
    return liveVNum;
  }
  return null;
})();
```

```
const conflicts = useMemo(() => {
```

```

</button>
<button
  onClick={() => setColors(NODES.map(() => -1
  className="flex items-center gap-1 bg-slate
  >
    <RotateCcw className="w-3.5 h-3.5" /> C6poc
  </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-[1fr
  <div className="bg-slate-950 rounded-xl border
  <div className="absolute inset-0 bg-[radial-g
  <svg className="w-full h-[340px] relative z-1
    {EDGES.map(([u, v], i) => {
      <line key={i} x1={NODES[u].x} y1={NODES[u]
        stroke="#475569" strokeWidth={2.2} />
    })}
    {NODES.map((n) => {
      const c = shownColors[n.id];
      const isConflict = conflicts.has(n.id);
      const isCurrent = liveCurrent === n.id;
      const fill = isConflict ? CONFLICT : c >=
      return (
        <g key={n.id} className="cursor-pointer
          {isCurrent && <circle cx={n.x} cy={n.y}
        />
        <circle cx={n.x} cy={n.y} r={14} fill=
      >
        <text x={n.x} y={n.y + 4} textAnchor=
      ">
        {n.label}
      </text>
      {c >= 0 && {
        <text x={n.x} y={n.y - 20} textAnch
      vents-none">
        color[{n.id}]=c
      </text>

```

```

        <div className="text-[10px] uppercase track"
        <div className={`text-2xl font-extrabold font-
          {compilerLinked && liveK !== null ? liveK
        </div>
        <div className="text-[10px] text-slate-500"
      </div>
      <div className="bg-slate-900/60 rounded-xl border
        <b className="text-slate-200">Жадный алгоритм
        дей.
        Для планарного графа он даст  $\leq 6$  цветов, а
        я и Хакена, 1976).
      </div>
    </div>
  </div>
</div>
</div>
);
}

```

ФАЙЛ 37/92: src/components/ComplexityClassesViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useMemo, useState } from "react";
import { RotateCcw } from "lucide-react";
import { useVizRuntime, vizArray, vizString } from "../viz";

/**
 * Тема 24 «Классы сложности, сведение задач».
 * Диаграмма вложенности классов  $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$ .
 * полиномиальных сведений  $A \leq_p B$ : если  $B$  решается легко, то и  $A$ .
 */

type ClassId = "P" | "NP" | "PSPACE" | "EXPTIME";

interface Problem {
  id: string;

```

```

        ветку рекурсии.",
    EXPTIME: "EXPTIME — есть алгоритм за  $2^{poly(n)}$ . Обобщение",
    .",
};

const RECT: Record<ClassId, { x: number; y: number; w: number; h: number }> = {
    P: { x: 40, y: 90, w: 220, h: 210 },
    NP: { x: 30, y: 40, w: 400, h: 260 },
    PSPACE: { x: 15, y: 20, w: 440, h: 320 },
    EXPTIME: { x: 5, y: 5, w: 470, h: 360 },
};

export function ComplexityClassesViz() {
    const [selected, setSelected] = useState<ClassId | null>(null);
    const [selectedProblem, setSelectedProblem] = useState<Problem | null>(null);

    // — Синхронизация с Python-компилятором: cls — имя класса
    // reductions — список стрелок вида ["3-SAT", "Vertex Cover"]
    const runtime = useVizRuntime();
    const vars = runtime?.variables;
    const liveCls = vizString(vars?.cls);
    const liveTask = vizString(vars?.task);
    const liveRed = vizArray(vars?.reductions);
    const compilerLinked = liveCls !== null || liveTask !== null || liveRed.length > 0;

    const liveRedPair = useMemo(() => {
        if (!liveRed || liveRed.length < 2) return null;
        return [String(liveRed[0]), String(liveRed[1])] as [string, string];
    }, [liveRed]);

    const activeCls: ClassId | null =
        compilerLinked && liveCls
            ? (Object.keys(CLASS_META).find((k) => k.toLowerCase() === liveCls.toLowerCase()) as ClassId)
            : selected;

    const problemHit = (p: Problem) => {
        if (compilerLinked && liveTask) return p.name.toLowerCase() === liveTask.toLowerCase();
        return selectedProblem === p.id;
    };
}

```

```
</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr]">
  <div className="bg-slate-950 rounded-xl border border-slate-800">
    <svg className="w-full" viewBox="0 0 480 380">
      {[["EXPTIME", "PSPACE", "NP", "P"] as ClassMeta[]].map((cls) => {
        const r = RECT[cls];
        const dim = activeCls !== null && activeCls === cls ? 3 : 1;
        return (
          <g key={cls} onClick={() => setSelected(cls)}>
            <rect
              x={r.x} y={r.y} width={r.w} height={r.h}
              fill={CLASS_META[cls].color + (dim > 1 ? "a" : "o")}
              stroke={CLASS_META[cls].color}
              strokeWidth={activeCls === cls ? 3 : 1}
              opacity={dim > 0.45 : 1}
            />
            <text x={r.x + 12} y={r.y + 22} fill="white">
              {CLASS_META[cls].title}
              {cls === "NP" && <tspan fontSize="1.2em">
                </tspan>
              }
            </text>
          </g>
        );
      })
    </svg>
  </div>
</div>
```

```
/* стрелки сведений */
REDUCTIONS.map(([from, to, label]) => {
  const a = PROBLEMS.find((p) => p.id === from);
  const b = PROBLEMS.find((p) => p.id === to);
  const hot = compilerLinked && liveRedPair[from] === to
    ? (liveRedPair[0] && a.name.toLowerCase() === b.name.toLowerCase()
      ? liveRedPair[1] && b.name.toLowerCase() === a.name.toLowerCase()
      : selectedProblem === from || selectedProblem === to)
    : false;
  const mx = (a.x + b.x) / 2;
  const my = (a.y + b.y) / 2 - 4;
  return (
    <g key={`_${from}-${to}`} opacity={hot ? 1 : 0.5}>
      <line x1={a.x} y1={a.y} x2={b.x} y2={b.y} />
    </g>
  );
})
```

```

        <button
          key={cls}
          onClick={() => setSelected(selected ===
            className={`rounded-lg border px-3 py-2
              activeCls === cls ? "text-white" : "text-
            `}}
          style={{
            borderColor: CLASS_META[cls].color +
            backgroundColor: activeCls === cls ? "
          }}
        >
          {CLASS_META[cls].title}
          <div className="text-[9px] font-normal
            {cls === "P" ? "быстро решаем" : cls
          ени"}
        </div>
      </button>
    )}}
  </div>
  <div className="bg-slate-900/60 rounded-xl bo
    {compilerLinked && {
      <div className="mb-2 text-[10px] font-mon
        k="${liveTask}" : ""}</div>
    }}
    {infoText}
  </div>
  <div className="bg-amber-950/30 border border
    <b>Главный вопрос</b> –  $P = NP$ ? Все верят,
    Несведение ( $P \neq NP$ ) доказать никто не смог
    ою задачу к 3-SAT, и она автоматически сложн
  </div>
</div>
</div>
</div>
);
}

```

```

    [4, 5], [5, 6], [6, 4], // остров 2: E F G (цикл)
    [7, 8], [8, 9], // остров 3: H I J
];

const ADJ: number[][] = (() => {
    const adj: number[][] = NODES.map(() => []);
    EDGES.forEach(([u, v]) => {
        adj[u].push(v);
        adj[v].push(u);
    });
    return adj;
})();

const COMP_COLORS = ["#6366f1", "#f59e0b", "#10b981", "#f87192"];

interface StepFrame {
    comp: (number | null)[];
    queue: number[];
    current: number | null;
    scanning: number | null;
    count: number;
    log: string;
}

/** Полный прогон BFS-разметки: кадры строим заранее (чтобы не пересчитывать) */
function buildFrames(): StepFrame[] {
    const frames: StepFrame[] = [];
    const comp: (number | null)[] = NODES.map(() => null);
    let count = 0;
    const push = (f: Omit<StepFrame, "comp" | "count"> & {
        comp: (number | null)[];
    }) => {
        frames.push({ comp: [...comp], count, ...f });
    };

    push({ queue: [], current: null, scanning: null, log: "Инициализация вершины." });

    for (let s = 0; s < NODES.length; s++) {
        if (comp[s] !== null) continue;
        count += 1;
    }
}

```



```

    }
    const t = window.setTimeout(() => setStep((s) => Ma
    return () => window.clearTimeout(t);
}, [playing, step]);

// — Синхронизация с Python-компилятором: v — текуща
// или список меток, q — очередь, count — число компо
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveVLabel = vizString(vars?.v);
const liveVNum = vizNumber(vars?.v);
const liveCompRec = vizRecord(vars?.comp);
const liveCompArr = vizArray(vars?.comp);
const liveQ = vizArray(vars?.q);
const liveCount = vizNumber(vars?.count);

const liveCompByLabel = useMemo(() => {
  if (liveCompRec) return liveCompRec;
  if (liveCompArr) return Object.fromEntries(liveComp
  return null;
}, [liveCompRec, liveCompArr]);

const frame = FRAMES[Math.min(step, FRAMES.length - 1
const compilerLinked =
  liveVLabel !== null || liveVNum !== null || !!liveC

const liveIndexOf = () => {
  if (liveVLabel) {
    const idx = NODES.findIndex((n) => n.label === li
    return idx >= 0 ? idx : null;
  }
  return liveVNum !== null && liveVNum >= 0 && liveVN
};
const liveCurrent = liveIndexOf();

const liveCompOf = (n: V): number | null => {
  if (liveCompByLabel && n.label in liveCompByLabel)
    const c = vizNumber(liveCompByLabel[n.label]);

```

```

        className="flex items-center gap-1 bg-slate-700"
      >
        <RotateCcw className="w-3.5 h-3.5" /> Сброс
      </button>
      <button
        onClick={() => setStep((s) => Math.max(0, s - 1))}
        disabled={step === 0}
        className="bg-slate-700 hover:bg-slate-600"
      >
        ◀ Шаг
      </button>
      <button
        onClick={() => setPlaying((p) => !p)}
        className={`flex items-center gap-1 px-3 py-2 border-1px rounded-md ${
          playing ? "bg-emerald-600 hover:bg-emerald-500" : "bg-slate-700 hover:bg-slate-600"
        }`}
      >
        {playing ? <Pause className="w-3.5 h-3.5" /> : <Play className="w-3.5 h-3.5" />}
        {playing ? "Пауза" : "Пуск"}
      </button>
      <button
        onClick={() => { setPlaying(false); setStep(0); }}
        disabled={step >= FRAMES.length - 1}
        className="flex items-center gap-1 bg-indigo-600 hover:bg-indigo-500 text-white px-3 py-2 border-1px rounded-md"
      >
        <SkipForward className="w-3.5 h-3.5" /> Шаг
      </button>
    </div>
  </div>

<div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr] gap-4" >
  <div className="bg-slate-950 rounded-xl border border-slate-800 p-4" >
    <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)]" style={{backgroundSize: 20px 20px}} >
      <svg className="w-full h-[320px] relative z-10" >
        {EDGES.map(([u, v], i) => {
          const a = NODES[u];
          const b = NODES[v];
          const sameComp = liveCompOf(a) !== null & liveCompOf(b) !== null & liveCompOf(a) === liveCompOf(b)}
        )}
      </svg>
    </div>
  </div>
  <div className="bg-slate-950 rounded-xl border border-slate-800 p-4" >
    <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)]" style={{backgroundSize: 20px 20px}} >
      <svg className="w-full h-[320px] relative z-10" >
        {EDGES.map(([u, v], i) => {
          const a = NODES[u];
          const b = NODES[v];
          const sameComp = liveCompOf(a) !== null & liveCompOf(b) !== null & liveCompOf(a) === liveCompOf(b)}
        )}
      </svg>
    </div>
  </div>
</div>

```

```

    <div className="absolute top-2 right-2 bg-emerald-300 z-20">
      синхронизация с Python
    </div>
  })
</div>

<div className="space-y-3">
  <div className="bg-slate-950 rounded-xl border"
    <div className="text-[10px] uppercase tracking-wider"
      <div className="flex flex-wrap gap-1 min-h-[100px]"
        {shownQueue.length === 0 && <span className="text-slate-500">
          {shownQueue.map((q, i) => (
            <span key={` ${q} - ${i} `} className="px-2 py-1 text-white font-
            -mono">
              {NODES[q].label}
            </span>
          ))}
        </div>
      </div>
    </div>
  <div className="bg-slate-950 rounded-xl border"
    <div className="text-[10px] uppercase tracking-wider"
      <div className="grid grid-cols-5 gap-1"
        {NODES.map((n) => {
          const c = liveCompOf(n);
          return (
            <div key={n.id} className={`text-center text-slate-500 border-transparent text-slate-500`}
              style={c === null ? {} : { background-color: LORS.length } }>
              <div className="text-slate-400">{n.label}</div>
              <div>{c === null ? "-" : c}</div>
            </div>
          );
        })}
      </div>
    </div>
  </div>
  <div className="bg-slate-900/60 rounded-xl border"

```

```

{ line: '                // ЭТО МОСТ!', highlight: "normal" },
{ line: "                }", highlight: "normal" },
{ line: "                }", highlight: "normal" },
{ line: "                }", highlight: "normal" },
{ line: "                }", highlight: "normal" },
{ line: "                }", highlight: "normal" },
};

```

```

interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
  activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

```

```

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },

```

```

    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
        activeEdge: null, backEdge: [2,0],
        log: "Видим соседа А (уже посещён)! Обратное ребро
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3],
        tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
        activeEdge: [2,3], backEdge: null,
        log: "Идём C→D. tin[D]=5, low[D]=5.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
        activeEdge: [3,4], backEdge: null,
        log: "D→E. tin[E]=6, low[E]=6.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
        activeEdge: [4,5], backEdge: null,
        log: "E→F. tin[F]=7, low[F]=7.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
        activeEdge: null, backEdge: [5,3],
        log: "Из F видим D (посещён). Обратное ребро (F,D).
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],

```

```

    activeCodeLine: []
  },
];

```

```

export function DfsBridgesSimulator() {
  const [dfsIdx, setDfsIdx] = useState(0);
  useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length - 1);
  const rawV = vizString(vars.v) ?? (vizNumber(vars.v) ?? null);
  const rawTo = vizString(vars.to) ?? (vizNumber(vars.to) ?? null);
  const toId = (value: string | null) => {
    if (value === null) return null;
    const byLabel = GRAPH_NODES.find((node) => node.label === value);
    return byLabel ?? (Number.isFinite(Number(value)) ? Number(value) : null);
  };
  const v = toId(rawV);
  const to = toId(rawTo);
  if (v === null) return null;
  const candidates = DFS_STEPS
    .map((step, index) => ({ step, index }))
    .filter(({ step }) => step.currentNode === v && step.to === to);
  return candidates.at(-1)?.index ?? null;
});
const [dfsAuto, setDfsAuto] = useState(false);
const dfsTimer = useRef<NodeJS.Timeout | null>(null);

useEffect(() => {
  if (dfsAuto) {
    dfsTimer.current = setInterval(() => {
      setDfsIdx(prev => {
        if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false); }
        return prev + 1;
      });
    }, 2500);
  }
  return () => { if (dfsTimer.current) clearInterval(dfsTimer.current); };
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

```

```

const bridge = step.bridges.includes(e.key)
const activeTree = step.activeEdge && (
  (step.activeEdge[0]===e.u && step.activeEdge[1]===e.v) ||
  (step.activeEdge[0]===e.v && step.activeEdge[1]===e.u)
);
const activeBack = step.backEdge && (
  (step.backEdge[0]===e.u && step.backEdge[1]===e.v) ||
  (step.backEdge[0]===e.v && step.backEdge[1]===e.u)
);
let sc = "#475569", sw = 2, dash = "none"
if (bridge) { sc = "#eab308"; sw = 4; }
else if (activeTree) { sc = "#10b981"; sw = 2; }
else if (activeBack) { sc = "#f43f5e"; sw = 2; }
else if (step.visitedIndices.includes(e.u) || step.visitedIndices.includes(e.v)) { sc = "#444"; sw = 2; }
return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
  stroke={sc} strokeWidth={sw} strokeDash={dash}
  className="transition-all duration-500" />
}})
{step.currentNode !== null && step.currentNode.id !== n.id}
const n = GRAPH_NODES.find(x => x.id === n.id)
if (!n) return null;
return <circle cx={n.x} cy={n.y} r={18} fill={fill} stroke={stroke} />
>;
}}())
{GRAPH_NODES.map(n => {
  const cur = step.currentNode === n.id;
  const vis = step.visitedIndices.includes(n.id);
  const st = step.stack.includes(n.id);
  return <g key={n.id}>
    <circle cx={n.x} cy={n.y} r={12}
      fill={cur ? "#10b981" : st ? "#064e3b" : "#444"}
      stroke={cur ? "#fff" : st ? "#34d399" : "#444"}
      strokeWidth={2.5}
      className="transition-all duration-300" />
    <text x={n.x} y={n.y+3} textAnchor="middle">
      {n.label}</text>
    {vis && (
      <div>

```

```

    </div>

    {/* Stack */}
    <div className="bg-slate-900/50 rounded-xl bo
      <div className="text-[10px] font-bold text-
        <span>Стек DFS</span>
        <span className="text-indigo-400">{step.s
      </div>
      <div className="flex flex-col-reverse gap-1
        {step.stack.length === 0
          ? <div className="text-[10px] text-slate
            : step.stack.map((id, i) => {
              <div key={id} className={`px-2 py-0
                i === step.stack.length-1
                  ? "bg-emerald-600/20 text-emera
                  : "bg-slate-900 text-slate-400 l
              }`}>Вершина {GRAPH_NODES.find(n=>n.i
                <span className="text-slate-500 m
              </div>
            })
          }
        </div>
      </div>
    </div>

    {/* Log */}
    <div className="bg-indigo-950/10 border borde
      <p className="text-[11px] text-slate-300 le
      <div className="mt-2 pt-2 border-t border-s
        <span className="text-slate-400">Мосты:</
        <span className="font-mono font-bold text
          {step.bridges.length > 0 ? step.bridges
        </span>
      </div>
    </div>
  </div>
</div>
</div>
);

```



```

    { u: 'C', v: 'G', w: 3 },
    { u: 'C', v: 'E', w: 8 },
    { u: 'B', v: 'G', w: 4 },
    { u: 'B', v: 'D', w: 5 },
    { u: 'G', v: 'D', w: 1 },
    { u: 'G', v: 'E', w: 2 },
    { u: 'D', v: 'F', w: 3 },
    { u: 'E', v: 'F', w: 1 },
  ];

```

```

const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
  adjList[e.u].push({ v: e.v, w: e.w });
});

```

```

const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push({v: start, d: 0})' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлекаем элемент с минимальным d' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbors(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push({v: v, d: dist[v]})' },
  { line: 11, text: '    return dist // Все кратчайшие пути найдены' },
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };

```

```

        distances: { ...dist }, visited: { ...visited },
        log: `  Смотрим соседа ${v}. Текущее расстояние
              ${dist[u] + edge.w}.`,
        activeCodeLine: 8,
    ));

    if (dist[u] + edge.w < dist[v]) {
        dist[v] = dist[u] + edge.w;
        prev[v] = u;
        simulationSteps.push({
            currentNode: u, checkingEdge: { u, v },
            distances: { ...dist }, visited: { ...visited },
            log: `  Улучшение (Релаксация)! Новое кратчайшее
                  расстояние до ${v} равно ${dist[v]}.`,
            activeCodeLine: 9,
        });
    }
}
}
}

```

```

simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ` Дейкстра всё доставил! Все возможные кратчайшие
          расстояния найдены.`,
    activeCodeLine: 11,
});

```

```

    setSteps(simulationSteps);
}, []));

```

```

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => {
            setCurrentStepIndex((prev) => prev + 1);
        }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
}, [currentStepIndex, isPlaying]);

```

```

<div className="flex items-center gap-2">
  <button
    onClick={() => { setIsPlaying(false); setCu
    className="flex items-center gap-1 bg-slate
    ion-colors"
  >
    Сброс
  </button>
  <button
    onClick={() => setIsPlaying(!isPlaying)}
    className={`flex items-center gap-1 px-4 py
      isPlaying ? 'bg-amber-600 hover:bg-amber-!
      `}
  >
    {isPlaying ? '  Пауза' : '▶ Пуск'}
  </button>
  <button
    onClick={() => { setIsPlaying(false); if (c
    disabled={currentStepIndex >= steps.length
    className="flex items-center gap-1 bg-indigo
    px-3 py-2 rounded-lg text-sm font-medium tr
  >
    ⏏
  </button>
</div>
</div>

```

```

<div className="flex flex-col lg:flex-row gap-6 ml
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-indigo-950/2
    stify-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-indi
      ont-bold shadow-sm">
      ⏏ {currentStepIndex + 1} из {steps.length}
    </div>

    <svg className="w-full h-auto max-h-[500px]"

```

```

        r="12" fill="#1e293b"
        stroke={isChecked ? '#f59e0b' : '#f87192'}
      />
      <text
        x={({uNode.x + vNode.x} / 2} y={({uNode.y + vNode.y} / 2})
        fill={isChecked ? '#f59e0b' : '#f87192'}
        fontSize="11" fontWeight="bold" text={label}
      >
        {edge.w}
      </text>
    </g>
  );
})))

{/* Вершины */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id
  const isVisited = currentStep.visited[node.id]
  const dist = currentStep.distances[node.id]

  return (
    <g key={node.id} className="transition-all duration-300">
      <circle
        cx={node.x} cy={node.y}
        r={isCurrent ? 22 : 18}
        fill={isCurrent ? '#4f46e5' : isVisited ? '#f87192' : '#d1d7db'}
        stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f59e0b' : '#d1d7db'}
        strokeWidth={isCurrent ? 4 : 2}
        className="transition-all duration-300"
      />
      <text x={node.x} y={node.y + 5} fill="black">
        {node.id}
      </text>
      <text x={node.x} y={node.y - 28} fill="white">
        {dist === 999 ? '∞' : `d=${dist}`}
      </text>
      <text x={node.x} y={node.y + 35} fill="white">
        {node.name}
      </text>
    </g>
  )
})}
```



```
        </div>
      </div>
    </div>
  </div>
</div>
);
}
```

ФАЙЛ 41/92: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '...';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from '...';

interface DPStep {
  id: number;
  n: number;
  action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
  desc: string;
  codeLine: number;
  memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "  if (n in memo) return memo[n];",
  /* 3 */ "  if (n <= 2) return 1;",
  /* 4 */ "  memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
  /* 5 */ "  return memo[n];",
  /* 6 */ "}";
];

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
```

```

        action: 'base_case',
        desc: `Базовый случай: n <= 2 (n=${n}). Возвр
        codeLine: 3,
        memoState: { ...memoMap }
    });
    return 1;
}

generatedSteps.push({
    id: stepId++,
    n,
    action: 'call',
    desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
    codeLine: 4,
    memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
    id: stepId++,
    n,
    action: 'calc',
    desc: `Сохраняем в кэш: мемо[${n}] = ${res1} +
    codeLine: 5,
    memoState: { ...memoMap }
});

return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
    id: stepId++,
    n: targetN,
```

```

        memoState,
        desc: `Компилятор: k=${liveK ?? "-"}; таблица m
    }
    : baseStep;

return (
  <div className="bg-slate-900 rounded-2xl border border-
    { /* Header & Controls */ }
    <div className="flex flex-col lg:flex-row lg:items-
      <div>
        <div className="flex items-center gap-3 mb-2">
          <div className="p-2 bg-emerald-500/10 border
            <RefreshCw className="w-6 h-6" />
          </div>
          <h3 className="text-2xl font-extrabold text-
        </div>
        <p className="text-sm text-slate-400">
          Смотри, как таблица DP кэширует результаты
        </p>
      </div>

    <div className="flex flex-wrap items-center gap-
      <div className="flex items-center gap-2 bg-sl
        <span>N:</span>
        <select
          value={targetN}
          onChange={(e) => setTargetN(Number(e.targ
          disabled={isPlaying}
          className="bg-transparent text-white font-
        >
          <option value={4}>N = 4</option>
          <option value={5}>N = 5</option>
          <option value={6}>N = 6</option>
          <option value={7}>N = 7</option>
        </select>
      </div>

    <div className="flex items-center bg-slate-950

```



```

    { /* DP Table View */ }
    <div className="mb-8 bg-slate-950 p-4 md:p-6 rounded"
      <h4 className="text-xs font-bold uppercase tracking-wider"
        <div className="flex flex-wrap items-center gap-2"
          {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
            const isCached = num in memoState || num <= 2;
            const val = num <= 2 ? 1 : memoState[num];
            const isCurrent = currentStep?.n === num;

            return (
              <div
                key={num}
                className={`flex flex-col items-center justify-center gap-2
                  ${isCurrent ? 'bg-emerald-950 border-emerald-500' : ''}
                  ${isCached ? 'bg-slate-900 border-emerald-500/50' : ''}
                  ${!isCurrent & !isCached ? 'bg-slate-900/50 border-slate-800' : ''}
                `}
              >
                <span className="text-xs text-slate-400">N</span>
                <span className={`text-xl md:text-2xl font-medium text-slate-500`}
                  {isCached ? val : '0'}>
                </span>
                {num <= 2 && <span className="text-[9px] font-mono">1</span>}
              </div>
            );
          })}
        </div>
      </div>
    );
  }
}
</div>

```

```

{currentStep && (
  <div className="mt-6 pt-4 border-t border-slate-800"
    <div className="text-sm font-medium text-slate-500"
      <span className="inline-block w-2 h-2 rounded-full bg-currentStep.color" style={{display: inline-block; width: 10px; height: 10px; border-radius: 50%; background-color: currentStep.color; margin-right: 5px; vertical-align: middle;"/>

```

```

    <p className="text-sm text-slate-400">
      В классической рекурсии для  $N=\{targetN\}$  по
      вычисленное значение в таблице, мы момента.
    </p>
    <div className="p-4 bg-slate-900 rounded-xl text-slate-300">
      <span>Всего шагов симуляции: {steps.length}</span>
      <span className="text-emerald-400 font-bold">
    </div>
  </div>
})

{activeTab === 'code' && (
  <div className="bg-slate-950 rounded-2xl border">
    <div className="bg-slate-900 px-6 py-3 border">
      <span className="text-xs font-bold font-mono">
      <span className="text-xs text-emerald-400">
    </div>
    <pre className="p-6 font-mono text-sm space-x-2">
      {DP_CODE_LINES.map((line, idx) => {
        const lineNum = idx + 1;
        const isActive = currentStep?.codeLine === lineNum;
        return (
          <div
            key={lineNum}
            className={`flex items-center px-4 py-2 ${
              isActive
                ? 'bg-emerald-600/20 border-l-4 border-emerald-500'
                : 'text-slate-300 hover:bg-slate-900'
            }`}
          >
            <span className="w-8 text-xs text-slate-400">{lineNum}</span>
            <span>{line}</span>
          </div>
        );
      })}
    </pre>
    <div className="p-6 bg-slate-900/50 border-l-4 border-emerald-500">

```

```

    { id: 3, label: "D", x: 280, y: 250 },
    { id: 4, label: "E", x: 100, y: 250 }
  ];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

export function EulerSimulator() {
  // — Синхронизация с Python-компилятором: path — пос.
  // вершин (["A", "B", "C"...]), degrees/deg — словарь с
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const livePathLabels = vizArray(vars?.path)?.map((x) => {

  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Кликни на вершину
  const [c1Done, setC1Done] = useState(false);

  const resetC1 = () => {
    setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
  };

  // Компилятор управляет обходом, если в коде есть path
  const compilerLinked = livePathLabels !== null && live
  const livePath = useMemo(() => {
    if (!compilerLinked) return null;
    const ids = livePathLabels!
      .map((lab) => C1_NODES.find((n) => n.label === lab
      .filter((x): x is number => x !== undefined);
    return ids;
  }, [compilerLinked, livePathLabels]);
  const shownPath = livePath ?? c1Path;
  const shownVisited = useMemo(() => {

```

```

const nextPath = [...c1Path, vId];
setC1VisitedEdges(nextEdges);
setC1Path(nextPath);

if (nextEdges.length === C1_EDGES.length) {
  const startDegree = C1_EDGES.filter(e => e.u === vId).length;
  const endDegree = C1_EDGES.filter(e => e.v === vId).length;
  const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
  if (isCycle && nextPath[0] === vId) {
    setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
  } else {
    setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С`);
  }
  setC1Done(true);
} else {
  setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $`);
}
};

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
          <Undo className="h-3.5 w-3.5" /> Сброс
        </button>
      </div>

      <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
        <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate_1px)] border border-transparent">
          <svg className="w-full h-[260px] z-10 relative">
            {C1_EDGES.map((e, i) => {
              const u = C1_NODES.find(n => n.id === e.u)!;
              const v = C1_NODES.find(n => n.id === e.v)!;
              const key = `${Math.min(e.u, e.v)}-${Math.max(e.u, e.v)}`;
              const used = shownVisited.includes(key);
            })}
          </svg>
        </div>
      </div>
    </div>
  )
)

```

```

        </div>
    </div>
);
}

```

ФАЙЛ 43/92: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
  id: number;
  question: string;
  options: string[];
  correctAnswer: number;
  explanation: string;
}

const quizQuestions: Question[] = [
  {
    id: 1,
    question: 'Что мы делаем в алгоритме Краскала, если ребро, красным) цвет?',
    options: [
      'Мы радуемся и добавляем его в минимальное остовное дерево',
      'Мы перекрашиваем их в синий цвет и делим граф по компонентам связности',
      'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
      'Мы вызываем алгоритм Дейкстры для поиска нового пути'
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане, добавление этого ребра нарушит структуру дерева.'
  },
  {
    id: 2,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска кратчайшего пути?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Краскала',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 2,
    explanation: 'Верно! Алгоритм Краскала используется для нахождения минимального остовного дерева, а не кратчайшего пути.'
  },
  {
    id: 3,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска в ширину?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Брестера',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 1,
    explanation: 'Верно! Алгоритм Дейкстры является алгоритмом поиска в ширину, но с учетом весов ребер.'
  },
  {
    id: 4,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска в глубину?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Брестера',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 1,
    explanation: 'Верно! Алгоритм Дейкстры является алгоритмом поиска в ширину, но с учетом весов ребер.'
  },
  {
    id: 5,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска в ширину?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Брестера',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 1,
    explanation: 'Верно! Алгоритм Дейкстры является алгоритмом поиска в ширину, но с учетом весов ребер.'
  },
  {
    id: 6,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска в глубину?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Брестера',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 1,
    explanation: 'Верно! Алгоритм Дейкстры является алгоритмом поиска в ширину, но с учетом весов ребер.'
  },
  {
    id: 7,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска в ширину?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Брестера',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 1,
    explanation: 'Верно! Алгоритм Дейкстры является алгоритмом поиска в ширину, но с учетом весов ребер.'
  },
  {
    id: 8,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска в глубину?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Брестера',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 1,
    explanation: 'Верно! Алгоритм Дейкстры является алгоритмом поиска в ширину, но с учетом весов ребер.'
  },
  {
    id: 9,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска в ширину?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Брестера',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 1,
    explanation: 'Верно! Алгоритм Дейкстры является алгоритмом поиска в ширину, но с учетом весов ребер.'
  },
  {
    id: 10,
    question: 'Какой из следующих алгоритмов не является алгоритмом поиска в глубину?',
    options: [
      'Алгоритм Дейкстры',
      'Алгоритм Брестера',
      'Алгоритм Флойда-Уоршелла',
      'Алгоритм Брестера'
    ],
    correctAnswer: 1,
    explanation: 'Верно! Алгоритм Дейкстры является алгоритмом поиска в ширину, но с учетом весов ребер.'
  }
];
```

```

    {
      id: 5,
      question: 'Какая структура данных идеально помогает',
      options: [
        'СНМ (Система непересекающихся множеств / Disjoint',
        'Стек (LIFO).',
        'Красно-черное дерево.',
        'Хэш-таблица со списками коллизий.'
      ],
      correctAnswer: 0,
      explanation: 'Блестяще! СНМ (DSU) позволяет за прак
        '
    }
  ];

```

```

export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<num
  const [selectedOption, setSelectedOption] = useState<
  const [isAnswered, setIsAnswered] = useState<boolean>
  const [score, setScore] = useState<number>(0);
  const [showResults, setShowResults] = useState<boolean>

```

```

  const currentQ = quizQuestions[currentQIndex];

```

```

  const handleSelectOption = (index: number) => {
    if (isAnswered) return;
    setSelectedOption(index);
    setIsAnswered(true);
    if (index === currentQ.correctAnswer) {
      setScore(prev => prev + 1);
    }
  };

```

```

  const handleNext = () => {
    if (currentQIndex + 1 < quizQuestions.length) {
      setCurrentQIndex(prev => prev + 1);
      setSelectedOption(null);
      setIsAnswered(false);
    }
  };

```

```

        className="inline-flex items-center gap-2 px-3 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
      >
        <RotateCcw className="w-5 h-5" />
        <span>Пройти тест заново</span>
      </button>
    </div>
  );
}

return (
  <div className="bg-slate-900/90 border border-slate-800" >
    <div className="flex items-center justify-between" >
      <div className="flex items-center gap-3" >
        <span className="bg-purple-600 text-white px-3 py-2 rounded" >
          Экспресс-тест
        </span>
        <h3 className="text-xl font-bold text-white" >
          {currentQ.question}
        </h3>
      </div>
      <span className="text-sm font-mono text-slate-400" >
        Вопрос {currentQIndex + 1} / {quizQuestions.length}
      </span>
    </div>

    <div className="mb-8" >
      <h4 className="text-xl md:text-2xl font-bold text-white" >
        {currentQ.question}
      </h4>

      <div className="space-y-4" >
        {currentQ.options.map((option, idx) => {
          const isSelected = selectedOption === idx;
          const isCorrect = idx === currentQ.correctAnswer;

          let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
          if (isSelected) {
            if (isCorrect) {
              optionStyle = "bg-green-500/80";
            } else {
              optionStyle = "bg-red-500/80";
            }
          }
          return (
            <div className={optionStyle} >
              {option}
            </div>
          );
        })}
      </div>
    </div>
  </div>
);

```

```

        <AlertCircle className="w-4 h-4" />
        <span>Объяснение:</span>
      </div>
      <p className="text-slate-300 text-sm leading-4">
        {currentQ.explanation}
      </p>
    </div>
  )}

  <div className="flex justify-end pt-4 border-t border-slate-200">
    <button
      onClick={handleNext}
      disabled={!isAnswered}
      className={
        "px-8 py-3.5 font-bold rounded-xl transition-colors duration-200"
        (!isAnswered
          ? "bg-slate-800 text-slate-500 border border-slate-700"
          : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700")
      }
    >
      {currentQIndex + 1 < quizQuestions.length ? 'Следующий' : 'Завершить'}
    </button>
  </div>
</div>
);
};

```

ФАЙЛ 44/92: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber } from '../viz';

interface Step {
  k: number; i: number; j: number;
}

```



```

5: [{ v: 6, w: 5 }],
6: [{ v: 1, w: 1 }],
};

const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V' },
  { line: 2, text: '    dist = copy(matrix)' },
  { line: 3, text: '    for k from 0 to V - 1: // Пос' },
  { line: 4, text: '        for i from 0 to V - 1:' },
  { line: 5, text: '            for j from 0 to V - 1' },
  { line: 6, text: '                if dist[i][k] + d' },
  { line: 7, text: '                    dist[i][j] = ' },
  { line: 8, text: '    return dist' },
];

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist = initialMatrix.map(row => [...row]);

  simulationSteps.push({
    k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: '  Инициализация матрицы прямых путей, ∞ (99'
    activeCodeLine: 2,
  });

  const N = 7;
  for (let k = 0; k < N; k++) {
    simulationSteps.push({
      k, i: -1, j: -1, matrix: dist.map(r => [...r]),
      log: `+ Внешний цикл. Разрешаем пути через про
      activeCodeLine: 3,
    });

    for (let i = 0; i < N; i++) {
      for (let j = 0; j < N; j++) {
        if (i === j || i === k || j === k) continue;

        const oldVal = dist[i][j];

```

```

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const rawMatrix = vizArray(vars?.d) ?? vizArray(vars?.d);
const liveMatrix = rawMatrix
  ? baseStep.matrix.map((row, i) => {
      const values = vizArray(rawMatrix[i]);
      return row.map((_ , j) => vizNumber(values?.[j]))
    })
  : undefined;
const compilerLinked = liveK !== null || liveI !== null || liveJ !== null;
const currentStep: Step = compilerLinked
  ? {
      ...baseStep,
      k: liveK ?? -1,
      i: liveI ?? -1,
      j: liveJ ?? -1,
      matrix: liveMatrix ?? baseStep.matrix,
      updated: !!runtime?.changed?.some(({name}) => name === 'matrix'),
      log: `Компилятор: k=${liveK ?? "-"}, i=${liveI ?? "-"}, j=${liveJ ?? "-"}`
    }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
    <div className="flex flex-wrap items-center justify-center gap-2">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white rounded-md">
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-emerald-300">Полный код
          </p>
        </div>
      </div>
    </div>

    <div className="flex items-center gap-2">

```

```

        <g key={`${u}-${edge.v}`}>
          <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
          <circle cx={({uNode.x + vNode.x) / 2
isVia ? '#818cf8' : '#64748b'} strokeWidth
          <text x={({uNode.x + vNode.x) / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
        </g>
      );
    });
  }}}
  {nodesSvg.map((node) => {
    const isK = currentStep.k === node.id;
    const isI = currentStep.i === node.id;
    const isJ = currentStep.j === node.id;
    let fill = '#1e293b'; let stroke = '#64748b';
    if (isK) { fill = '#4f46e5'; stroke = '#a1887f'; }
    else if (isI) { fill = '#b45309'; stroke = '#a1887f'; }
    else if (isJ) { fill = '#047857'; stroke = '#a1887f'; }
    return (
      <g key={node.id} className="transition-
        <circle cx={node.x} cy={node.y} r={is
sJ ? 4 : 2} className="transition-colors du
        <text x={node.x} y={node.y + 5} fill=
        <text x={node.x} y={node.y + 32} fill=
      </text>
    </g>
    );
  }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
  <p className="font-semibold text-amber-400
  <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}

```

```

<div className="overflow-x-auto">
  <table className="w-full text-center border
    <thead><tr className="bg-rose-950/40 text
      <th className="p-2 border-r border-rose
        {nodeNames.map((n, idx) => <th key={idx
-400 font-bold' : ''}`}>{n.split(' ')[0]} {
      </tr></thead>
      <tbody className="text-xs font-mono">
        {currentStep.matrix.map((row, i) => (
          <tr key={i} className="border-b borde
            <td className={`p-2 bg-rose-950/40
tStep.k === i ? 'text-amber-400' : ''}`}>
              {nodeNames[i].split(' ')[0]} {i}
            </td>
            {row.map((val, j) => {
              const isUpd = currentStep.i === i
              const isVia = currentStep.k !== -
tep.j === j});
              let bg = 'bg-slate-900/40 text-sl
              if (isUpd) bg = 'bg-emerald-900/8
              else if (isVia) bg = 'bg-indigo-9
              else if (currentStep.i === -1 &&
                bg = 'bg-indigo-950/60 text-ind
              else if (i === j) bg = 'bg-slate-
              return (<td key={j} className={`p
=== 999 ? '∞' : val}</td>);
                }}}
              </tr>
            }}}
          </tbody>
        </table>
      </div>
    </div>

```

/* Код алгоритма */

```

<div className="w-full lg:w-1/2 bg-slate-900/60
  <div>
    <h4 className="text-lg font-bold text-slate

```

```

const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];

const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};

type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
  queueOrStack?: string[];
  visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
  const steps: Action[] = [];
  const vis = new Set<string>();
  const stack: string[] = []; // for visualization, j

  function dfs(v: string) {
    vis.add(v);
    stack.push(v);
    steps.push({ type: 'visit', node: v, desc: `Замечание: ${v} посещено`,
      visitedNodes: Array.from(vis) });

    for (const u of adj[v]) {
      if (!vis.has(u)) {

```

```

    }
  }

  steps.push({ type: 'backtrack', node: '', desc: `04
    om(vis)] });

  return steps;
}

```

```

export function GraphTraversalViz() {
  const [mode, setMode] = useState<"dfs" | "bfs">("df
  const [autoPlay, setAutoPlay] = useState(false);
  const [stepIdx, setStepIdx] = useState(0);

  const steps = useMemo(() => mode === 'dfs' ? genera
  useVizStepSync(stepIdx, setStepIdx, steps.length -
    const v = vizString(vars.v);
    const structure = vizArray(vars.stack) ?? vizArray
    const visited = vizArray(vars.visited);
    if (v === null && !structure && !visited) return
    const candidates = steps
      .map(({step, index}) => ({ step, index }))
      .filter(({ step }) =>
        (v === null || step.node === v) &&
        (!structure || step.queueOrStack?.length ===
        (!visited || step.visitedNodes?.length === vi
      );
    return candidates.at(-1)?.index ?? null;
  });

  useEffect(() => {
    setStepIdx(0);
    setAutoPlay(false);
  }, [mode]);

  useEffect(() => {
    if (autoPlay) {

```

```

    { /* Edges */
    {edges.map((e, i) => {
      const u = nodes.find(n => n
      const v = nodes.find(n => n
      // Check if edge is part of
      const edgeTraversed = isVis

      return (
        <line key={i} x1={u.x}
          className={`stroke-
500' : 'stroke-emerald-500') : 'stroke-slate
        );
      })}

    { /* Nodes */
    {nodes.map(n => {
      const visited = isVisited(n
      const current = isCurrent(n

      let fillColor = "fill-slate
      let strokeColor = "stroke-s
      let textColor = "fill-slate
      let radius = 18;

      if (visited) {
        fillColor = mode === 'd
        strokeColor = mode ===
        textColor = "fill-white
      }

      if (current && step.type !==
        strokeColor = "stroke-a
        textColor = "fill-amber
        radius = 22;
      }

      return (
        <g key={n.id} className=

```

```

        }}
      </div>
    )))
    {!!step.queueOrStack || step.queue}
    <div className="text-slate-500" style={stepStyle}
      >
    </div>
  </div>
</div>

<div className="flex flex-col sm:flex-row gap-2" style={stepStyle} >
  { /* Controls */ }
  <div className="flex bg-slate-900 border border-slate-800 rounded-lg p-2 text-slate-400 hover:text-white hover:bg-slate-800" style={stepStyle} >
    <Undo strokeWidth={3} size={16} />
    <button onClick={() => setAutoPlay(!autoPlay)}
      justify-center min-w-[80px] ${mode === 'default' ? '' : 'disabled'}
    >
      {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
    </button>
    <button onClick={() => {setAutoPlay(!autoPlay); setStepIndex(steps.length - 1)}} className="p-2 text-slate-400 hover:bg-transparent" >
      <SkipForward strokeWidth={3} size={16} />
    </button>
    <button onClick={() => {setAutoPlay(!autoPlay); setStepIndex(0)}}
      bg-slate-800 rounded-lg ml-2 border-l border-slate-700
    >
      <RefreshCw strokeWidth={3} size={16} />
    </button>
  </div>

  { /* Description Log */ }
  <div className="flex-1 bg-slate-900/50 border border-slate-800 p-2" style={stepStyle} >
    <div className="text-[10px] text-slate-400" style={stepStyle} >
      {steps.length}</div>
  </div>
</div>

```



```

    }
    return a;
}

function siftDown(arr: Patient[]): Patient[] {
    const a = arr.map(x => ({ ...x }));
    const n = a.length;
    let idx = 0;
    while (true) {
        let largest = idx;
        const l = 2 * idx + 1;
        const r = 2 * idx + 2;
        if (l < n && a[l].priority > a[largest].priority) largest = l;
        if (r < n && a[r].priority > a[largest].priority) largest = r;
        if (largest !== idx) {
            [a[largest], a[idx]] = [a[idx], a[largest]];
            idx = largest;
        } else break;
    }
    return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
    return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
    const level = Math.floor(Math.log2(index + 1));
    const posInLevel = index - (Math.pow(2, level) - 1);
    const count = Math.pow(2, level);
    const x = ((posInLevel + 0.5) / count) * 100;
    const y = 10 + level * 24;
    return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT PRESETS = [

```

```

        emoji: " ",
        symptom: `индекс ${index}`,
        priority,
        animState: "idle",
    });
  })
  : heap;
const [customName, setCustomName] = useState('');
const [customPriority, setCustomPriority] = useState(0);
const [log, setLog] = useState<string>('');
const [shakeEmpty, setShake] = useState(false);
const [busy, setBusy] = useState(false);
const [healingPatient, setHealing] = useState<Patient>({});

const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

// --- INSERT: Новый пациент поступает в приемный покой
const insertPatient = useCallback((name: string, priority: number) => {
  if (busy || heap.length >= 15) {
    addLog('⚠ Все палаты переполнены! Дождитесь выписки');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);

  const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
  const finalName = name || preset.name.split(' ')[0];
  const finalEmoji = preset.emoji;
  const finalSymptom = preset.symptom;

  const newPatient: Patient = {
    id: `p${uid++}`,
    name: finalName,
    emoji: finalEmoji,
    symptom: finalSymptom,
    priority,
    animState: 'in',
  };
  heap.push(newPatient);
  setLog(prev => [...prev, `Пациент ${finalName} (${finalEmoji}) поступил в приемный покой с ${finalSymptom} (приоритет: ${priority})`]);
  setShake(false);
  setBusy(false);
}, [heap, uid]);

```

```

    setBusy(true);
    const topPatient = heap[0];
    setHealing(topPatient);

    addLog(` СРОЧНО! Забираем самого тяжелого: ${topPa

// Step 1: Root lights up as winner
setHeap(prev => prev.map((x, i) => i === 0 ? { ...x

setTimeout(() => {
    // Step 2: Root exits the tree, tree reorganizes
    setHeap(prev => prev.map((x, i) => i === 0 ? { ..

    setTimeout(() => {
        // Run binary heap extraction
        setHeap(prev => {
            if (prev.length === 0) return [];
            const a = prev.map(x => ({ ...x }));
            a[0] = { ...a[a.length - 1] };
            a.pop();
            return siftDown(a).map(x => ({ ...x, animStat
        }));

        // Patient gets healed in the bed
        setTimeout(() => {
            setHealing(prev => prev ? { ...prev, animStat
            addLog(` Успех! Пациент ${topPatient.name} п
            setTimeout(() => {
                setHealing(null);
                setBusy(false);
            }, 600);
        }, 800);

    }, 350);
}, 650);
}, [busy, heap]);

const reset = () => {

```

```

<div className="bg-emerald-950/40 border border-e
  <div className="text-3xl"> </div>
  <div>
    <h3 className="font-black text-emerald-400 te
      ty Queue)</h3>
    <p className="text-xs text-slate-300 mt-1 lea
      В реанимации не важно, кто пришел раньше, а
      остояние</b> (максимальный приоритет).
      Куча (Heap) автоматически перестраивает пац
      оказывался на самом верху дерева (в корне).
    </p>
  </div>
</div>

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex flex-col lg:flex-row items-s

  {/* Добавить случайного */}
  <button
    onClick={handleRandomInsert}
    disabled={busy || heap.length >= 15}
    className="flex-1 min-w-[150px] bg-gradient-t
      opacity-40 disabled:cursor-not-allowed text
      ve:scale-95 transition-all cursor-pointer f
  >
    <span> Привезти по Скорой (INSERT)</span>
  </button>

```

```

{/* Добавить кастомного */}
<form onSubmit={handleCustomInsert} className="
  <input
    type="text"
    required
    value={customName}
    onChange={e => setCustomName(e.target.value
    placeholder="Имя пациента"
    className="flex-1 bg-slate-800 border borde
    er-emerald-500 transition-colors placeholde

```

```

        Сброс
      </button>
    </div>

    {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
    <div className="grid grid-cols-1 lg:grid-cols-12" style={{
      gridTemplateColumns: '1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr',
      gap: '10px',
    }}>

      {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
      <div className={`lg:col-span-8 bg-slate-900 rounded-2xl p-4
        -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`}>
        <div className="absolute top-4 left-4 text-[14px] font-medium" style={{
          color: 'white',
        }}>
          Сортировка по тяжести (Двоичная куча / Max-Heap)
        </div>

        <div className="relative w-full h-full min-h-[380px]" style={{
          position: 'relative',
        }}>
          <svg className="absolute inset-0 w-full h-full" style={{
            fill: 'none',
            stroke: 'white',
            stroke-width: 2,
          }}>
            {edges}
          </svg>

          {visualHeap.map((patient, idx) => {
            const pos = nodePos(idx);
            const isRoot = idx === 0;
            const isCompilerNode = liveI === idx;
            return (
              <div
                key={patient.id}
                className={`
                  absolute flex flex-col items-center
                  -translate-x-1/2 -translate-y-1/2 transition-all
                  ${patient.animState === 'in' ? 'animate-in' : ''}
                  ${patient.animState === 'winner' ? 'animate-winner' : ''}
                  ${patient.animState === 'out' ? 'animate-out' : ''}
                  ${isCompilerNode ? 'ring-4 ring-amber-500' : ''}
                  ${isRoot ? 'bg-rose-950/80 border-rose-500' : ''}
                `}
                style={{
                  left: `${pos.x}%`,
                  top: `${pos.y}%`,
                }}
              >
                {patient.name}
                {patient.status}
              </div>
            )
          })}
        </div>
      </div>
    </div>
  </div>
)

```

```

        </div>
        <div>
          <h4 className="text-white font-black">
            <span>Пациент: {healingPatient.name}</span>
          </h4>
          <p className="text-emerald-400 text-x">
            Состояние: Стабилизация ({healingPa
          </p>
          <div className="flex justify-center g">
            <span className="w-2 h-2 rounded-fu">
              <Heart className="w-4 h-4 text-rose
            </div>
          </div>
        </div>
      </div>
    ) : (
      <div className="text-center text-slate-60">
        <span className="text-5xl mb-2 block">
          <p className="text-xs font-bold font-mo">
            <p className="text-[10px] mt-1">Ждём са
          </p>
        </div>
      </div>
    )}
  </div>

  <div className="w-full h-3 bg-gradient-to-r f">
  </div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
  <div className="text-[10px] font-mono text-slate
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-ce
      >
        {idx === 0 ? <span className="text-emerald-!
        <span>{entry}</span>

```

```

    useEffect(() => {
      if (steps <= 1 || paused) return;
      const t = setInterval(() => setStep((s) => (s + 1) % 4), period);
      return () => clearInterval(t);
    }, [steps, period, paused]);

    return step;
  }

export const C = {
  idle: "#334155",
  idleStroke: "#475569",
  text: "#e2e8f0",
  dim: "#94a3b8",
  active: "#6366f1",
  done: "#10b981",
  warn: "#f59e0b",
  bad: "#f43f5e",
  edge: "#475569",
};

export const Svg: React.FC<{ children: React.ReactNode;
const paused = useContext(DemoPauseContext);
return (
  <div className="w-full">
    <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
      {children}
    </svg>
    {caption && (
      <div className="text-[10px] text-slate-400 text-center">
        {caption}
      </div>
    )}
    <div className="text-[9px] text-slate-600 text-center">
      {paused ? "пауза – курсор на карточке" : "наведи"}
    </div>
  </div>
);

```

ФАЙЛ 48/92: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7
  const pos: Record<string, [number, number]> = { i: [4, 1], j: [4, 4], k: [3, 4] };
  const viaOpen = step >= 1;
  const relaxed = step >= 2;
  return (
    <Svg
      caption={
        step === 0
          ? "k ещё запрещена: путь i → j = 9"
          : step === 1
            ? "Разрешаем пересадку через k: 3 + 4 = 7"
            : "7 < 9 → D[i][j] = 7"
        }
    >
      <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.good} />
      <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.good} />
      <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.good} />

      <text x={130} y={112} textAnchor="middle" fontSize=
        9
      </text>
      <text x={72} y={54} textAnchor="middle" fontSize=
        3
      </text>
      <text x={188} y={54} textAnchor="middle" fontSize=
```



```
    });
};
```

```
/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
  const step = useLoop(3, 1300);
  const pos: Record<string, [number, number]> = { A: [500, 100], B: [500, 300], C: [500, 500], D: [500, 700], E: [500, 900] };
  const edges: [string, string, number][] = [
    ["A", "B", 1],
    ["B", "C", 1],
    ["C", "D", 1],
    ["D", "E", 1],
    ["A", "C", 2],
    ["B", "D", 2],
    ["C", "E", 2],
    ["A", "E", 3],
    ["B", "E", 3],
    ["C", "E", 3],
  ];
  return (
    <Svg
      caption={
        step === 0
          ? "Вершины – объекты (города, слова, состояния)"
          : step === 1
            ? "Рёбра – связи между ними"
            : "Вес ребра – цена перехода"
      }
    >
      {edges.map(([u, v, w], i) => {
        const a = pos[u];
        const b = pos[v];
        return (
          <g key={i}>
            <Edge a={a} b={b} color={step >= 1 ? C.active : C.inactive}>
              {step >= 2 && (
                <text
                  x={{(a[0] + b[0]) / 2}}
                  y={{(a[1] + b[1]) / 2 - 4}}
                  textAnchor="middle"
                  fontSize={10}
                  fontWeight="700"
                  fill={C.warn}
                >
                  {w}
                </text>
              )}
            </g>
          )}
      )}
    </Svg>
  );
};
```

```

        <rect x={30 + i * 52} y={62} width={46} hei
        <text x={53 + i * 52} y={77} textAnchor="mi
            {v}
        </text>
    </g>
  )))

  {step >= 2 &&
    raw.map(({v, i}) => {
      <g key={`c${i}`}>
        <rect x={startX + i * (boxW + 4)} y={98} wi
        <text x={startX + i * (boxW + 4) + boxW / 2
          {sorted.indexOf(v)}
        </text>
      </g>
    })
  </Svg>
);
};

/* ----- Splay: три случая поворота ---

type Pt = [number, number];

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, st
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : i

  return (
    <Svg caption={captions[step]}>
      {f.edges.map(([a, b], i) => {
        <line

```

```

    { pos: { g: [176, 24], p: [126, 70], x: [76, 112] }
    { pos: { p: [130, 26], x: [78, 78], g: [186, 78] }
    { pos: { x: [102, 24], p: [152, 70], g: [202, 112] }
  }}
  />
};

/** Zig-Zag – x и p с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
  <SplayFrames
    captions=["Зигзаг: g влево, x вправо", "Поворот НИ
    frames=[
      { pos: { g: [176, 24], p: [110, 70], x: [152, 112] }
      { pos: { g: [176, 24], x: [110, 70], p: [64, 112] }
      { pos: { x: [130, 30], p: [80, 100], g: [180, 100] }
    ]}
  />
};

```

ФАЙЛ 49/92: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
  const order = ["A", "B", "C", "D", "E", "F"];
  const step = useLoop(order.length + 1, 850);

```

```

    { id: "трусы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["трусы", "штаны"],
    ["носки", "штаны"],
    ["носки", "боты"],
    ["штаны", "боты"]
  ];
  const order = ["трусы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, n.x, n.y]));
  return (
    <Svg caption={`Порядок: ${placed.join(" → ") || "..."} `}
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
      })}
      {nodes.map((n) => {
        const idx = placed.indexOf(n.id);
        return (
          <g key={n.id}>
            <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
            <text x={n.x} y={n.y} textAnchor="middle" dx={-10} dy={-10}>
              {n.id}
            </text>
            {idx >= 0 && <circle cx={n.x + 30} cy={n.y} r={10} />
            {idx >= 0 && {
              <text x={n.x + 30} y={n.y - 12} textAnchor="left">
                {placed[idx]}
              </text>
            }}
          </g>
        );
      })}
      <text x={224} y={62} textAnchor="middle" fontSize={16}>
        Порядок: {placed.join(" → ") || "..."}
      </text>
      <text x={224} y={74} textAnchor="middle" fontSize={16}>
        {placed[placed.length - 1]}
      </text>
    </Svg>
  );
};

```

```

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;

```

```

const pos: Record<string, [number, number]> = {
  A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
};
const edges: [string, string][] = [["A", "B"], ["A", "C"], ["A", "D"], ["B", "C"], ["B", "D"], ["C", "D"]];
const cut = step === 1;
return (
  <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}
    {edges.map(([, i]) => {cut && (i === 2 || i === 3) ? "" : i})}
    {Object.entries(pos).map(([, [x, y]]) =>
      cut && x === 130 ? (
        <circle key={x} cx={x} cy={y} r={12} fill="none" />
      ) : (
        <Node key={x} x={x} y={y} label={x} fill={x} />
      )
    )}
  </Svg>
);
};

```

```

export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100], E: [220, 100]
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);
  const taken = new Set(chosen.slice(0, step));
  return (
    <Svg caption={`Жадно берём самые лёгкие рёбра без циклов`}
      {edges.map((e, i) => {
        const on = taken.has(`${e.u}-${e.v}`);
        return (
          <g key={i}>
            <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
          </g>
        );
      })}
    </Svg>
  );
};

```

```

export const HeapDemo: React.FC = () => {
  const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [
  const step = useLoop(states.length, 1000);
  const heap = states[step];
  const pos: [number, number][] = [[130, 24], [80, 68],
  const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
  return (
    <Svg caption="Новый элемент всплывает наверх, пока м
      <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b=
      <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
      {heap.map((v, i) => (
        <Node key={i} x={pos[i][0]} y={pos[i][1]} label=
      ))}
    </Svg>
  );
};

export const StackDemo: React.FC = () => {
  const states = [["A"], ["A", "B"], ["A", "B", "C"], [
  const step = useLoop(states.length, 800);
  const items = states[step];
  const popping = step >= 3;
  return (
    <Svg caption={popping ? "поп: забираем ВЕРХНИЮЮ тапе
      <rect x={90} y={22} width={80} height={96} rx={6}
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={94} y={100 - i * 26} width={72} height=
            fill={i === items.length - 1 ? (popping ? C
          <text x={130} y={111 - i * 26} textAnchor="mi
        </g>
      ))}
      <text x={130} y={16} textAnchor="middle" fontSize=
    </Svg>
  );
};

```

```

        ) : null
      })
      {pos.map(([x, y], i) => (<Node key={i} x={x} y={y}
</Svg>
    ));
  };

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

export const TrieDemo: React.FC = () => {
  const step = useLoop(TRIE_NODES.length + 1, 700);
  return (
    <Svg caption="Путь от корня по буквам = слово; общий"
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
      ]}
      {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
    </Svg>
  ));
};

export const TrieBfsDemo: React.FC = () => {
  const levels = [[0], [1, 2], [3, 4, 5]];
  const step = useLoop(levels.length + 1, 950);
  const doneIds = new Set(levels.slice(0, step).flat());
  const wave = step > 0 && step <= levels.length ? leve
  return (
    <Svg caption={`BFS по бору: уровень ${Math.max(0, M
      {TRIE_NODES.filter((n) => n.parent !== null).map(
        const p = TRIE_NODES[n.parent as number];

```

```

    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
    { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
    { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых"
      {nodes.map((n, i) => (
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
            fill={n.lvl === step ? C.active : C.idle} stroke={C.dim} />
          <text x={n.x} y={n.y} textAnchor="middle" domClass={n.domClass} />
        </g>
      )
    )}
    </Svg>
  );
};

```

```

export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;
  return (
    <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] - P[${l}]` : "о префиксные суммы P"}>
      <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
      {a.map((v, i) => (
        <g key={i}>
          <rect x={20 + i * 39} y={24} width={34} height={12}
            fill={show && i >= l && i <= r ? C.active : C.idle} stroke={C.dim} />
          <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
        </g>
      )
    )}
    </Svg>
  );
};

```



```

-----
const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
const avg = step ? (total / step).toFixed(2) : "-";
return (
  <Svg caption={`Сделано ${step} операций, суммарно $
    {costs.map((c, i) => (
      <rect key={i} x={14 + i * 30} y={104 - c * 10} width={30} height={10}
      fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}
    )}
    <line x1={8} y1={106} x2={252} y2={106} stroke={C.done}
    <text x={130} y={20} textAnchor="middle" fontSize={14}
  </Svg>
);
};

export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (
    <Svg caption={step === 0 ? "Найти решение – долго (N^2)" : `N^2 = ${step} операций`}
      <ellipse cx={130} cy={64} rx={112} ry={48} fill={C.done}
      <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333"
      <text x={92} y={64} textAnchor="middle" dominantBaseline="middle"
      <text x={206} y={34} textAnchor="middle" fontSize={14}
      <circle cx={196} cy={84} r={13} fill={step === 1 ? C.done : C.bad}
      <text x={196} y={84} textAnchor="middle" dominantBaseline="middle"
      ?`}</text>
      <text x={130} y={124} textAnchor="middle" fontSize={14}
    </Svg>
  );
};

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс` : "Префикс"}
  );
};

```

```

        });
      })
    })
    <text x={130} y={122} textAnchor="middle" fontSize={16}>
      каждая клетка считается один раз и переиспользуется
    </text>
  </Svg>
};
};
};

```

ФАЙЛ 51/92: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
  AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
  SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"
  | "segment-tree" | "bridge" | "articulation" | "mst"
  | "prefix-function" | "dp" | "floyd" | "components"
  | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,

```

```
        onMouseLeave={() => setPaused(false)}
      >
        <Cmp />
      </div>
    </DemoPauseContext.Provider>
  );
};
```

ФАЙЛ 52/92: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
```

```
interface Props {
  vizId: string | null;
  onClose: () => void;
}
```

```
/** Модальное окно с полноценным симулятором – вызывает
```

```
export const SimulatorModal: React.FC<Props> = ({ vizId
```

```
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);
```

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}
```

```
interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
  onCardLeave: () => void;
}
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor,
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: number}>({
```

```
  useLayoutEffect(() => {
    if (!anchor) {
      setPos(null);
      return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300

    let left = anchor.rect.left + anchor.rect.width / 2
```

```

    }}
    role="dialog"
  >
  <div className="bg-slate-900 border border-indigo-500" >
    <div className="flex items-start gap-2 px-3 pt-3" >
      <h4 className="text-sm font-bold text-indigo-400" >Подсказка
      <button
        onClick={onClose}
        className="text-slate-500 hover:text-white"
        aria-label="Заккрыть подсказку"
      >
        <X className="w-4 h-4" />
      </button>
    </div>

    <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
      <p className="text-[13px] leading-relaxed text-slate-300">
        {entry.demo && <MiniDemo kind={entry.demo} />}

        {entry.formal && (
          <p className="text-[11.5px] leading-relaxed text-slate-300">
            {entry.formal}
          </p>
        )}

        {entry.complexity && (
          <div className="text-[11px] font-mono text-slate-300">
            Сложность: {entry.complexity}
          </div>
        )}

      <div className="p-3" >
        {viz && simulatorId && (
          <button
            onClick={() => onOpenSimulator(simulatorId)}
            className="w-full flex items-center justify-center text-white rounded-lg py-2 transition-colors"
          >

```

```

    { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx:
    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight:
    { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight:
    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight:
  ];

const isNodeVisible = (id: string) => {
  if (id === 'S' && (step === 0 || step === 7)) return true;
  return false;
};

const getPotential = (id: string) => {
  if (step >= 2 && step < 7) {
    if (id === 'S') return 0;
    if (id === 'A') return 0;
    if (id === 'B') return -2;
    if (id === 'C') return -1;
  }
  return null;
};

const getEdgeColorClass = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "opacity-0";
    if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
    if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
    return "stroke-slate-700 stroke-dasharray-4";
  }

  if (e.id === 'AB') {
    if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 4) return "stroke-emerald-500";
  }

  if (e.id === 'BC') {
    if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 5) return "stroke-emerald-500";
  }

  if (e.id === 'CA') {

```

```

        return e.newWeight;
    }
    if (e.id === 'CA') {
        if (step < 6) return e.weight;
        if (step === 6) return `${e.weight} + (${ -1
        return e.newWeight;
    }
    return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
    if (e.id === 'AB' && step === 4) return true;
    if (e.id === 'BC' && step === 5) return true;
    if (e.id === 'CA' && step === 6) return true;
    return false;
};

const isEdgeTextEmerald = (e: any) => {
    if (e.id === 'AB' && step > 4) return true;
    if (e.id === 'BC' && step > 5) return true;
    if (e.id === 'CA' && step > 6) return true;
    return false;
};

const getDesc = () => {
    switch (step) {
        case 0: return (
            <div>
                <b>Исходный граф.</b> Имеется ребро
                Если мы запустим быстрый алгоритм Д
            </div>
        );
        case 1: return (
            <div>
                <b>Шаг 1: Точка отсчета.</b><br/>
                Добавляем фиктивную вершину S. Соед
            ми</span>.
                Это наша база для расчета потенциал

```

```

    );
    case 7: return (
      <div className="text-emerald-300">
        <b>Готово!</b> Фиктивную вершину S
        Все новые веса ребер в графе стали
        При этом старые кратчайшие пути оста
шин!
      </div>
    );
    default: return "";
  }
};

return (
  <div className="bg-slate-800 p-4 rounded-xl border">
    <h4 className="text-sm md:text-base font-bo">
      <span className="w-2.5 h-2.5 bg-amber-400">
        Визуализация: Перевзвешивание алгоритмом
      </h4>

    <div className="flex flex-col md:flex-row gap-4">
      <div className="bg-[#0f172a] flex-1 min-h-100 p-4">
        <svg width="400" height="300" class="dark">
          <defs>
            <marker id="arrow-slate" viewBox="0 0 10 5"
              start-reverse">
              <path d="M 0 0 L 10 5 L 10 0" />
            </marker>
            <marker id="arrow-slate-dim" viewBox="0 0 10 5"
              uto-start-reverse">
              <path d="M 0 0 L 10 5 L 10 0" />
            </marker>
            <marker id="arrow-amber" viewBox="0 0 10 5"
              start-reverse">
              <path d="M 0 0 L 10 5 L 10 0" />
            </marker>
            <marker id="arrow-pink" viewBox="0 0 10 5"
              start-reverse">
              <path d="M 0 0 L 10 5 L 10 0" />
            </marker>
          </defs>
          <g>
            <path class="arrow-slate" d="M 0 0 L 10 5 L 10 0" />
            <path class="arrow-slate-dim" d="M 0 0 L 10 5 L 10 0" />
            <path class="arrow-amber" d="M 0 0 L 10 5 L 10 0" />
            <path class="arrow-pink" d="M 0 0 L 10 5 L 10 0" />
          </g>
        </svg>
      </div>
    </div>
  </div>
);

```



```

                                textEmer
                                isS ? 'fi
                                `}>
                                {getEdgeContent
                                </text>
                                </g>
                                )
                                }}}

    { /* Nodes */
    {nodes.map(n => {
      if (!isNodeVisible(n.id)) r
      const pot = getPotential(n.

      return (
        <g key={n.id} className=
          <circle
            cx={n.x} cy={n.y}
            className={`str
              ${n.id === 'S'
                ? 'fill-[#2
                : 'fill-sla
            />
          <text x={n.x} y={n.y}
- white">
            {n.label}
          </text>

          {pot !== null && (
            <g transform={`
fade-in zoom-in duration-500">
              <rect x="-100
k-500 stroke-1" />
              <text x="0"
font-bold fill-pink-400">
                {pot}
              </text>
            </g>

```

```
                {Array.from({length: MAX_STEP + 1,
                <div key={i} className={`w-125' : i < step ? 'bg-indigo-900' : 'bg-slate-500'
                }}}
            </div>
        </div>
    </div>
</div>
);
}
```

ФАЙЛ 55/92: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...";
import {
    Play,
    Pause,
    RotateCcw,
    Layers,
} from "lucide-react";

interface GNode {
    id: number;
    x: number;
    y: number;
    label: string;
}

interface GEdge {
    u: number;
    v: number;
}
```

```

    .filter(({ step }) => {v === null || step.currentNode === null})
    return candidates.at(-1)?.index ?? null;
  });
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  // Generate step-by-step Kosaraju steps
  const generatedSteps: AlgoStep[] = [];
  const visited = new Set<number>();
  const order: number[] = [];
  const sccMap: Record<number, number> = {};

  const recordStep = (
    phase: AlgoStep["phase"],
    desc: string,
    currentNode: number | null,
    transposed: boolean,
    currentSccId: number,
  ) => {
    generatedSteps.push({
      phase,
      desc,
      visited: new Set(visited),
      currentNode,
      order: [...order],
      transposed,
      sccMap: { ...sccMap },
      currentSccId,
      activeEdges: transposed
        ? initialEdges.map((e) => ({ u: e.v, v: e.u }))
        : [...initialEdges],
    });
  };

  recordStep(
    "init",
    "Инициализация алгоритма Косарайю. Начинаем первый шаг",
    null,
  );
}, []);

```

```

        dfs1(i);
    }
}

const topoOrder = [...order].reverse();
recordStep(
    "transpose",
    `Первый проход завершен! Стек выхода в топологическ
    ебра.`,
    null,
    true,
    -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
    visited.add(u);
    sccMap[u] = currentScc;
    recordStep(
        "dfs2",
        `DFS 2: заходим в ${u} на транспонированном гра
        u,
        true,
        currentScc,
    );

    for (const to of adjT[u]) {
        if (!visited.has(to)) {
            dfs2(to, currentScc);
        }
    }
}

```

```

const step = steps[currentStepIdx] || {
  phase: "init",
  desc: "Загрузка...",
  visited: new Set(),
  currentNode: null,
  order: [],
  transposed: false,
  sccMap: {},
  currentSccId: -1,
  activeEdges: initialEdges,
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1)
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1500);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getSccColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord

```

```

        viewBox="0 0 10 10"
        refX="22"
        refY="5"
        markerWidth="6"
        markerHeight="6"
        orient="auto-start-reverse"
    >
        <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
<marker
    id="arrow-active"
    viewBox="0 0 10 10"
    refX="22"
    refY="5"
    markerWidth="6"
    markerHeight="6"
    orient="auto-start-reverse"
>
    <path d="M 0 1 L 10 5 L 0 9 z" fill="#1
</marker>
<marker
    id="arrow-transposed"
    viewBox="0 0 10 10"
    refX="22"
    refY="5"
    markerWidth="6"
    markerHeight="6"
    orient="auto-start-reverse"
>
    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
</defs>

{/* Edges */}
{initialEdges.map((edge, idx) => {
    const u = initialNodes.find((n) => n.id =
    const v = initialNodes.find((n) => n.id =

```

```

        y2={y2 + dy}
        stroke={stroke}
        strokeWidth="2.5"
        markerEnd={`url(#${markerId})`}
        className="transition-all duration-300"
      />
    );
  }}}

{/* Nodes */}
{initialNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited =
    step.visited.has(node.id) ||
    (step.phase === "dfs1" && step.currentNode === node.id);
  const sccId = step.sccMap[node.id];

  let classes = "fill-slate-800 stroke-slate-800";
  if (sccId) {
    classes = getSccColor(sccId);
  } else if (isCurrent) {
    classes = "fill-amber-600 stroke-amber-600";
  } else if (isVisited && step.phase === "dfs2") {
    classes = "fill-emerald-700 stroke-emerald-700";
  }

  return (
    <g key={node.id}>
      <circle
        cx={node.x}
        cy={node.y}
        r="20"
        className={`transition-all duration-300`}
        strokeWidth="3"
      />
      <text
        x={node.x}
        y={node.y}

```

```
<h4 className="text-xs font-bold text-slate-400">
  Порядок обхода и стек
</h4>
```

```
<div className="bg-slate-900 border border-slate-800 p-4">
  <p className="text-xs text-white leading-relaxed">
    {step.desc}
  </p>
</div>
```

```
<div className="flex-1 space-y-4 overflow-y-auto">
  {/* Top-Sort output / Order stack representation */}
  <div>
    <span className="text-[10px] text-slate-500">
      СТЕК ВЫХОДА (DFS 1 ORDER):
    </span>
    <div className="flex flex-wrap gap-1 bg-slate-900 p-4">
      {step.order.length === 0 ? (
        <span className="text-slate-600 text-sm">Пустой стек</span>
      ) : (
        [...step.order].reverse().map(({val, idx}) => {
          <div
            key={idx}
            className="bg-indigo-950 border border-indigo-800 p-2 text-white text-sm flex items-center gap-1"
          >
            {val}
          </div>
        })
      )}
    </div>
  </div>
```

```
{/* List of actions log */}
<div>
  <span className="text-[10px] text-slate-500">
    ЛОГ ОПЕРАЦИЙ:
  </span>
```



```
        </div>
      </div>
    </div>
  );
};
```

ФАЙЛ 56/92: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';
import { useVizRuntime, vizNumber, vizString } from './vizRuntime';

interface Vertex {
  id: number;
  label: string;
  x: number;
  y: number;
  color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}

interface Edge {
  id: string;
  u: number;
  v: number;
  weight: number;
  status: 'pending' | 'inspecting' | 'included' | 'rejected';
  color: string;
}

interface StepLog {
  title: string;
  description: string;
  type: 'info' | 'success' | 'warning' | 'error';
}
```

```
// — Синхронизация с Python-компилятором: i — индекс
// списке, u/v — метки вершин, w — вес. Подсвечиваем
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveW = vizNumber(vars?.w);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveEdges: Edge[] = (() => {
  if (liveI === null && liveW === null && liveU === null && liveV === null)
    const target =
      (liveI !== null && initialEdges[liveI]) ||
      (liveU && liveV
        ? initialEdges.find(
            (e) =>
              (initialVertices[e.u].label === liveU &&
               initialVertices[e.v].label === liveV &&
               e.w === liveW)
          : null) ||
      (liveW !== null ? initialEdges.find((e) => e.w === liveW) : null);
  if (!target) return edges;
  return edges.map((e) => (e.id === target.id && e.status === 'active' ? e : null));
})();
const compilerLinked = liveI !== null || liveW !== null;

const [logs, setLogs] = useState<StepLog[]>([
  {
    title: 'Введение в раскраску (Краскал)',
    description: '«Краскал — это раскраска?». Представим, что мы хотим начать!',
    type: 'info',
  },
]);

// --- KRUSKAL STEPS ---
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
```

```

() => {
  setEdges(prev => prev.map(e => e.id === '1-2' ? {
  setLogs(prev => [{
    title: 'Шаг 3: Берем ребро B-C (вес 4)',
    description: 'Оно соединяет красную вершину B и
    type: 'info'
  }, ...prev]));
},
// Step 6: Include B-C
() => {
  setVertices(prev => prev.map(v => v.id === 2 ? {
  setEdges(prev => prev.map(e => e.id === '1-2' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро B-C в осто́ве',
    description: 'Вершина C перекрашена в красный ц
    type: 'success'
  }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
  setLogs(prev => [{
    title: 'Шаг 4: Берем ребро A-D (вес 5)',
    description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
      ю — мы перекрашиваем всё в один цвет!"',
    type: 'warning'
  }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
  setVertices(prev => prev.map(v => v.color === 'bl
  setEdges(prev => prev.map(e => e.status === 'incl
  setLogs(prev => [{
    title: 'Слияние кланов: Ребро A-D добавлено',
    description: 'Мы объединили красный и синий клан
    type: 'success'
  }, ...prev]));
},

```

```

() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Берем ребро D-G (вес 8)',
    description: 'Оно соединяет фиолетовую вершину I
    type: 'info'
  }, ...prev]));
},
// Step 14: Include D-G
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро D-G в осто́ве',
    description: 'Вершина G окрашена в фиолетовый.'
    type: 'success'
  }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '2-5' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Берем ребро C-F (вес 9)',
    description: 'Оно соединяет вершины C и F, кото
    type: 'warning'
  }, ...prev]));
},
// Step 16: Reject C-F
() => {
  setEdges(prev => prev.map(e => e.id === '2-5' ? {
  setLogs(prev => [{
    title: 'Отказ: Цикл обнаружен!',
    description: 'Выбрасываем ребро C-F.',
    type: 'error'
  }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {

```

```

        setLogs(prev => [{
            title: 'Шаг 11: Берем ребро H-I (вес 12)',
            description: 'Соединяет фиолетовую H и последнюю I',
            type: 'info'
        }, ...prev]));
    },
    // Step 22: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
            id: 8,
            color: 'purple'
        } : v), ...prev);
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
            id: '7-8',
            weight: 12,
            color: 'purple'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Успех: MST построено Краскалом!',
            description: 'Все 9 вершин окрашены в единый фиолетовый цвет. Суммарный вес: 4 + 5 + 7 + 8 + 10 + 12 = 51.',
            type: 'success'
        }, ...prev]));
    },
];

// --- PRIM STEPS ---
const primSteps = [
    // Step 1: Start at E
    () => {
        setVertices(prev => prev.map(v => v.id === 4 ? {
            id: 4,
            color: 'blue'
        } : v), ...prev);
        setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)'], ...prev);
        setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8'].includes(e.id) ? {
            id: e.id,
            weight: e.weight,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
            description: 'Мы выбрали вершину E как стартовую вершину. Добавили ребра в приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7), E-G(8).',
            type: 'info'
        }, ...prev]));
    },
    // Step 2: Pop D-E (3)
    () => {
        setEdges(prev => prev.map(e => e.id === '3-4' ? {
            id: '3-4',
            weight: 3,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
            description: 'Ребро D-E (3) добавлено к MST. Очередь обновлена: B-E(6), E-F(7), E-G(8).',
            type: 'info'
        }, ...prev]));
    },
    // Step 3: Add D-E (3)
    () => {
        setVertices(prev => prev.map(v => v.id === 3 ? {
            id: 3,
            color: 'blue'
        } : v), ...prev);
        setEdges(prev => prev.map(e => e.id === '3-4' ? {
            id: '3-4',
            weight: 3,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 3: Добавление ребра D-E (3) к MST',
            description: 'Ребро D-E (3) добавлено к MST. Очередь обновлена: B-E(6), E-F(7), E-G(8).',
            type: 'info'
        }, ...prev]));
    },
    // Step 4: Pop B-E (6)
    () => {
        setEdges(prev => prev.map(e => e.id === 'B-E' ? {
            id: 'B-E',
            weight: 6,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 4: Куча выдает следующее дешевое ребро B-E (6)',
            description: 'Ребро B-E (6) добавлено к MST. Очередь обновлена: E-F(7), E-G(8).',
            type: 'info'
        }, ...prev]));
    },
    // Step 5: Add B-E (6)
    () => {
        setVertices(prev => prev.map(v => v.id === 6 ? {
            id: 6,
            color: 'blue'
        } : v), ...prev);
        setEdges(prev => prev.map(e => e.id === 'B-E' ? {
            id: 'B-E',
            weight: 6,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 5: Добавление ребра B-E (6) к MST',
            description: 'Ребро B-E (6) добавлено к MST. Очередь обновлена: E-F(7), E-G(8).',
            type: 'info'
        }, ...prev]));
    },
    // Step 6: Pop E-F (7)
    () => {
        setEdges(prev => prev.map(e => e.id === 'E-F' ? {
            id: 'E-F',
            weight: 7,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 6: Куча выдает следующее дешевое ребро E-F (7)',
            description: 'Ребро E-F (7) добавлено к MST. Очередь обновлена: E-G(8).',
            type: 'info'
        }, ...prev]));
    },
    // Step 7: Add E-F (7)
    () => {
        setVertices(prev => prev.map(v => v.id === 7 ? {
            id: 7,
            color: 'blue'
        } : v), ...prev);
        setEdges(prev => prev.map(e => e.id === 'E-F' ? {
            id: 'E-F',
            weight: 7,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 7: Добавление ребра E-F (7) к MST',
            description: 'Ребро E-F (7) добавлено к MST. Очередь обновлена: E-G(8).',
            type: 'info'
        }, ...prev]));
    },
    // Step 8: Pop E-G (8)
    () => {
        setEdges(prev => prev.map(e => e.id === 'E-G' ? {
            id: 'E-G',
            weight: 8,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 8: Куча выдает следующее дешевое ребро E-G (8)',
            description: 'Ребро E-G (8) добавлено к MST. Очередь обновлена: (пусто).',
            type: 'info'
        }, ...prev]));
    },
    // Step 9: Add E-G (8)
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
            id: 8,
            color: 'blue'
        } : v), ...prev);
        setEdges(prev => prev.map(e => e.id === 'E-G' ? {
            id: 'E-G',
            weight: 8,
            color: 'blue'
        } : e), ...prev);
        setLogs(prev => [{
            title: 'Шаг 9: Добавление ребра E-G (8) к MST',
            description: 'Ребро E-G (8) добавлено к MST. MST построен.',
            type: 'info'
        }, ...prev]));
    },
    // Step 10: Final MST
    () => {
        setLogs(prev => [{
            title: 'Шаг 10: Итоговый MST',
            description: 'Итоговый MST построен. Все вершины окрашены в единый фиолетовый цвет. Суммарный вес: 51.',
            type: 'success'
        }, ...prev]));
    }
];

```

```

() => {
  setEdges(prev => prev.map(e => e.id === '0-1' ? {
  setLogs(prev => [{
    title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
    description: 'Плесень стремительно бежит к вершине A.',
    type: 'info'
  }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
  setVertices(prev => prev.map(v => v.id === 1 ? {
  setEdges(prev => prev.map(e => e.id === '0-1' ? {
    eap' } : e));
  setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
  setLogs(prev => [{
    title: 'Успех: Вершина B захвачена плесенью',
    description: 'В кучу добавлено B-C(4). Ребро B-E(6) не добавлено, так как образует цикл.',
    type: 'success'
  }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
  setEdges(prev => prev.map(e => e.id === '1-2' ? {
  setLogs(prev => [{
    title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
    description: 'Плесень тянется к вершине C.',
    type: 'info'
  }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
  setVertices(prev => prev.map(v => v.id === 2 ? {
  setEdges(prev => prev.map(e => e.id === '1-2' ? {
    eap' } : e));
  setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (9)']);
  setLogs(prev => [{
    title: 'Успех: Вершина C захвачена',
    description: 'Ребро C-F(9) добавлено в кучу.',

```

```

        title: 'Успех: Вершина F захвачена',
        description: 'Ребро F-I(13) добавлено в кучу. Р',
        type: 'success'
    }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
        description: 'Плесень расширяется вниз к G.',
        type: 'info'
    }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
        eap' } : e));
    setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
    setLogs(prev => [{
        title: 'Успех: Вершина G захвачена',
        description: 'В кучу добавлено G-H(10).',
        type: 'success'
    }, ...prev]));
},
// Step 16: Pop C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
        description: 'Вершины C и F уже заражены плесенью',
        type: 'warning'
    }, ...prev]));
},
// Step 17: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {

```

```

    () => {
      setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setHeapQueue(['H-I (12)', 'F-I (13)']));
      setLogs(prev => [{
        title: 'Отказ: Игнорируем E-H',
        description: 'Выбрасываем из кучи.',
        type: 'error'
      }, ...prev]));
    },
    // Step 22: Pop H-I (12)
    () => {
      setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
          title: 'Шаг 12: Куча выдает H-I (вес 12)',
          description: 'Плесень тянется к последней чистой вершине',
          type: 'info'
        }, ...prev]));
    },
    // Step 23: Include H-I
    () => {
      setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setHeapQueue(['F-I (13 - цикл)']));
      setLogs(prev => [{
        title: 'Успех: Плесень захватила весь граф',
        description: 'Все 9 вершин поглощены. Итоговый граф - полный',
        type: 'success'
      }, ...prev]));
    },
  ];

  // --- BORUVKA STEPS ---
  const boruvkaSteps = [
    // Step 1: Five-Year Plan 1 (Inspecting outgoing)
    () => {
      // Highlight the cheapest outgoing edges for EACH node
      // Node 0(A) -> A-B(2)

```



```

        description: 'Все выбранные дороги построены ра
            оз {D, E, F, G, H, I}.',
        type: 'success'
    }, ...prev]));
},
// Step 3: Five-Year Plan 2 (Inspecting bridge betw
() => {
    // Now both kolkhozes look at all outgoing roads
    // Outgoing roads between {A, B, C} and {D, E, F,
    // A-D (5), B-E (6), C-F (9).
    // Cheapest is A-D (5).
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Вторая пятилетка: Колхозы выбирают мост
        description: 'Теперь Красный и Синий колхозы од
            -D с весом 5. Остальные мосты B-E(6) и C-F(9)
        type: 'warning'
    }, ...prev]));
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
    setVertices(prev => prev.map(v => ({ ...v, color:
    setEdges(prev => prev.map(e =>
        e.status === 'included' || e.id === '0-3' ? { .
    ));
    setLogs(prev => [{
        title: 'Объединение завершено: Один гигантский
        description: 'Оба колхоза объединились по мосту
        type: 'success'
    }, ...prev]));
},
// Step 5: Check remaining edges & mark as rejected
() => {
    setEdges(prev => prev.map(e => e.status === 'pend
    setLogs(prev => [{
        title: ' Успех: MST построен Борувкой за 2 шаг
        description: 'Все лишние ребра (B-E, E-H, C-F,
            вес: 51. Благодаря параллельности, мы потрати

```

```

    },
  ]});
} else {
  setLogs([
    {
      title: 'Введение в Коллективизацию (Борувка)',
      description: 'Логика «Борувки»: Каждая из 9 д
        чтобы объединиться в колхоз.',
      type: 'info',
    },
  ]});
}
};

```

```

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex
    case 'blue': return 'bg-blue-500 border-blue-300
    case 'purple': return 'bg-purple-600 border-purple
    case 'mold': return 'bg-emerald-500 border-emerald
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIn
        return 'bg-slate-700 border-emerald-500 text-
      }
      return 'bg-slate-700 border-slate-500 text-slate
    }
  };

```

```

const getEdgeStroke = (status: string, color: string)
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
    if (color === 'mold') return '#10b981';
  }
};

```

```

        <button
          onClick={() => handleReset('prim')}
          className={
            "flex items-center gap-1.5 px-3 py-2
+
            (activeAlgo === 'prim'
              ? 'bg-emerald-600 text-white shadow-md'
              : 'text-slate-400 hover:text-slate-500')
          >
          <Layers className="w-3.5 h-3.5" />
          <span>Прима (Билет 19)</span>
        </button>
        <button
          onClick={() => handleReset('boruvka')}
          className={
            "flex items-center gap-1.5 px-3 py-2
+
            (activeAlgo === 'boruvka'
              ? 'bg-rose-600 text-white shadow-md'
              : 'text-slate-400 hover:text-slate-500')
          >
          <Users className="w-3.5 h-3.5" />
          <span>Борувка (Билет 20)</span>
        </button>
      </div>

      <button
        onClick={() => handleReset(activeAlgo)}
        className="flex items-center justify-center gap-2 py-2
-200 font-semibold text-xs rounded-xl border"
      >
        <RotateCcw className="w-3.5 h-3.5" />
        <span>Сбросить</span>
      </button>

      <button

```



```

        dy="-1.5"
      >
        {edge.weight}
      </text>
    </g>
  );
  }}}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
  {vertices.map(vertex => {
    <div
      key={vertex.id}
      style={{ top: vertex.y + "%", left: vertex.x + "%" }}
      className={"absolute -translate-x-1/2 -translate-y-1/2"}
      font-bold text-base border-2 transition-all duration-200
      id)}
    >
      <span>{vertex.label}</span>
      {vertex.id === 4 && activeAlgo === 'prim' ? (
        <span className="text-[8px] block -mt-10px">
          MST построено с помощью {activeAlgo}
        </span>
      ) : null}
    </div>
  )}}
</div>

{stepIndex >= currentSteps.length && (
  <div className="absolute inset-x-6 bottom-6"
    shadow-xl z-20">
    <p className={"font-bold text-base " + "text-rose-300"}>
      MST построено с помощью {activeAlgo}
    </p>
    <p className="text-slate-300 text-xs mt-2">
      Общий вес минимального остова: 51. Всего ребер: 5.
    </p>
  </div>
)}
```

```

        ? 'bg-emerald-950/40 border-emera
        : log.type === 'warning'
        ? 'bg-amber-950/40 border-amber-5
        : log.type === 'error'
        ? 'bg-rose-950/40 border-rose-500
        : 'bg-slate-900/60 border-slate-70
    }
  >
    <div className="flex items-center gap-2"
      {log.type === 'success' && <CheckCi
      {log.type === 'warning' && <HelpCirc
      {log.type === 'error' && <XCircle c
      {log.type === 'info' && <Play class
      <h5 className="font-bold text-sm le
    </div>
    <p className="text-xs text-slate-300
  </div>
    )})
  </div>
</div>
</div>
</div>

```

```

{/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
  <div className="flex flex-col sm:flex-row items
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-40
      <h4 className="text-xl font-bold text-white
    </div>

  <div className="flex bg-slate-950 p-1 rounded
    <button
      onClick={() => setActiveCheatTab('kruskal
      className={"px-3 py-1.5 rounded text-xs fr
      === 'kruskal' ? "bg-indigo-600 text-white" :
    >

```

Краскал (Билет 18)

```

        <p className="text-xs text-slate-300 le
ди и красим вершины в один цвет, бракуя цик
    </div>
</div>
<div className="lg:col-span-2">
    <div className="bg-slate-950 p-4 rounded-:
        <p className="text-slate-400 text-xs for
            <Code className="w-3.5 h-3.5 text-ind
        </p>
        <pre className="text-xs text-emerald-400
            80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))`}
        </pre>
    </div>
</div>
</div>
}}

{activeCheatTab === 'prim' && {
    <div className="grid grid-cols-1 lg:grid-cols

```



```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
  {
    img: dijkstraImg,
    alt: 'Добрый робот Дейкстра',
    title: '    Добрый (Дейкстра)',
    desc: 'Быстрый, жадный, но работает только с',
    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
    rest: 'весами. Подсунь отрицательное ребро – сломает',
    complexity: ' $O((V+E) \log V)$ ',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: '    Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый – зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: ' $O(V \cdot E)$ ',
    border: 'border-rose-500/30 hover:border-rose-400/60',
    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Все-ко-Все Флойд',
    title: '    Все ко Все (Флойд)',
    desc: 'Матрица + три цикла (',
```

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы — показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedCh
  const [open, setOpen] = useState(false);

  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : "";
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escap
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
  }, []);

  return (
    <>
      <div className="lg:hidden sticky top-[57px] z-40"
        <button
          onClick={() => setOpen(true)}

```

```
        </div>
      )}
    </>
  );
};
```

ФАЙЛ 59/92: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { BookOpen, Sparkles, FileDown, FileText, Code2,
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
  /** Открыта ли боковая панель компилятора. */
  compilerOpen: boolean;
  onToggleCompiler: () => void;
}

export const Navbar: React.FC<NavbarProps> = ({ compilerOpen,
  const [busy, setBusy] = useState(false);

  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-slate-800" >
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8" >
```

```

    }` }
>
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-guide-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/guide
download="guide_all_topics.pdf"
target="_blank"
rel="noreferrer"
className="flex items-center justify-center
ver:bg-indigo-600/20 border border-indigo-500
title="Скачать PDF-конспект: текст всех тем
>
    <GraduationCap className="w-4 h-4" /> PDF (
</a>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
target="_blank"
rel="noreferrer"
className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald-500
title="Скачать полный PDF сборник всех страниц
>
    <FileDown className="w-4 h-4" /> Скачать PDF
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
target="_blank"
rel="noreferrer"
className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 transition
title="Скачать полный TXT файл со всем кодом
>

```

```

        {id:0,x:160,y:40},
        {id:1,x:270,y:100},
        {id:2,x:230,y:230},
        {id:3,x:90,y:230},
        {id:4,x:50,y:100}
    ];
    const edges = [
        [0,1],[0,2],[0,3],[0,4],
        [1,2],[1,3],[1,4],
        [2,3],[2,4],
        [3,4]
    ];
    function findPt(id:number) { return pts.f[id]; }
    function intersect(a:number,b:number,c:number,d:number) {
        if (a===c||a===d||b===c||b===d) return false;
        const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
        function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
        for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
                if (!crosses.includes(i)) crosses.push(i);
                if (!crosses.includes(j)) crosses.push(j);
            }
        }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
        const xing = crosses.includes(i);
        lines.push({key:`l${i}`,x1:{x:p1.x,y:p1.y},x2:{x:p2.x,y:p2.y},
            stroke={xing ? "#f43f5e" : "#4f46e5"}
        });
    }

```

```

    }
    return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
    && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
  }
  const crosses:number[]=[];
  for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++)
    if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
      if (!crosses.includes(i)) crosses.push(i);
      if (!crosses.includes(j)) crosses.push(j);
  }
  const lines = []; const circles = [];
  for(let i=0;i<edges.length;i++) {
    const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
    const xing=crosses.includes(i);
    lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
      stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
  }
  const labels = ["\u04141", "\u04142", "\u04143"];
  for(const p of pts) {
    circles.push(<g key={`c${p.id}`}>
      <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5"/>
      <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
    </g>);
  }
  return <>{lines}{circles}</>;
  })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-2 z-20">
  V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

```

<div className="bg-amber-950/40 border border-amber-950/40">

▲ Запомни: K_5 и $K_{3,3}$ – два кита непланарности.

Гомеоморфный K_5 или $K_{3,3}$ – он непланарен. Теорема

```

    baseVarName,
    buildDebugRunner,
    changedVars,
    DEBUG_STEP_CAP,
    orderVars,
    type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
    /** Страница, с которой синхронизируется редактор. */
    chapterId: string;
    chapterTitle: string;
    /** Размер панели (px) управляется родителем и сохран
    width?: number;
    height?: number | null;
    onWidthChange?: (w: number) => void;
    onHeightChange?: (h: number) => void;
    /** Перейти к странице пособия (посмотреть визуализац
    onOpenGuide: () => void;
    /** Скрыть панель. */
    onClose: () => void;
}

type PyodideRuntime = {
    runPythonAsync: (source: string) => Promise<unknown>;
    setStdout: (options: { batched: (message: string) =>
    setStderr: (options: { batched: (message: string) =>
    setStdin: (options: { stdin: () => string | number |
};

type PyodideModule = {

```

```

items: [
    {
        label: "arr.sort() / sorted()",
        tip: "In-place сортировка по возрастанию и новая",
        code: 'arr = [5, 2, 8, 1]\narr.sort()
              ) # копия по убыванию\n',
    },
    {
        label: "sorted(key=...)",
        tip: "Сортировка по ключу: рёбра по весу, пары",
        code: 'edges = [(3, "A", "B"), (1, "B", "C"), (2, "A',
              es)\n',
    },
],
},
{
    group: "Циклы",
    items: [
        {
            label: "for i in range(n)",
            tip: "Вложенные циклы по i и j – как обход матри",
            code: "n, m = 3, 4\nfor i in range(n):",
        },
        {
            label: "while ...",
            tip: "Классический while-счётчик – как n-1 итер",
            code: "i = 1\nwhile i < 5:\n    print(f\"итераци",
        },
        {
            label: "for i, x in enumerate(...)",
            tip: "Индекс и элемент одновременно: i – позици",
            code: "for i, x in enumerate([\\"a\\", \\"b\\", \\"c",
        },
    ],
},
{
    group: "Структуры",
    items: [

```



```

    },
  ];

/** Переживает сккрытие панели: код пользователя не теряется
let savedEditor: { code: string; template: string; chapterId: string } = {
  code: "",
  template: "",
  chapterId: ""
};

export function PythonCompiler({ chapterId, chapterTitle, chapterTemplate, compilerProps }) {
  /** Активная вкладка демонстрации страницы (у sparse-редактора)
  const [demoId, setDemoId] = useState<string | null>(null);
  const sync = useMemo(() => getPageSync(chapterId, demoId)) as Sync;
  const syncVarBases = useMemo(
    () =>
      [...new Set(
        sync.variables.flatMap((v) =>
          v.name
            .split(/\\s*(?:,|\\/)\s*/)
            .map(baseVarName)
            .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/))
      )],
    [sync]
  );
  const syncVarSet = useMemo(() => new Set(syncVarBases)) as Set<string>;
  /** Полный эталон: глаз сразу вставляет его в обычный редактор
  const template = useMemo(() => buildSyncTemplate(chapterTemplate, syncVarSet)) as string;
  /** Дефолт редактора: только инициализация демо; она не зависит от chapterId
  const initTemplate = useMemo(() => buildInitTemplate(chapterTemplate, syncVarSet)) as string;

  const [code, setCode] = useState(() => {
    if (!savedEditor) return initTemplate;
    if (savedEditor.code === savedEditor.template || savedEditor.chapterId !== chapterId)
      return savedEditor.code;
    return initTemplate;
  });
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Код выполнится");
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);

```

```

    savedEditor = { code, template: initTemplate, chapterId, [code, initTemplate, chapterId] });

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId(d)), [chapterId]);
useEffect(() => () => clearVizState(chapterId), [chapterId]);

// Смена страницы (или вкладки демо): нетронутый редактор
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
  const prev = prevTplRef.current;
  if (initTemplate === prev.base && template === prev.full) {
    prevTplRef.current = { base: initTemplate, full: template };
    setPlaying(false);
    setEditHold(null);
    setDebug(null); // трасса ссылается на строки старого кода
    if (code === prev.base || code === prev.full || code === prev.full) {
      setCode(initTemplate);
      setDesyncedFrom(null);
    } else {
      setDesyncedFrom(chapterId);
    }
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
  prevTplRef.current = { base: initTemplate, full: template };
  setCode(initTemplate);
  setDesyncedFrom(null);
  setStripTab("vars");
  setPlaying(false);
  setEditHold(null);
  setDebug(null);
  setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {

```

```

        if (scalar && !assigned.has(name) && name in after
            variables[name] = value;
            if (step?.locals[name] !== undefined) locals[name] = value;
        }
    }
    const changed = changedVars(step?.locals, locals);
    return {
        line: step?.line,
        func: step?.func,
        locals,
        variables,
        changed: Object.keys(changed).filter((name) => changed[name] !== undefined),
    };
}, []));

const publishDebug = useCallback(
    (result: DebugResult, idx: number, source: string) => {
        if (!hasViz || result.steps.length === 0) return;
        const snapshot = snapshotAt(result, idx, source);
        emitVizState(chapterId, {
            line: snapshot.line,
            func: snapshot.func,
            variables: snapshot.variables,
            changed: snapshot.changed,
        });
    },
    [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
    if (debug) publishDebug(debug.result, debug.idx, debug.source);
}, [debug, publishDebug]);

const goDebug = useCallback(
    (nextIdx: number) => {
        setDebug((d) => {

```

```

const debugRun = useCallback(async (autoPlay = false)
  if (running) return;

  const source = code;
  const input = stdin;
  const request = traceRequest;
  setRunning(true);
  setPlaying(false);
  setEditHold(null);
  setDebug(null);
  clearVizState(chapterId);
  setOutputOpen(false);
  setRuntimeMessage(runtimeReady ? "Обновляю трассу..."

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
  const runtime = await getPythonRuntime();
  setRuntimeReady(true);
  runtime.setStdout({ batched: (message) => stdout.push(message)});
  runtime.setStderr({ batched: (message) => stderr.push(message)});
  runtime.setStdin({ stdin: () => (inputLines.length > 0) ? inputLines.shift() : null });

  const raw = await runtime.runPythonAsync(buildDebugRequest(code, input));
  const result = JSON.parse(String(raw)) as DebugResult;

  setDebug({ result, idx: 0, src: source, input, request });
  setPlaying(autoPlay && result.steps.length > 1);
  const printed = [...stdout, ...stderr].join("");
  setOutput(printed || "Программа ничего не вывела");
  setRuntimeMessage(
    result.truncated
      ? `Выполнение остановлено после ${DEBUG_STEP_LIMIT} шагов`
      : "Трасса готова автоматически. ← → — шаги, Ctrl+Enter — продолжить";
  );
} catch (e) {
  console.error(e);
  setRuntimeMessage("Ошибка: " + e.message);
}

```

```

        if (e.key === " ") {
            e.stopPropagation();
            e.preventDefault();
            setPlaying((v) => !v);
        }
        if (e.key === "Escape") stopDebug();
    };
    // capture-фаза: чтобы глобальный обработчик навига
    window.addEventListener("keydown", onKey, true);
    return () => window.removeEventListener("keydown",
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
const curDebug = debug ? { step: debug.result.steps[d
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
    if (curDebugLine === undefined || !listRef.current)
        listRef.current
            .querySelector(`[data-line="${curDebugLine}"]`)
            ?.scrollIntoView({ block: "nearest" });
}, [curDebugLine]);

// производные данные шага: что изменилось, в каком п
// Строка, которую назначает текущий шаг, подсвечивае
// локали берём со следующего шага трассы (для послед
// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
    if (!debug || !curDebug?.step) return {};
    return snapshotAt(debug.result, debug.idx, debug.sr
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(cu
const debugOrdered = curDebug?.step ? orderVars(shown

/** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
    if (running) return;
    if (!debug || debug.result.steps.length === 0) {

```

```

{ /* Верхний край меняет высоту на телефоне и деск
{onHeightChange && {
  <div
    role="separator"
    aria-orientation="horizontal"
    aria-label="Потяните, чтобы изменить высоту т
    title="Потяните, чтобы изменить высоту термин
    className="absolute -top-1.5 left-0 right-0 h
    onPointerDown={(event) => {
      const el = event.currentTarget;
      el.setPointerCapture(event.pointerId);
      const startY = event.clientY;
      const startH = el.parentElement?.getBounding
      const onMove = (moveEvent: PointerEvent) =>
        const maxHeight = window.innerHeight - (w
        const minHeight = Math.min(260, maxHeight
        const next = Math.round(startH + startY -
        onHeightChange(Math.max(minHeight, Math.m
      });
      const onUp = () => {
        el.removeEventListener("pointermove", onM
        el.removeEventListener("pointerup", onUp
        el.removeEventListener("pointercancel", on
      };
      el.addEventListener("pointermove", onMove a
      el.addEventListener("pointerup", onUp as Ev
      el.addEventListener("pointercancel", onUp a
    }}
  >
    <div className="absolute left-1/2 top-1/2 h-1
      digo-400 transition-colors" />
    </div>
  })
}

```

```

{ /* Левый край меняет ширину на десктопе. */}
{onWidthChange && {
  <div
    role="separator"

```

```

<button
  type="button"
  onClick={runOrTogglePlayback}
  disabled={running}
  className={`inline-flex items-center gap-
0 ${
  playing
    ? "bg-amber-500/15 text-amber-300 hover:
      : "bg-emerald-500/15 text-emerald-300
    }`}
  aria-label={running ? "Python выполняется"
>
  {running ? <Loader2 className="h-4 w-4 an
" />}
  <span>{playbackSpeed}x</span>
</button>
</Tooltip>
<Tooltip side="bottom" content="Скопировать к
  <button
    type="button"
    onClick={() => void copyCode()}
    className="p-1.5 rounded-lg text-slate-400
    aria-label="Скопировать код"
  >
    {copied ? <Check className="w-4 h-4 text-
  </button>
</Tooltip>
<Tooltip side="bottom" content="Вернуть коммен
  <button
    type="button"
    onClick={resync}
    className="p-1.5 rounded-lg text-slate-400
    aria-label="Сбросить к шаблону страницы"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
</Tooltip>
<Tooltip side="bottom" content="Скрыть панель

```

```

        aria-label="Скорость автопрохода"
        value={playbackSpeed}
        onChange={(event) => setPlaybackSpeed(Num
        className="cursor-pointer rounded bg-slate-800
    >
    {PLAYBACK_SPEEDS.map((speed) => (
        <option key={speed} value={speed}>{speed}
    )))}
    </select>
  </label>
</div>
</div>

{/* — Мини-вкладки: переменные страницы / шпаргалка */}
<div className="shrink-0 border-b border-slate-800" >
  <div className="flex items-center gap-1 px-2 pt-2" >
    <button
      type="button"
      onClick={() => setStripTab("vars")}
      className={`inline-flex items-center gap-1.5
        stripTab === "vars" ? "bg-slate-800 text-white" : "text-slate-400"
      }` >
    >
      <Variable className="w-3.5 h-3.5" /> Переменные
    </button>
    <button
      type="button"
      onClick={() => setStripTab("hints")}
      className={`inline-flex items-center gap-1.5
        stripTab === "hints" ? "bg-slate-800 text-white" : "text-slate-400"
      }` >
    >
      <HelpCircle className="w-3.5 h-3.5" /> Подсказки
    </button>
  </div>
  <div className="ml-auto pr-1">
    <Tooltip
      side="bottom"
      content={

```



```

    >
    <button
      type="button"
      onClick={resync}
      className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
    >
      <Unlink className="w-3.5 h-3.5 shrink
        Страница сменилась – подставить её пер
      </button>
    </Tooltip>
  })
  {hasViz && (
    <div className="flex flex-wrap items-centr
      {sync.variables.map((v) => (
        <Tooltip
          key={v.name}
          side="bottom"
          content={
            <span>
              <span className="block font-bol
                <code className="font-mono te
              </span>
              <span className="block">{v.role
                {v.range && <span className="bl
            </span>
          }
        >
        <code
          tabIndex={0}
          className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emeral
        >
          {v.name}
        </code>
      </Tooltip>
    )}}
    {sync.stepNote && (

```

```

        </div>
      })
    </div>

    { /* — Редактор + ввод ————— */
    <div className="flex-1 min-h-0 flex flex-col">
      {debug ? (
        <div className="flex-1 min-h-[80px] flex flex-col">
          { /* панель управления шагами */ }
          <div className="shrink-0 flex items-center">
            <Tooltip content={playing ? `Пауза автопро
            роиграть трассу заново со скоростью ${playbackRate} раз
            л)`}>
              <button
                type="button"
                onClick={runOrTogglePlayback}
                className={`mx-0.5 inline-flex items-center justify-center
                w-500/15 hover:bg-amber-500/25`} : "text-emerald-500"
                aria-label={playing ? `Пауза автопроигрывания`}>
                {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
                <span className="text-[9px] font-bold">
                {playing ? "Пауза" : "Воспроизведение"}
                </span>
              </button>
            </Tooltip>
            <Tooltip content="В начало трассы">
              <button type="button" onClick={() => {
                runFromStart()
              }}>
                <SkipBack className="w-4 h-4" />
              </button>
            </Tooltip>
            <Tooltip content="Шаг назад (←) – автопроигрывание">
              <button type="button" onClick={() => {
                runStepBack()
              }}>
                <ChevronLeft className="w-4 h-4" />
              </button>
            </Tooltip>
            <Tooltip content="Шаг вперёд (→) – автопроигрывание">
              <button type="button" onClick={() => {
                runStepForward()
              }}>
                <ChevronRight className="w-4 h-4" />
              </button>
            </Tooltip>
          </div>
        </div>
      ) : null}
    </div>
  )}
</div>

```

```

        {hasViz ? "имена → демо" : "свободный"}
      </span>
    </Tooltip>
    <Tooltip content="Вернуться в редактор (Е
      <button type="button" aria-label="Редак
p-1 rounded-md text-rose-400 hover:text-wh
      <Square className="w-3.5 h-3.5" />
      <span className="text-[9px] font-bold
    </button>
  </Tooltip>
</div>

{/* листинг с подсветкой текущей строки */}
<div ref={listRef} className="flex-1 min-h-1
  {debug.src.split("\n").map((line, i) => {
    const ln = i + 1;
    const active = ln === curDebug?.step?.l
    return {
      <div
        key={i}
        data-line={ln}
        className={`flex whitespace-pre px-1
          active ? "bg-emerald-500/15 text-
        }`}
      >
        <span
          className={`w-7 shrink-0 select-n
            active ? "text-emerald-300" : "
          }`}
        >
          {active ? "►" : ln}
        </span>
        <span>{line || " "}</span>
      </div>
    );
  })}
</div>

```

```

        </div>
    </div>
) : (
    <textarea
      ref={editorRef}
      value={code}
      onChange={(event) => {
        setEditHold(null);
        setCode(event.target.value);
      }}
      onKeyDown={(event) => {
        if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
          event.preventDefault();
          void debugRun();
        }
        event.stopPropagation();
      }}
      spellCheck={false}
      aria-label="Код Python"
      className="flex-1 min-h-[90px] w-full resize-none focus:ring-2 focus:ring-inset focus:ring-slate-400"
      placeholder="Напишите Python-код..."
    />
  )}
<div className="shrink-0 border-t border-slate-400" >
  <label htmlFor="python-stdin" className="text-slate-400">
    stdin для input()
  </label>
  <input
    id="python-stdin"
    value={stdin}
    onChange={(event) => {
      setEditHold(null);
      setStdin(event.target.value);
    }}
    spellCheck={false}
    placeholder="строки ввода через ↵"
    className="w-full bg-transparent font-mono"
  />
</div>

```

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray, vizString } from '../
```

```
interface Customer {
  id: string;
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
  { name: 'Студент Вася', emoji: '👨🎓', color: 'from-blue',
    text: 'матана!' },
  { name: 'Кодер Семён', emoji: '👨💻', color: 'from-violet',
    text: 'рцию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
    text: 'плиз.' },
  { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
    text: '' },
  { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
    text: 'пливо для нашего НЛО!' },
  { name: 'Бизнесмен', emoji: '👔', color: 'from-slate',
    text: 'тартапы.' },
];
```

```
let counter = 3;
```

```
export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'red',
      order: 'Хочу картошку фри!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: '🍝', color: 'blue',
      order: 'Дай мне пиццу!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'green',
      order: 'Дай мне котлетку!', animState: 'idle' },
  ])
}
```

```

    ...preset,
    animState: 'in',
  });

  setQueue(prev => [...prev, newGuest]);
  addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

  setTimeout(() => {
    setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c), 450);
  }, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
  if (isServing) return;
  if (queue.length === 0) {
    setShake(true);
    addLog('⚠ DEQUEUE: В очереди никого нет! Повар скучает');
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsServing(true);
  const headGuest = queue[0];

  addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

  // Step 1: Head guest gets eating animation
  setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'eat' } : c));

  setTimeout(() => {
    // Step 2: Guest leaves with happy face, queue shifts
    setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'happy' } : c));

    setServedHistory(prev => [...prev, headGuest]);

    setTimeout(() => {
      setQueue(prev => prev.slice(1));
    }, 400);
  }, 400);
}, [queue, isServing]);

```

```

{ /* УПРАВЛЕНИЕ */ }
<div className="flex items-center gap-3 flex-wrap"
  <button
    onClick={enqueue}
    disabled={isServing}
    className="flex-1 min-w-[150px] bg-gradient-to-r
      from-slate-800 to-slate-900 opacity-40 disabled:cursor-not-allowed text-white
      hover:scale-95 transition-all cursor-pointer"
  >
    <span> Встать в очередь (ENQUEUE)</span>
  </button>

  <button
    onClick={dequeue}
    disabled={isServing || queue.length === 0}
    className="flex-1 min-w-[150px] bg-gradient-to-r
      from-slate-800 to-slate-900 opacity-40 disabled:cursor-not-allowed text-white
      hover:scale-95 transition-all cursor-pointer flex"
  >
    <span> Налить суп первому (DEQUEUE)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800
      text-white cursor-pointer border border-slate-700"
  >
    Сброс
  </button>
</div>

```

```

{ /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
<div className="grid grid-cols-1 md:grid-cols-12"
  >
    { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
    <div className={`md:col-span-9 bg-slate-900 rounded-3xl
      min-h-[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`
    >

```

```

        <p className="text-[10px]">Все сыты
    </div>
  ) : (
    displayQueue.map(({customer, idx} => {
      const isHead = idx === 0;
      const isTail = idx === displayQueue
      return (
        <div
          key={customer.id}
          className={`
            flex flex-col items-center gap-
            ${customer.animState === 'in'
              ? '10px' : '20px'}
            ${customer.animState === 'eat'
              ? 'text-center' : ''}
            ${customer.animState === 'out'
              ? 'text-left' : ''}
          `}
        >
          {/* Облачко мыслей */}
          <div className={`
            absolute -top-16 left-1/2 -tr
            8 text-center text-slate-200 leading-tight
            ${isHead ? 'opacity-100 border' : ''}
          `}>
            <div className="font-bold text-
              {customer.bubbleText}
            <div className="absolute -bot
              it rotate-45"></div>
            </div>

            {/* Персонаж */}
            <div className={`
              w-14 h-14 rounded-2xl border-1
              relative
              ${customer.color}
              ${isHead ? 'ring-4 ring-amber' : ''}
            `}>
              <span className="text-3xl">{c

            {/* Индексы HEAD/TAIL */}

```



```

        <p className="text-xs font-bold font-mono">
        <p className="text-[10px] mt-1">Здесь 6
    </div>
  ) : (
    <div className="flex flex-col gap-1.5 w-full">
      {servedHistory.map(({item, idx}) => {
        <div
          key={idx}
          className="w-full py-1.5 px-3 rounded-
0 flex items-center gap-2 text-[11px] text-r
        >
          <span>{item}</span>
        </div>
      })}
    </div>
  )}
</div>

  {/* Пол */}
  <div className="w-full h-3 bg-gradient-to-r f
</div>
</div>

  {/* ЛОГ ДЕЙСТВИЙ */}
  <div className="bg-slate-900 border border-slate-
    <div className="text-[10px] font-mono text-slate
      {log.map(({entry, idx}) => {
        <div
          key={idx}
          className={`text-xs font-mono flex items-ce
        >
          {idx === 0 ? <span className="text-indigo-5
          <span>{entry}</span>
        </div>
      })}
    </div>
  </div>
);

```

```
// Регулярное моргание рыбы
useEffect(() => {
  const blinkInterval = setInterval(() => {
    setIsBlinking(true);
    setTimeout(() => setIsBlinking(false), 200);
  }, 4000);
  return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
  setIsTalking(true);
  const timer = setTimeout(() => setIsTalking(false),
  return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
  const newNodes: { [key: number]: TrieNode } = {
    0: { id: 0, char: 'ROOT', parent: 0, children: {} }
  };
  let count = 0;

  wordList.forEach((word) => {
    let curr = 0;
    for (let i = 0; i < word.length; i++) {
      const c = word[i].toUpperCase();
      if (newNodes[curr].children[c] === undefined) {
        count++;
        newNodes[curr].children[c] = count;
        newNodes[count] = {
          id: count,
          char: c,
          parent: curr,
          children: {},
          fail: 0,
          term link: 0,

```

```

    setNodes(newNodes);
    setSimulationStep(0);
    setBfsQueue([]);
    setCurrentNode(null);
    setSpeechText(
      `Отлично! Я построил базовое префиксное дерево (БПД),
      чтобы расставить fail-ссылки!`
    );
  };

useEffect(() => {
  buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
  e.preventDefault();
  const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
  if (!cleanWord) return;
  if (words.includes(cleanWord)) {
    setSpeechText(`Слово «${cleanWord}» уже есть в слове!`);
    return;
  }
  const updated = [...words, cleanWord];
  setWords(updated);
  setNewWord('');
  buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем работать!');
    return;
  }
  const updated = words.filter((w) => w !== wordToRemove);
  setWords(updated);
  buildInitialTrie(updated);
};

```

```

let logs = `Рассматриваем вершину #${r} (${rNode.children}

for (const [char, u] of Object.entries(rNode.children)) {
  newQueue.push(u);
  let v = rNode.fail;
  while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
    v = updatedNodes[v].fail;
  }
  const failTarget = updatedNodes[v].children[char];
  updatedNodes[u].fail = failTarget;

  // Рассчитываем терминальную ссылку (term_link)
  if (updatedNodes[failTarget].is_terminal) {
    updatedNodes[u].term_link = failTarget;
  } else {
    updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
  }

  logs += `Для #${u} ('${char}') fail = #${failTarget}\n`;
}

if (childrenIds.length === 0) {
  logs += 'У этой вершины нет детей, просто переходим к следующему\n';
}

setNodes(updatedNodes);
setBfsQueue(newQueue);
setCurrentNode(r);
setSpeechText(logs);
};

return (
  <div className="bg-slate-800/90 rounded-2xl p-6 border-1px border-slate-700" >
    {/* Шапка виджета */}
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center space-x-3" >
        <span className="text-3xl" > </span>
        <div>

```

```

<path
  d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
  fill="#be123c"
  className="origin-[35px_100px] animate-
/>

{/* Плавники */}
<path d="M 90 55 L 70 30 L 110 55 Z" fill=
<path d="M 90 145 L 70 170 L 110 145 Z" f
<path
  d="M 120 115 L 100 135 L 130 130 Z"
  fill="#f87171"
  className="origin-[120px_115px] animate
/>

{/* Глаз (с морганием) */}
<circle cx="145" cy="85" r="12" fill="#ff
{isBlinking ? (
  <line x1="133" y1="85" x2="157" y2="85"
) : (
  <
    <circle cx="148" cy="85" r="6" fill="
    <circle cx="151" cy="82" r="2" fill="
  </>
)}

{/* Рот (анимация разговора) */}
{isTalking ? (
  <path d="M 165 105 Q 155 120 165 125 Q
) : (
  <path d="M 165 110 Q 155 115 168 118" s
)}

{/* Улыбка / жабры */}
<path d="M 130 80 Q 125 100 130 120" stro
<path d="M 120 85 Q 115 100 120 115" stro
</svg>

```

```

        >
        <Play className="w-4 h-4" /> Запустит
    </button>
  })
  {simulationStep === 1 && {
    <button
      onClick={stepBFS}
      className="bg-blue-600 hover:bg-blue-500
dow-lg shadow-blue-900/40 transition-transf
    >
    <span>Шаг BFS</span>
    <span className="bg-blue-800 px-1.5 p
    </button>
  })
  {simulationStep === 2 && {
    <button
      onClick={() => buildInitialTrie(words
      className="bg-indigo-600 hover:bg-ind
      transition-transform active:scale-95"
    >
    <RotateCcw className="w-4 h-4" /> Пер
    </button>
  })
</div>
</div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-s
  <h5 className="text-white font-bold mb-3 flex i
    <span> </span> Дерево и Суффиксные (fail) ссы
  </h5>

  {(() => {
    let maxX = 0;
    let maxY = 0;
    Object.values(nodes).forEach(n => {

```

```

        y={{parent.y + node.y) / 2}
        fill="#94a3b8"
        fontSize="12"
        fontWeight="bold"
    >
        {node.char}
    </text>
</g>
});
}}}

{/* Draw Fail Links */}
{simulationStep > 0 && Object.values(nodes)
    if (node.id === 0 || node.fail === 0) r
n draw all! But maybe wait, we can just draw
    // Wait, all nodes get fail link. Let's
tep >= 1 and computed it
    // To avoid too many lines, let's draw
const target = nodes[node.fail];
if (!target || !node.x || !node.y || !t

// Simple bezier curve for fail link
const isCurrent = currentNode === node.

return (
    <path
        key={`fail-${node.id}`}
        d={`M ${node.x} ${node.y - 10} Q ${
- 16}`}
        fill="none"
        stroke={isCurrent ? '#f43f5e' : 'rgb(
        strokeWidth={isCurrent ? 2 : 1.5}
        strokeDasharray="4 4"
        markerEnd="url(#fail-arrow)"
    />
    );
}}}
```

```

        >
        {isRoot ? 'R' : node.char}
      </text>
      {!isRoot && {
        <text
          x={node.x + 12}
          y={node.y - 12}
          fill="#94a3b8"
          fontSize="9"
        >
          {node.id}
        </text>
      }}
    </g>
  );
  }}}
</svg>
);
}}())}
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
  <div className="bg-slate-900/50 p-5 rounded-xl">
    <div>
      <h5 className="text-white font-bold mb-3">
        <span> </span> Словарь (Паттерны)
      </h5>
      <div className="flex flex-wrap gap-2 mb-4">
        {words.map((w) => {
          <span
            key={w}
            className="bg-slate-800 border border-slate-700 padding-2 gap-1 group"
          >
            {w}
            <button
              onClick={() => removeWord(w)}
            >

```



```

    </span>
  </h5>

  <div className="max-h-[220px] overflow-y-auto" >
    <table className="w-full text-left border-collapse" >
      <thead>
        <tr className="border-b border-slate-700" >
          <th className="py-2 px-3">ID Вершины</th>
          <th className="py-2 px-3">Символ</th>
          <th className="py-2 px-3">Родитель</th>
          <th className="py-2 px-3">fail-ссылка</th>
          <th className="py-2 px-3">term_link</th>
          <th className="py-2 px-3">Дети</th>
          <th className="py-2 px-3">Слова (dict) </th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-800" >
        {Object.values(nodes).map((node) => {
          const isCurrent = currentNode === node.id
          const isInQueue = bfsQueue.includes(node.id)

          return (
            <tr
              key={node.id}
              className={`transition-colors ${
                isCurrent
                  ? 'bg-blue-900/40 font-bold text-white'
                  : isInQueue
                  ? 'bg-slate-800/80'
                  : 'hover:bg-slate-800/40'
              }`}
            >
              <td className="py-2.5 px-3 flex items-center" >
                <span
                  className={`w-2 h-2 rounded-full ${
                    node.id === 0
                      ? 'bg-purple-500'
                      : isCurrent

```

```

                ? 'ø'
                : Object.entries(node.children)
                    .map(([c, id]) => `${c}→#` + id)
                    .join(', ')
            </td>
            <td className="py-2.5 px-3">
                {node.words.length === 0 ? (
                    <span className="text-slate-600">
                        {node.id}
                    </span>
                ) : (
                    <span className="bg-emerald-900 text-white">
                        {node.words.join(', ')}
                    </span>
                )}
            </td>
        </tr>
    </tbody>
</table>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 64/92: src/components/SegmentTreeVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizState';

```

```

// ===== TYPES =====
interface Step {

```

```

    "    }", // 4
    "    mid = (l + r) >> 1", // 5
    "    build(2v, l, mid)", // 6
    "    build(2v+1, mid+1, r)", // 7
    "    tree[v] = tree[2v]+tree[2v+1]", // 8
    "}", // 9
];
void CODE;

function go(v: number, l: number, r: number) {
    if (l === r) {
        steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
            , highlightNodes: [v], arrayHighlight: [l, r] });
        tree[v] = arr[l];
        steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
            l, r] });
        return;
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
        { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v, l, m);
    steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r})`,
        highlightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1]`,
        tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень: tree[1]`,
    Highlight: [0, n - 1] });
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
    const n = arr.length;

```

```

const steps: Step[] = [];
let id = 0;
let l = qL + n;
let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, codeLine: 3, treeState: tree, highlightNodes: [l]`});

while (l < r) {
  const highlights: number[] = [];
  if (l & 1) {
    res += tree[l] ?? 0;
    steps.push({ id: id++, desc: `l=${l} нечётный → l`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l], result: res }, { id: id++, desc: `Сумма = ${res}. l++`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l], result: res }, { id: id++, desc: `l=${l} нечётный → l`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l], result: res }, { id: id++, desc: `Сумма = ${res}. l++`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l], result: res }
    highlights.push(l);
    l++;
  }
  if (r & 1) {
    r--;
    res += tree[r] ?? 0;
    steps.push({ id: id++, desc: `r=${r + 1} нечётный`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlight: [r], result: res }, { id: id++, desc: `Сумма = ${res}. r--`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlight: [r], result: res }, { id: id++, desc: `r=${r + 1} нечётный`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlight: [r], result: res }, { id: id++, desc: `Сумма = ${res}. r--`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlight: [r], result: res }
    highlights.push(r);
  }
  l >>= 1;
  r >>= 1;
  if (l < r) {
    steps.push({ id: id++, desc: `Поднимаемся: l = ${qL + n}, codeLine: 3, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res }
    , arrayHighlight: [qL, qR], result: res }));
  }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], codeLine: 3, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res }
return steps;
}

```

```

    " while (l < r) {",
    "     if (l & 1) res += tree[l++];",
    "     // -----",
    "     if (r & 1) res += tree[--r];",
    "     l >>= 1; r >>= 1;",
    " return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(700);

  useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

  useEffect(() => {
    if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
      const t = setTimeout(() => setIdx(i => i + 1), speed);
      return () => clearTimeout(t);
    }
  }, [playing, idx, steps, speed]);

  const step = steps[idx] || steps[0];
  if (!step) return null;

  const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' } }[color];

  return { step, idx, setIdx, playing, setPlaying, speed };
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arrow }) {

```

```

return (
  <div className={`rounded-2xl border overflow-hidden
    ? 'border-emerald-500/30' : 'border-amber-500/30'}
    {/* Header */}
    <div className={`px-4 py-2.5 flex items-center justify-between
      -emerald-950/50' : 'bg-amber-950/50'} border-b border-emerald-500/30
      <h4 className="font-bold text-sm text-white flex-1" />
      <span className="text-[10px] font-mono text-slate-300" />
    </div>

    {/* Array strip */}
    <div className="px-3 py-2 bg-slate-950 border-b border-emerald-500/30"
      {arr.map((v, i) => {
        const inRange = step.arrayHighlight && i >= step.startLine;
        return (
          <div key={i} className={`flex flex-col items-center justify-between
            go' ? 'bg-indigo-950 border-indigo-500' : 'bg-emerald-950 border-emerald-500'}
            -translate-y-0.5` : 'bg-slate-900 border-slate-800'}
            <span className="text-[7px] text-slate-500" />
            <span className="text-[11px] font-bold font-mono text-white" />
          </div>
        );
      })}
    </div>

    {/* Tree */}
    <div className="flex-1 p-2 overflow-x-auto bg-slate-950"
      <div className="scale-[0.85] origin-top min-w-max" />
    </div>

    {/* Code */}
    <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono"
      x">
      {code.map((line, i) => {
        const ln = i + 1;
        const active = step.codeLine === ln;
        return (

```

```

        text-[9px] font-bold rounded px-1.5 py-1 outline
        <option value={1200}>Медл.</option>
        <option value={700}>Норма</option>
        <option value={350}>Быстро</option>
        <option value={120}>Турбо</option>
      </select>
    </div>

    {/* Progress bar */}
    <div className="h-1 bg-slate-950"><div className="
      + 1) / total) * 100 : 0}%` }} /></div>
  </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const runtime = useVizRuntime();
  const liveI = vizNumber(runtime?.variables?.i);
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
  const [customInput, setCustomInput] = useState('5, 8,');
  const [qL, setQL] = useState(1);
  const [qR, setQR] = useState(4);
  const [view, setView] = useState<'build' | 'queries'>'build';

  // Clamp query range when array changes
  useEffect(() => {
    setQL(prev => Math.min(prev, arr.length - 1));
    setQR(prev => Math.min(prev, arr.length - 1));
  }, [arr]);

  const handleApply = (e: React.FormEvent) => {
    e.preventDefault();
    const parsed = customInput.split(',').map(s => parseInt(s));
    if (parsed.length >= 2 && parsed.length <= 16) {
      setArr(parsed);
    }
  };
}

```

```

        ition-colors">Применить</button>
      </div>
      <div className="flex flex-wrap gap-1.5 pt-1" >
        {arr.map((v, i) => {
          <span key={i} className={`border px-2 py-1 text-amber-200 ring-2 ring-amber-400/30` : "border-amber-200 ring-2 ring-amber-400/30" : "border-amber-200 ring-2 ring-amber-400/30"}>
            <span className={liveI === i ? "text-amber-200" : "text-slate-500"}>{v}</span>
          </span>
        })}
        <span className="text-xs text-slate-500 font-medium">{arr.length}</span>
      </div>
    </form>

    {/** Query range */}
    <div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
      <label className="text-[10px] font-bold uppercase">Query range</label>
      <div className="flex items-center gap-2">
        <div className="flex items-center gap-1">
          <span className="text-xs text-slate-400">From</span>
          <input
            type="number"
            min={0}
            max={arr.length - 1}
            value={qL}
            onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
            className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
          />
        </div>
        <div className="flex items-center gap-1">
          <span className="text-xs text-slate-400">To</span>
          <input
            type="number"
            min={0}
            max={arr.length - 1}
            value={qR}
            onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
            className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
          />
        </div>
      </div>
    </div>
  </div>
</div>

```



```

        color="indigo"
        arr={arr}
      />
    })

{view === 'queries' && {
  <div className="grid grid-cols-1 xl:grid-cols-2">
    <SimPanel
      key={`qtd-${arr.join(' - ')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Сверху`}
      icon="↑"
      steps={queryTDSteps}
      code={QUERY_TD_CODE}
      color="emerald"
      arr={arr}
    />
    <SimPanel
      key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Снизу`}
      icon="↓"
      steps={queryBUSteps}
      code={QUERY_BU_CODE}
      color="amber"
      arr={arr}
    />
  </div>
})

{view === 'theory' && {
  <div className="space-y-5">
    {/* Why it splits this way */}
    <div className="bg-slate-950 p-5 rounded-xl border">
      <h4 className="text-base font-bold text-white">
        Почему разбиение именно так для N={arr.length}?
      </h4>
      <p className="text-sm text-slate-300 leading-relaxed">
        Каждый узел берёт <code className="bg-slate-900 px-1">
        и получает <code className="bg-slate-900 px-1">
        <code className="bg-slate-900 px-1 rounded font-mono text-emerald-400">

```

```

        <p className="text-xs text-slate-300 mb-3">
          Стартуем с двух указателей (l и r) на левом и правом
          ём его левого соседа. Поднимаемся пока l {".
        </p>
        <div className="space-y-1.5 text-xs">
          <div className="flex justify-between border-b border-slate-200">
            <div className="flex justify-between border-b border-slate-200">
              <span className="text-emerald-400 font-mono">
                <div className="flex justify-between"><span
                00 font-bold">x сложно</span></div>
              </div>
            </div>
          </div>
        </div>

    {/* Verdict */}
    <div className="bg-slate-950 p-5 rounded-xl border border-slate-800">
      <h4 className="font-bold text-blue-400 mb-2">Вердикт
      <p className="text-sm text-slate-300 leading-relaxed">
        <strong className="text-white">Итеративный подход гарантирует
        кальности. Но если нужны <strong className="text-white">рекурсивный (сверху-вниз)<
      </p>
    </div>
  </div>
  )}
</div>
);
}

```

ФАЙЛ 65/92: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";

```



```
/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерево
 * поднять отмеченный узел в корень. Клик по узлу = один
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */

export interface TNode {
  key: number;
  left: TNode | null;
  right: TNode | null;
}

const leaf = (key: number): TNode => ({ key, left: null, right: null })

const clone = (t: TNode | null): TNode | null =>
  t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
  {
    name: "Линия влево",
    hintCase: "Zig-Zig",
    target: 1,
    build: () => {
      const n1 = leaf(1);
      const n2: TNode = { key: 2, left: n1, right: leaf(3) };
      const n4: TNode = { key: 4, left: n2, right: leaf(5) };
      return { key: 6, left: n4, right: leaf(7) };
    },
  },
  {
    name: "Змейка",
    hintCase: "Zig-Zag",
    target: 3,
```

```

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return root;

    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const gg = path.length >= 3 ? path[path.length - 3] : null;

    if (p.left === x) {
        p.left = x.right;
        x.right = p;
    } else {
        p.right = x.left;
        x.left = p;
    }

    if (!gg) return x;
    if (gg.left === p) gg.left = x;
    else gg.right = x;
    return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
    const path = findPath(root, key);
    if (path.length < 2) return null;
    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const g = path.length >= 3 ? path[path.length - 3] : null;

    if (!g) return { node: x.key, kind: "Zig" };

    const xLeft = p.left === x;
    const pLeft = g.left === p;
    if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
    return { node: x.key, kind: "Zig-Zag (сначала нижняя" };
}

```

 /* ----- КОМПОНЕНТ -----

```
export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.build());
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; correct: boolean }>([]);
  const [showAnswer, setShowAnswer] = useState(false);

  const target = preset.target;
  const solved = tree.key === target;

  const reset = useCallback(
    (idx = presetIdx) => {
      setPresetIdx(idx);
      setTree(PRESETS[idx].build());
      setHistory([]);
      setMoves([]);
      setShowAnswer(false);
    },
    [presetIdx]
  );

  const { nodes, edges, width, height } = useMemo(() => preset.build(), [preset]);
  const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
  const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)), [tree, target]);

  const clickNode = (key: number) => {
    if (solved) return;
    const p = findPath(tree, key);
    if (p.length < 2) return; // корень крутить не с кем

    const want = canonicalNext(tree, target);
    setHistory((h) => [...h, clone(tree) as TNode]);
    setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
    setTree((t) => rotateUp(clone(t) as TNode, key));
  };
}
```

```

        i === presetIdx ? "bg-indigo-600 text-white" :
      }` }
    >
    {p.name}
  </button>
)}}
</div>

```

```

<div className="bg-slate-900 rounded-xl border border-slate-700" >
  <svg
    viewBox={`0 0 ${width} ${height}`}
    className="h-auto block mx-auto"
    style={{ minWidth: Math.min(width * 1.5, 420) }}
    role="img"
  >
    {edges.map(([a, b], i) => {
      const na = nodes.find((n) => n.key === a);
      const nb = nodes.find((n) => n.key === b);
      if (!na || !nb) return null;
      const onPath = path.has(a) && path.has(b);
      return (
        <line
          key={i}
          x1={na.x}
          y1={na.y}
          x2={nb.x}
          y2={nb.y}
          stroke={onPath ? "#6366f1" : "#475569"}
          strokeWidth={onPath ? 3 : 1.8}
          style={{ transition: "all 700ms cubic-bezier(0.16, 1, 0.3, 0.9)" }}
        />
      );
    })}
  </div>

```

```

{nodes.map((n) => {
  const isTarget = n.key === target;
  const clickable = !solved && n.depth > 0;
  return (
    <div
      key={n.key}
      className={isTarget ? "bg-indigo-600 text-white" : "bg-slate-900 text-slate-400"}
      style={{ width: 40px, height: 40px, display: flex, align-items: center, justify-content: center, border: 1px solid #33415d, border-radius: 5px }}
    >
      {n.key}
    </div>
  );
})}

```



```

        className="flex items-center gap-1.5 px-3 py-3
        ext-xs font-bold transition-colors"
    >
        <RotateCcw className="w-3.5 h-3.5" /> Заново
    </button>
    <button
        onClick={() => setShowAnswer((s) => !s)}
        className={`flex items-center gap-1.5 px-3 py-3
        showAnswer
            ? "bg-amber-600/20 border-amber-500 text-amber-500"
            : "bg-slate-800 border-slate-700 text-slate-500"
        }`
    >
        {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
        {showAnswer ? "Скрыть разбор" : "Сдаться / посмотреть"}
    </button>

    <span className="text-[11px] text-slate-500 ml-2">
        поворотов: {moves.length} • минимум: {minimalMoves}
    </span>
</div>

{showAnswer && !solved && expected && (
    <div className="mt-3 text-[12px] bg-amber-950/40 p-3">
        Сейчас splay крутил бы узел{" "}
        <span className="font-mono font-bold">{expected}
    </div>
)}

{solved && (
    <div
        className={`mt-3 rounded-lg border px-3 py-2.5
        wrong === 0
            ? "bg-emerald-950/40 border-emerald-700/60 text-emerald-500"
            : "bg-rose-950/30 border-rose-800/60 text-rose-500"
        }`
    >
        <div className="flex items-center gap-2 font-

```

```
* Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
*
* Что сделано ради понятности:
*   * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
*   * сразу видно, что дерево остаётся деревом поиска;
*   * под деревом печатается обход слева направо: он ОДНОЭТАПНЫЙ;
*   * это и есть доказательство, что поворот ничего не ломает;
*   * кадр «прицел» ничего не двигает, только подсвечивает;
*   * на кадре «результат» остаются призраки – пунктирные узлы в
*     местах и стрелки «откуда → куда»;
*   * по умолчанию анимация НЕ крутится сама: пользователь
*     идёт в своём темпе. Автопрокрутка включается кнопкой.
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел». */
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
  moved?: NodeId[];
  ms: number;
}
```

```
interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах. */
}
```

```

        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [160, 50], A: [80, 130], p: [245, 130],
        edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
        kind: "result",
        title: "Стало: 20 – корень",
        text: "40 стало правым сыном двадцатки. Поддерев
            ева от 40 – это одно и то же место.",
        moved: ["x", "p", "B"],
        ms: RESULT_MS,
    },
    ],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
    pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
    edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
    pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
    edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
    key: "zig-zig",
    label: "Zig-Zig",
    when: "x и p – дети с ОДНОЙ стороны (оба слева или об
    rotations: "2 поворота, первым – верхний",
    keys: { x: 20, p: 40, g: 60 },
    inorder: ["A", "20", "B", "40", "C", "60", "D"],
    ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
    frames: [
        {
            ...ZZ_S0,

```

```

        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    },
],
};

/* ----- Zig-Zag -----
const ZG_S0 = {
    pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
    edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
    pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
    edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
    key: "zig-zag",
    label: "Zig-Zag",
    when: "x и p – дети с РАЗНЫХ сторон (змейка)",
    rotations: "2 поворота, первым – нижний",
    keys: { x: 40, p: 20, g: 60 },
    inorder: ["A", "20", "B", "40", "C", "60", "D"],
    ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
    frames: [
        {
            ...ZG_S0,
            kind: "start",
            title: "Было: 60 → влево на 20 → вправо на 40",
            text: "Узлы идут «змейкой»: сначала шаг влево, по
                ",
            ms: RESULT_MS,
        },
        {
            ...ZG_S0,
            kind: "aim",
            title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",

```

```

g: { fill: "#f59e0b", stroke: "#fcd34d" },
sub: { fill: "#1e293b", stroke: "#475569" },
};

const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "х — ищем его",
  p: "р — родитель",
  g: "г — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])

  return (
    <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
      <defs>
        <marker id="splay-arrow" viewBox="0 0 10 10" re
          <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
        </marker>
      </defs>

    /* призраки: где узел стоял до поворота */
    {ghosts.map((id) => {
      const from = prev[id]!;
      const to = pos[id]!;
      const r = isSubtree(id) ? 15 : 20;
      const dx = to[0] - from[0];
      const dy = to[1] - from[1];
      const len = Math.hypot(dx, dy) || 1;
      const pad = r + 8;
      return (
        <g key={`ghost-${id}`} opacity={0.55}>
          <circle cx={from[0]} cy={from[1]} r={r} fil
            {len > pad * 2 + 6 && (

```

```

const sub = isSubtree(id);
const c = COLOR[sub ? "sub" : id];
const r = sub ? 15 : 20;
const hot = Boolean(focus && focus.includes(id));
return (
  <g
    key={id}
    style={{
      transform: `translate(${p[0]}px, ${p[1]}px)`,
      transition: MOVE,
      opacity: dim(id) ? 0.28 : 1,
    }}
  >
    {hot && <circle r={r + 7} fill="none" stroke="red" />}
    <circle r={r} fill={c.fill} stroke={c.stroke} />
    <text
      textAnchor="middle"
      dominantBaseline="central"
      fontSize={sub ? 12 : 15}
      fontWeight="700"
      fill={sub ? "#94a3b8" : "#fff"}
    >
      {sub ? id : keys[id]}
    </text>
    {!sub && (
      <text textAnchor="middle" y={-r - 7} font-size={10}
        >{id}</text>
    )}
  </g>
);
}})
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const runtime = useVizRuntime();

```

```

<div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800" style={{borderRadius: 24px}}>
  <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar" style={{padding: 10px}}>
    {CASES.map((c, i) => (
      <button
        key={c.key}
        onClick={() => setCaseIdx(i)}
        className={`px-3 sm:px-4 py-2 rounded-lg text-sm transition-colors
          ${i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"}
        `}
      >
        {c.label}
      </button>
    ))}
  </div>

```

```

<div className="mb-3 text-xs sm:text-[13px] text-slate-400" style={{padding: 10px}}>
  <span className="font-bold text-indigo-300">Корпус</span>
  <span className="text-slate-500">({active.rotation}°)</span>
</div>

```

/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение

```

<div className="mb-3 rounded-xl border border-slate-800" style={{padding: 10px}}>
  <div className="flex items-center gap-2 mb-1 flex-wrap" style={{padding: 5px}}>
    <span
      className={`text-[10px] font-bold uppercase transition-colors
        ${current.kind === "aim"
          ? "bg-amber-500/20 text-amber-300"
          : current.kind === "start"
            ? "bg-slate-700 text-slate-300"
            : "bg-indigo-500/20 text-indigo-300"}
      `}
    >
      {current.kind === "aim" ? "прицел" : current.kind}
    </span>
    <span className="text-sm font-bold text-white" style={{marginLeft: 10px}}>
      Шаг {idx + 1} из {total}. {current.title}
    </span>
  </div>

```

```

<div className="flex flex-wrap items-center gap-2"
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f - 1 + total) % total);
    }}
    disabled={idx === 0}
    className="px-3 py-2 rounded-lg bg-slate-800 text-white
      d: hover:text-slate-300 text-xs font-bold transition-colors"
  >
    ← Назад
  </button>
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f + 1) % total);
    }}
    className="px-4 py-2 rounded-lg bg-indigo-600 text-white
      w-indigo-900/40"
  >
    {last ? "⏮ Сначала" : "Дальше →"}
  </button>
  <button
    onClick={() => setPlaying((p) => !p)}
    className="flex items-center gap-1.5 px-3 py-2 text-white
      text-xs font-bold transition-colors"
  >
    {playing ? <Pause className="w-3.5 h-3.5" /> : <Play />}
    {playing ? "Стоп" : "Авто"}
  </button>
  <button
    onClick={() => setSlow((s) => !s)}
    title="Автопрокрутка ещё медленнее"
    className={`flex items-center gap-1.5 px-3 py-2 text-white
      ${slow
        ? "bg-emerald-600/20 border-emerald-500 transition-colors"
        : "bg-slate-800 border-slate-700 text-slate-300"}
    `}
  >
    {slow ? "⏸ Медленнее" : "Авто"}
  </button>
</div>

```



```
import React, { useEffect, useMemo, useRef, useState }
import { useVizRuntime, vizNumber } from "../data/vizState"
```

```
/**
```

```
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
```

```
 *
```

```
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот
```

```
 *   aim – прицел на тройку g–p–x, назван случай;
```

```
 *   cut – отстёгиваем среднее поддереву β (пунктир, поворот)
```

```
 *   rot – поворот: зелёным подсвечены рёбра, которых не было
```

```
 * Рядом с логом всегда висит врезка «Механика одного поворота»
```

```
 * На каждом кадре – живая автопроверка in-order (сортировка)
```

```
 *
```

```
 * Чистые функции (splaySteps, rotateUp, bstInsert, cloneWithParents)
```

```
 */
```

```
export interface SN {
```

```
  key: number;
```

```
  left: SN | null;
```

```
  right: SN | null;
```

```
  parent: SN | null;
```

```
}
```

```
export function cloneWithParents(t: SN | null): SN | null {
```

```
  if (!t) return null;
```

```
  const n: SN = { key: t.key, left: null, right: null, parent: null };
```

```
  const l = t.left ? cloneWithParents(t.left) : null;
```

```
  const r = t.right ? cloneWithParents(t.right) : null;
```

```
  if (l) {
```

```
    n.left = l;
```

```
    l.parent = n;
```

```
  }
```

```
  if (r) {
```

```
    n.right = r;
```

```
    r.parent = n;
```

```
  }
```

```
  return n;
```

```

    }
    child.parent = g;
    if (!g) return;
    if (g.left === p) g.left = child;
    else g.right = child;
}

/** Рёбра в виде "p→c" — для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
    const es: string[] = [];
    const walk = (n: SN | null) => {
        if (!n) return;
        if (n.left) es.push(`${n.key}→${n.left.key}`);
        if (n.right) es.push(`${n.key}→${n.right.key}`);
        walk(n.left);
        walk(n.right);
    };
    walk(t);
    return es.sort();
}

const diffEdges = (before: string[], after: string[]):
    const rm = before.filter((e) => !after.includes(e));
    const ad = after.filter((e) => !before.includes(e));
    const parts: string[] = [];
    for (const a of ad) {
        const [p, c] = a.split("→");
        if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}
        else parts.push(`поддереву ${c} переехало к ${p}`);
    }
    return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
    kind: SplayStepKind;
    root: SN | null;
    aimEdge?: [number, number];
}

```

```

    kind: "search",
    root: cloneWithParents(root)!,
    pathKeys: [...pathKeys],
    note: isInsert
      ? `Вставили ${key} обычным BST-спуском. Теперь по
      : found
      ? `Спуск по ключу ${key}: путь ${pathKeys.join(
      : `Ключа ${key} нет. Спуск уперся в ${last.key}
    rotCount: 0,
  ));

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
  while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string) => {
  steps.push({
    kind: "aim",
    root: cloneWithParents(root)!,
    aimEdge,
    caseLabel: label,
    note: aimNote,
    rotCount,
  });
  const p = child.parent!;
  const mid = p.left === child ? child.right : child.left;
  steps.push({
    kind: "cut",
    root: cloneWithParents(root)!,
    cutEdge: [child.key, mid ? mid.key : child.key],
    cutLabel: mid ? `β = поддерево ${mid.key}` : "β не найдено",
    cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : null,
    caseLabel: label,
    note: mid
      ? `Отстёгиваем среднее поддерево β = ${mid.key}

```

```

        `Zig-Zig (${side}: линия ${g.key}—${p.key}—${target.key}, а ${target.key} пока не трогаем.`),
    );
    steps.push({
      kind: "aim",
      root: cloneWithParents(root)!,
      aimEdge: [target.parent.key, target.key],
      caseLabel: "Zig-Zig",
      note: `Теперь обычный зиг: ребро ${target.parent.key}—${target.key}`,
      rotCount,
    });
    emitRotation(
      target,
      "Zig-Zig",
      [target.parent.key, target.key],
      `Доводим: ${target.key} над ${target.parent.key}`,
    );
  } else {
    const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}`;
    emitRotation(
      target,
      "Zig-Zag",
      [p.key, target.key],
      `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: п...`,
    );
    steps.push({
      kind: "aim",
      root: cloneWithParents(root)!,
      aimEdge: [target.parent.key, target.key],
      caseLabel: "Zig-Zag",
      note: `Второе ребро: ${target.parent.key}—${target.key}`,
      rotCount,
    });
    emitRotation(
      target,
      "Zig-Zag",
      [target.parent.key, target.key],
      `Доводим: ${target.key} над ${target.parent.key}`,
    );
  }
}

```

```

{ line: 2, text: "    p = parent(x); g = parent(p)" },
{ line: 3, text: "    if g is None:                                # Z"
{ line: 4, text: "        rotate(x over p)" },
{ line: 5, text: "    elif p,g,x на одной линии:                                # "
{ line: 6, text: "        rotate(p over g)                                # B"
{ line: 7, text: "        rotate(x over p)" },
{ line: 8, text: "    else:                                                # Z"
{ line: 9, text: "        rotate(x over p)                                # H"
{ line: 10, text: "        rotate(x over g)" },
];

```

/* ----- врезка «механика одного по

```

const MechanismInset: React.FC<{ step: SplayStep | null
  const isCut = step?.kind === "cut";
  const isRot = step?.kind === "rot";
  return (
    <div className="bg-slate-950/80 rounded-lg border b
      <h4 className="text-xs font-bold text-slate-300 m
      <p className="text-[10px] text-slate-500 mb-2">
        Как вправление вывиха: одна кость ходит вокруг ,
      </p>
      <svg viewBox="0 0 340 190" className="w-full">
        <g transform="translate(10,6)">
          <text y="10" fill="#94a3b8" fontSize="11" fon
          <line x1="55" y1="46" x2="30" y2="86" stroke=
          <line x1="55" y1="46" x2="80" y2="86" stroke=
          <line x1="30" y1="86" x2="12" y2="126" stroke
          <line
            x1="30" y1="86" x2="48" y2="126"
            stroke={isCut ? "#f59e0b" : "#64748b"}
            strokeWidth={isCut ? 3 : 2}
            strokeDasharray={isCut ? "5,3" : "none"}
          />
          <circle cx="55" cy="46" r="15" fill="#1e293b"
          <text x="55" y="50" textAnchor="middle" fill=
          <circle cx="30" cy="86" r="15" fill="#78350f"
          <text x="30" y="90" textAnchor="middle" fill=

```

```

        strokeWidth={isRot ? 2.5 : 1.5}
      />
      <text x="63" y="130" textAnchor="middle" fill="black">
      <circle cx="95" cy="126" r="13" fill="#0f172a" stroke="black" />
      <text x="95" y="130" textAnchor="middle" fill="black">
      <text x="112" y="92" fill={isRot ? "#10b981" : "black"}>
        {isRot ? "↓ β теперь при р" : "β при р"}
      </text>
      <text x="30" y="160" fill="#64748b" fontSize=14>
      </g>
    </svg>
    <p className="text-[10px] text-slate-500 leading-5">
      Обход слева-направо α x β p γ не меняется никог
      числа поворотов. Единственный, кто меняет хозяи
    </p>
  </div>
);
};

/* ----- рендер -----

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
  const pos = new Map<number, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: SN | null, d: number) => {
    if (!n) return;
    walk(n.left, d + 1);
    pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
    slot += 1;
    walk(n.right, d + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

    if (idx >= steps.length - 1) {
      setPlaying(false);
      baseRef.current = steps[steps.length - 1].root ??
      setTreeVers((v) => v + 1);
      return;
    }
    timer.current = setTimeout(() => setIdx((v) => v +
    return () => {
      if (timer.current) clearTimeout(timer.current);
    });
  }, [playing, idx, steps]);

```

```

const step = steps ? steps[Math.min(idx, steps.length
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot
const nodes = useMemo(() => [...pos.entries()].map(([
const edges = useMemo(() => {
  const es: { a: number; b: number }[] = [];
  const walk = (n: SN | null) => {
    if (!n) return;
    if (n.left) es.push({ a: n.key, b: n.left.key });
    if (n.right) es.push({ a: n.key, b: n.right.key });
    walk(n.left);
    walk(n.right);
  };
  walk(viewRoot);
  return es;
}, [viewRoot, treeVers, idx]);

```

```

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.p
const autoH = Math.max(300, ...nodes.map((n) => n.py

```

```

const dBefore = useMemo(() => (steps ? depthOf(steps[
const dNow = steps ? depthOf(viewRoot, steps[steps.le
const hBefore = useMemo(() => (steps ? heightOf(steps
const hNow = heightOf(viewRoot);

```

```

// переехавшие рёбра: разница с предыдущим кадром

```

```

    const e = step.aimEdge ?? step.cutEdge!;
    if (e.includes(k)) return "#f59e0b";
  }
  if (step.pathKeys?.includes(k)) return "#38bdf8";
  return "#64748b";
};

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border"
    {/* Wanka */}
    <div className="flex flex-wrap items-center justify"
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white"
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white"
            <p className="text-sm text-indigo-300">Кликн
          </div>
        </div>
      <div className="flex flex-wrap items-center gap"
        <input
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter") {
              const v = parseInt(input, 10);
              if (!Number.isNaN(v)) {
                const has = inorderKeys(baseRef.current
                  if (has) runKey(v);
                  else doInsert();
                  setInput("");
                }
              }
            }
          }}
          placeholder="ключ + Enter"
          className="w-28 bg-slate-950 border border-
            s:border-indigo-500"

```



```

<button onClick={() => { setPlaying(false); s
  -lg text-xs font-bold bg-slate-700 text-whi
  ← ⏏
</button>
<button onClick={() => setPlaying(!playing)}
  er-600" : "bg-emerald-600"}`}>
  {playing ? "  Пауза" : "▶ Пуск"}
</button>
<button onClick={() => { setPlaying(false); s
  className="px-3 py-1.5 rounded-lg text-xs fo
  ⏏ →
</button>
<span className="text-xs text-slate-400">
  ⏏ {idx + 1} из {steps.length} • поворотов
</span>
{step?.caseLabel && (
  <span className="px-2.5 py-1 rounded-full t
  00/60">
    {step.caseLabel}
  </span>
)}
</div>
)}

{/* Дерево */}
<div className="bg-indigo-950/20 rounded-xl p-4 b
  {!steps && (
    <div className="absolute top-3 left-3 bg-indi
    ont-bold">
      Кликни по любому узлу – splay покадрово
    </div>
  )}
<svg viewBox={`0 0 ${autoW} ${autoH}`} classNam
  {edges.map((e) => {
    const a = pos.get(e.a);
    const b = pos.get(e.b);
    if (!a || !b) return null;
    const isAim = step?.aimEdge && step.aimEdge

```

```

        fill={stepColor(n.key)}
        stroke={strokeColor(n.key)}
        strokeWidth={isAimNode ? 4 : 2}
        className="transition-all duration-300"
      />
      <text y={4} fill="#ffffff" fontSize="13">
        {n.key}
      </text>
    </g>
  );
}
</svg>

```

```

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row" >
  <div className="flex-1 bg-slate-950/80 p-4 rounded" >
    <p className="font-semibold text-amber-400" >
      {step?.kind === "aim"
        ? " Прицел:"
        : step?.kind === "cut"
        ? " ∞ Отстёгиваем среднее поддерево:"
        : step?.kind === "rot"
        ? " Поворот:"
        : step?.kind === "search"
        ? " Спуск:"
        : "Статус:"}
    </p>
    <p className="min-h-[40px] flex items-center" >
      {step ? step.note : "Кликни по узлу или в
      ке справа – механика одного поворота."}
    </p>
    {steps && step && step.kind !== "search" &&
      <p className={`text-xs mt-2 ${checkOk ? "text-amber-400" : "text-slate-500"}`} >
        {checkOk ? " " : " "} проверка кадра: 0
        {inorderKeys(viewRoot).length}
        {movedCount > 0 ? ` · переехало рёбер: ` : ""}
      </p>
    }
  </div>
</div>

```

```

        {CODE_LINES.map(({l}) => {
          const zig = step?.caseLabel === "Zig" &&
          const zz1 = step?.caseLabel === "Zig-Zig"
          const zz2 = step?.caseLabel === "Zig-Zig"
          const za1 = step?.caseLabel === "Zig-Zag"
          const za2 = step?.caseLabel === "Zig-Zag"
          const active = zig || zz1 || zz2 || za1 |
          return (
            <div key={l.line} className={`py-1 px-2
ber-400" : "text-slate-400"}`>
              <span className="text-slate-600 selec
                {l.text}
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
};

export default SplayWalkViz;

```

ФАЙЛ 70/92: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSte

interface Plate {
  id: string;
  name: string;
  emoji: string;

```

```

const [cleanRack, setCleanRack] = useState<Plate[]>([
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([

const addLog = (msg: string) => setLog(prev => [msg,

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут р
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

const preset = PLATE_PRESETS[counter % PLATE_PRESET
counter++;

const newPlate: Plate = {
  id: Date.now().toString(),
  ...preset,
  isDirty: true,
  animState: 'in',
};

setDirtyStack(prev => [...prev, newPlate]);
addLog(`  PUSH: Положили ${newPlate.emoji} сверху с

setTimeout(() => {
  setDirtyStack(prev => prev.map(p => p.id === newP
}, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)

```

```

        State: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы',      emoji: '🍷',
      imState: 'in' },
    { id: 'r3', name: 'Блюдец от торта',           emoji: '🍰',
      mState: 'in' },
  ]});
setCleanRack([]);
setIsWashing(false);
setShowSoap(false);
setLog([' Стол сброшен. Посуда снова грязная!']);
};

```

```

const topIndex = displayStack.length - 1;

```

```

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-blue-950/40 border border-blue-950">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-blue-400 text-2xl">
          Ты складываешь грязные тарелки друг на друга.
        </h3>
        <p className="text-xs text-slate-300 mt-1 leading-4">
          А когда приходит время мыть, ты берёшь именную тарелку.
          Попытаешься вытащить нижнюю — всё рухнет!
        </p>
      </div>
    </div>

    {/* УПРАВЛЕНИЕ */}
    <div className="flex items-center gap-3 flex-wrap">
      <button
        onClick={pushPlate}
        disabled={isWashing}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
          opacity-40 disabled:cursor-not-allowed text-white font-medium
          rounded-95 transition-all cursor-pointer flex items-center justify-center">
        Добавить тарелку
      </button>
    </div>
  </div>
);

```

```

    { /* Мыльные пузыри при мытье */ }
    { showSoap && {
      <div className="absolute inset-0 pointer-events-none">
        <div className="absolute w-6 h-6 rounded-3xl border-2 border-white">
          <div className="absolute w-4 h-4 rounded-3xl border-2 border-white">
            <div className="absolute w-5 h-5 rounded-3xl border-2 border-white">
              <div className="absolute text-4xl anim-curve">
                </div>
            </div>
          </div>
        </div>
      </div>
    } }

    { displayStack.length === 0 ? {
      <div className="flex-1 flex flex-col items-center justify-center">
        <span className="text-6xl mb-3 animate-pulse">
          <p className="text-white font-black text-4xl">
            <p className="text-slate-500 text-xs mt-1">
              </p>
            </div>
          </div>
        </div>
      <div className={`flex-1 flex flex-col justify-center`}
        { /* Указатель TOP */ }
        <div
          className="absolute right-8 flex items-center justify-center"
          style={{ bottom: `${24 + topIndex * 44}px` }}
        >
          <span className="text-xs font-mono text-white">
            er gap-1">
              <ArrowUp className="w-3.5 h-3.5" />
              <span>ВЕРШИНА (TOP)</span>
            </span>
            <span className="text-blue-400 animate-pulse">
              </div>
            </div>

            { /* Тарелки (отрисовываем в обратном порядке) */ }
            { displayStack.slice().reverse().map((plate) => {

```

```

        order-emerald-800/80 text-[9px] font-mono">
        <Sparkles className="w-3.5 h-3.5 animate-pu
        <span>ЧИСТО </span>
    </div>

    <div className="flex-1 flex flex-col justify-c
    {cleanRack.length === 0 ? {
        <div className="flex-1 flex flex-col item
            <span className="text-5xl mb-2"> </span>
            <p className="text-xs font-bold font-mo
            <p className="text-[10px] mt-1">Здесь 6
        </div>
    ) : {
        <div className="flex flex-col gap-2 w-ful
            {cleanRack.map(plate => {
                <div
                    key={plate.id}
                    className="w-full h-8 rounded-full
items-center justify-center gap-2 opacity-1
                >
                    <span className="text-sm"> {plate.n
                    <span className="text-[10px] text-e
                </div>
            })}
        </div>
    })
</div>

    {/* Столешница сушилки */}
    <div className="w-full h-4 bg-gradient-to-r f
    </div>
</div>

```

```

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-:
    <div className="text-[10px] font-mono text-slat
        {log.map((entry, idx) => {
            <div

```

```

        steps.push({ i, j, pi: [...pi], highlight:
яд назад: откат j = pi[j - 1] = ${pi[j -
j = pi[j - 1];
    }

    if (s[i] === s[j]) {
        steps.push({ i, j, pi: [...pi], highlight:
ска увеличивается.` });
        j++;
    } else {
        steps.push({ i, j, pi: [...pi], highlight:
овпадений нет.` });
    }

    pi[i] = j;
    steps.push({ i, j, pi: [...pi], desc: `Записыва

    if (j === pattern.length) {
        steps.push({ i, j, pi: [...pi], found: true
    }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

        if (i <= r) {

```



```

const [pattern, setPattern] = useState("aba");
const [text, setText] = useState("abacaba");

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

const { s: demoString, steps, patternLength: demoPa
  if (mode === "kmp") return generateKmpSteps(safe
  else return generateZSteps(safePattern, safeText
}, [mode, safePattern, safeText]);
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?
const s = runtimeString ?? demoString;
const patternLength = runtimeString ? runtimeString

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
  setStepIdx(0);
  setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
  if (autoPlay) {
    timer.current = setInterval(() => {
      setStepIdx(prev => {
        if (prev >= steps.length - 1) { set
        return prev + 1;
      });
    }, 1200);
  }
  return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

```

```

" : "text-slate-400 hover:text-slate-300"}`
      Z-ФУНКЦИЯ

```

```

    </button>

```

```

  </div>

```

```

    <div className="flex items-center gap-2"

```

```

      <div className="bg-slate-800 p-1 fl

```

```

        <span className="text-[10px] te

```

```

        <input value={pattern} onChange=

```

```

      xt-white text-xs font-bold font-mono px-2 p

```

```

    </div>

```

```

    <div className="bg-slate-800 p-1 fl

```

```

      <span className="text-[10px] te

```

```

      <input value={text} onChange={e

```

```

    te text-xs font-bold font-mono px-2 py-1 ou

```

```

    </div>

```

```

  </div>

```

```

</div>

```

```

<div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">

```

```

  <div className="flex gap-1.5 px-4 h-ful

```

```

    {s.split("").map((char, index) => {

```

```

      const isPattern = index < patte

```

```

      const isSeparator = index === p

```

```

      let borderColor = "border-slate

```

```

      let bgColor = "bg-slate-900";

```

```

      let textColor = isSeparator ? "

```

```

      // KMP Logic

```

```

      if (mode === "kmp") {

```

```

        if (step.highlight?.include

```

```

          borderColor = "border-a

```

```

          bgColor = step.match ===

```

```

      -900/40");

```

```

    }

```

```

  }

```

```

z-20">
        { /* KMP fingers */
        {mode == "kmp" && step
            <div className="abs
                <span className
            </div>
        }}
        {mode == "kmp" && step
            <div className="abs
                <span className
            </div>
        }}
        {mode == "kmp" && step
            <div className="abs
                <span className
            </div>
        }}

20">
        { /* Z fingers */
        {mode == "z" && step.i
            <div className="abs
                <span className
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span className
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span className
            </div>
        }}

e z-20">
        { /* Z fingers */
        {mode == "z" && step.i
            <div className="abs
                <span className
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span className
            </div>
        }}
        {mode == "z" && step.z
            <div className="abs
                <span className
            </div>
        }}
    
```

```

erflow-hidden">
    <div className="absolute top-0 right-0" >
        <span className="font-mono text-sm text-slate-400">
            {steps.length}
        </span>
    </div>
    <div className="text-[10px] text-slate-400">
        {steps.length}
    </div>
    <div className="text-sm text-slate-400">
        {step.desc}
    </div>
</div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl">
    <span className="text-2xl mt-1"> </span>
    <p className="text-xs text-slate-400">
        Обрати внимание на хитрость с решёткой.
        Мы пишем <b>Шаблон + # + Текст</b>.
        Когда значени {mode === 'kmp' ? "пробел" : "не пробел"}
        было найдено!
        Пройди шаги вручную, посмотри, как работает алгоритм
        для пропуска работы (в Z).
    </p>
</div>

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">
        <span>Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})</span>
    </div>
    <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());` : `vector<int> pi(s.length());`}
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}
    </pre>
</div>

```

```

const [open, setOpen] = useState(false);

const hiddenShift = side === "top" ? "translate-y-1"

return (
  <span
    className={`relative inline-flex ${className} ?? ""
      onMouseEnter={() => setOpen(true)}
      onMouseLeave={() => setOpen(false)}
      onFocus={() => setOpen(true)}
      onBlur={() => setOpen(false)}
    >
    {children}
    <span
      role="tooltip"
      className={`pointer-events-none absolute left-1
        bg-slate-900 px-3 py-2 text-left text-[11px]
        l duration-150 ${
          side === "top" ? "bottom-full mb-2" : "top-full
        } ${open ? "visible translate-y-0 opacity-100"
      >
      {content}
      <span
        className={`absolute left-1/2 h-2 w-2 -transl
          side === "top" ? "-bottom-[5px] border-b bo
        }`}
      />
    </span>
  </span>
);
};

```

```

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps, {
    const v = vizNumber(vars.v);
    const orderLength = vizArray(vars.order)?.length;
    if (v === null && orderLength === undefined) return null;
    const candidates = steps
      .map((step, index) => ({ step, index }))
      .filter(({ step }) => (v === null || step.currentNode !== v));
    return candidates.at(-1)?.index ?? null;
  });
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();
    const topoList: number[] = [];
    const dfsStack: number[] = [];

    const recordStep = (desc: string, currentNode: number) => {
      listSteps.push({
        desc,
        visited: new Set(visited),
        fullyProcessed: new Set(fullyProcessed),
        currentNode,
        topoList: [...topoList],
      });
    };
  });
};

```

```

        ,
        u,
    );
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        recordStep(
            `Запускаем новый обход DFS от нетронутой верши
            null,
        );
        dfs(i);
    }
}

recordStep(
    "Топологическая сортировка завершена! Полученный
    null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
    desc: "Запуск...",
    visited: new Set(),
    fullyProcessed: new Set(),
    currentNode: null,
    topoList: [],
    dfsStack: [],
};

useEffect(() => {
    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1)
        timer = setInterval(() => {
            setCurrentStepIdx((prev) => prev + 1);

```

```

    })
    {isPlaying ? "Пауза " : "Старт "}
</button>

<button
  onClick={reset}
  className="flex items-center justify-center
ors"
>
  <RotateCcw className="w-4 h-4" />
</button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12 "
  {/* Playfield SVG */}
  <div className="lg:col-span-8 bg-[#0B1120] rela
    <svg className="w-full h-full max-w-[650px]"
      <defs>
        <marker
          id="dag-arrow"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
        </marker>
        <marker
          id="dag-arrow-active"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"

```



```

void textColor;

if (isCurrent) {
    fill = "#312e81";
    stroke = "#818cf8";
    textColor = "text-white font-bold";
} else if (isProcessed) {
    fill = "#022c22";
    stroke = "#059669";
    textColor = "text-emerald-300";
} else if (isVisited) {
    fill = "#3730a3";
    stroke = "#6366f1";
    textColor = "text-indigo-200";
}

return (
    <g key={node.id}>
        <rect
            x={node.x - 55}
            y={node.y - 22}
            width="110"
            height="44"
            rx="8"
            fill={fill}
            stroke={stroke}
            strokeWidth="2"
            className="transition-all duration-!
        />
        <text
            x={node.x}
            y={node.y - 4}
            textAnchor="middle"
            fill="white"
            fontSize="11px"
            fontWeight="bold"
            className="pointer-events-none"
        >

```

```

pr-1 text-slate-300">
    {step.topoList.length === 0 ? (
        <span className="text-slate-600 text-
            ожидаем обработку узлов...
        </span>
    ) : (
        getTopoListString()
    )}
</div>
</div>

{/* Color key */}
<div className="space-y-1">
    <span className="text-[10px] text-slate-5
        ВЕРШИНЫ В DFS:
    </span>
    <div className="flex items-center gap-2 t
        <span className="w-3.5 h-3.5 rounded bg
        <span className="text-slate-400 text-[1
            Серые (зашли)
        </span>
    </div>
    <div className="flex items-center gap-2 t
        <span className="w-3.5 h-3.5 rounded bg
        <span className="text-slate-400 text-[1
            Черные (готово)
        </span>
    </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-sla
    <span>
        ⏮ {currentStepIdx + 1} из {steps.length}
    </span>
    <div className="flex gap-1">
        <button
            disabled={currentStepIdx === 0}

```

Универсальное пособие • полный исходный код

```
-----
*   erase   – удаление ПРОИЗВОЛЬНОЙ точки: найти по x,
*   layers  – миф «дети в массиве по  $2^k/2^{k+1}$ »: работает
*   search  – «отсортирую бинарным поиском»: поиск не с
*              сортировка платит  $\geq n \cdot \log n$  сравнений, со
*
* Канонический набор: все x и все y различны  $\Rightarrow$  дерево
* Узел идентифицируется парой  $id = "x:y"$ .
*/
```

```
export interface Pair {
  key: number;
  grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>
```

```
interface TNode {
  id: NodeId;
  key: number;
  grade: number;
  left: TNode | null;
  right: TNode | null;
}
```

```
export const POINTS: Pair[] = [
  { key: 45, grade: 8 },
  { key: 3, grade: 6 },
  { key: 6, grade: 5 },
  { key: 2, grade: 4 },
  { key: 9, grade: 3 },
  { key: 7, grade: 2 },
  { key: 12, grade: 0 },
  { key: 1, grade: -4 },
];
```

```
/** Порядок ввода (как в черновике) – намеренно НЕ отсор
export const INPUT_ORDER: Pair[] = [
  { key: 3, grade: 5 },
```

```

    if (t.key <= x) {
      const sub = go(t.right);
      trace.push({ atId: t.id, goLeft: true, note: `(${t.key}
        о правое поддереву.` });
      t.right = sub.l;
      return { l: t, r: sub.r };
    }
    const sub = go(t.left);
    trace.push({ atId: t.id, goLeft: false, note: `(${t.key}
      о левое поддереву.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
  };
  const res = go(root);
  return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
  root: TNode | null;
  trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; }
}

export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
  const trace: MergeResult["trace"] = [];
  const go = (a: TNode | null, b: TNode | null): TNode | null {
    if (!a || !b) {
      trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null,
        note: `Обе ветви пустые, возвращаемое целиком.` });
      return a ?? b;
    }
    if (a.grade > b.grade) {
      trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
        note: `Выборим его правое поддереву с (${b.key}, ${b.grade})` });
      a.right = go(a.right, b);
      return a;
    }
    trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
      note: `Выборим его левое поддереву с (${a.key}, ${a.grade})` });
    b.left = go(a, b.left);
  };
  const res = go(a, b);
  return { root: res, trace };
}

```

```

    if (!n) return;
    if (n.key <= min || n.key > max) bst = false;
    if (parentGrade !== null && n.grade > parentGrade)
    walk(n.left, min, n.key, n.grade);
    if (prev !== null && n.key < prev) bst = false;
    prev = n.key;
    walk(n.right, n.key, max, n.grade);
  };
  walk(root, -Infinity, Infinity, null);
  return { bst, heap };
}

```

/* ----- раскладка сцены дерева -----

```

const layoutTree = (root: TNode | null) => {
  const pos = new Map<NodeId, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 44 });
    slot += 1;
    walk(n.right, depth + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
  const es: { from: NodeId; to: NodeId }[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ from: n.id, to: n.left.id }); }
    if (n.right) { es.push({ from: n.id, to: n.right.id }); }
  };
  walk(root);
  return es;
};

```

```

    });
  }}}
  {nodes.map((n) => {
    const p = pairParts(n.id);
    const pulse = pulseId === n.id;
    return (
      <g key={`tn-${n.id}`} transform={`translate
        <circle r={pulse ? 24 : 20} fill={nodeColor}
        } strokeWidth={pulse ? 4 : 2} className="tr
        <text y={-2} fill="#fff" fontSize="13" for
        <text y={13} fill="#cbd5e1" fontSize="10"
      </g>
    );
  }}}
</svg>
{subtitle && <p className="text-xs text-slate-400
</div>
});
};

const pairParts = (id: NodeId): [number, number] => {
  const [a, b] = id.split(":").map(Number);
  return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H

interface PlaneProps {
  points: Pair[];
  drawnEdges: { a: Pair; b: Pair }[];
  pending: Pair | null;
  interactive?: boolean;
  selected?: Pair | null;

```

```

        e === e.a.grade));
    return <line key={`pe-${i}`} x1={a.x} y1={a.y}
      2} strokeDasharray={isBad ? "4,3" : "none"}
    }}}

    {/* временная линия игры */}
    {interactive && selected && cursor && (
      <line
        x1={PX(selected.key)} y1={PY(selected.grade)}
        stroke="#818cf8" strokeWidth={2.5} strokeDash
      />
    )}

    {points.map((p) => {
      const done = drawnEdges.some((e) => (e.a.key ===
      const isSel = selected && selected.key === p.key
      return (
        <g
          key={`pp-${p.key}:${p.grade}`}
          className="transition-all duration-500"
          onPointerDown={interactive && onPointDown ?
          onPointerUp={interactive && onPointUp ? ()
          style={interactive ? { cursor: "pointer" }
        >
          <circle cx={PX(p.key)} cy={PY(p.grade)} r={
            troke={isSel ? "#a5b4fc" : done ? "#38bdf8"
          <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
            ">{p.key}</text>
          </g>
        );
      )}
    }}}

    {pending && (
      <g className="transition-all duration-500">
        <circle cx={PX(pending.key)} cy={PY(pending.g
        >
        <text x={PX(pending.key)} y={PY(pending.grade
          ly="monospace">{pending.key}</text>

```

```
};
```

```
/** Проверка BST леса: у узла  $\leq 1$  ребёнка слева и  $\leq 1$  справа.
export function forestIsBst(edges: GEdge[], all: Pair[]) {
  const kids = new Map<string, Pair[]>();
  for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || []), e]);
  const hasParent = new Set(edges.map((e) => gpid(e.b)));
  const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
  const ok = (n: Pair | undefined, lo: number, hi: number) => {
    if (!n) return true;
    if (n.key <= lo || n.key > hi) return false;
    const ch = (kids.get(gpid(n)) ?? []).slice().sort((c1, c2) => c1.key - c2.key);
    const left = ch.filter((c) => c.key < n.key);
    const right = ch.filter((c) => c.key > n.key);
    if (left.length > 1 || right.length > 1) return false;
    return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
  };
  return roots.every((r) => ok(r, -Infinity, Infinity));
}
```

```
/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
  const heap = edges.every((e) => e.a.grade > e.b.grade);
  const oneParent = new Set(edges.map((e) => gpid(e.b)));
  const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
  const bst = forestIsBst(edges, all);
  const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === p.a));
  const complete = edges.length === all.length - 1 && roots.length === 1;
  return { heap, oneParent, acyclic, bst, complete, win: heap && complete };
}
```

```
/** Попытка хода: почему нельзя — или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
  if (sameP(a, b)) return { ok: false, reason: "Это одна линия" };
  if (a.grade === b.grade) return { ok: false, reason: "«!» равны — «кто выше?»" };
  const parent = a.grade > b.grade ? a : b;
  const child = a.grade > b.grade ? b : a;
  if (edges.some((e) => e.a === parent && e.b === child)) return { ok: false, reason: "У родителя уже есть ребёнок" };
  if (edges.some((e) => e.a === child && e.b === parent)) return { ok: false, reason: "У ребёнка уже есть родитель" };
  return { ok: true, reason: "Ход возможен" };
}
```



```
-----
    return { pairs, errors, equalX: new Set(xs).size !== 0 };
};
```

```
const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,
```

```
/* ----- игра «соедини мышкой»: чистая логика
```

```
/** Ребро игры: a = родитель (выше по y), b = ребёнок.
```

```
export interface GameEdge {
```

```
  a: Pair;
```

```
  b: Pair;
```

```
}
```

```
const pid = (p: Pair) => `${p.key}:${p.grade}`;
```

```
export function canConnect({
```

```
  edges: GameEdge[],
```

```
  x: Pair,
```

```
  y: Pair,
```

```
): { ok: boolean; edge?: GameEdge; reason?: string } {
```

```
  if (x.key === y.key && x.grade === y.grade) return { ok: true };
```

```
  for (const e of edges) {
```

```
    if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
```

```
        return { ok: false, reason: "такая линия уже проведена" });
```

```
  }
```

```
  if (x.grade === y.grade) return { ok: false, reason: "одинаковые уровни" };
```

```
  " };
```

```
  const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
```

```
  if (edges.some((e) => pid(e.b) === pid(child)))
```

```
    return { ok: false, reason: `y точки (${child.key})` };
```

```
  // цикл: родитель уже сидит в компоненте ребёнка (ребёнок уже в компоненте родителя)
```

```
  const childComp = new Set<string>([pid(child)]);
```

```
  let grew = true;
```

```
  while (grew) {
```

```
    grew = false;
```

```
    for (const e of edges) {
```

```
      if (childComp.has(pid(e.b)) && !childComp.has(pid(e.a)))
```

```
        childComp.add(pid(e.a));
```

```
      grew = true;
```

```

const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
    const seen = new Set<string>();
    const inorder: number[] = [];
    const walk = (p: Pair): void => {
        if (seen.has(pid(p))) {
            bstOk = false; // цикл
            return;
        }
        seen.add(pid(p));
        const kids = children.get(pid(p)) ?? [];
        if (kids.length > 2) {
            bstOk = false;
            return;
        }
        const sorted = [...kids].sort((u, v) => u.key - v
        let left: Pair | null = null;
        let right: Pair | null = null;
        if (sorted.length === 2) {
            if (sorted[0].key === sorted[1].key || sorted[0
                bstOk = false; // двое детей с равным x или с
                return;
            }
            left = sorted[0];
            right = sorted[1];
        } else if (sorted.length === 1) {
            // одиночный ребёнок: сторона важна – определяет
            if (sorted[0].key <= p.key) left = sorted[0];
            else right = sorted[0];
        }
        if (left) walk(left);
        inorder.push(p.key);
        if (right) walk(right);
    }
}

```

```

    return build(rootPair);
}

const BuildMode: React.FC = () => {
  const [raw, setRaw] = useState(DEMO_RAW);
  const [phase, setPhase] = useState<BuildPhase>("sort");
  const [algo, setAlgo] = useState<SortAlgo>("none");
  const [order, setOrder] = useState<ConnectOrder>("y-d");
  const [sortIdx, setSortIdx] = useState(0);
  const [connIdx, setConnIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [sortCost, setSortCost] = useState<string | null>();
  const live = useTreapLive();

  const parsed = useMemo(() => parsePairs(raw), [raw]);
  const userPairs = parsed.errors.length ? [] : parsed.pairs;

  const n = userPairs.length;

  const userTree = useMemo(() => (userPairs.length >= 1 ?
  const canonEdges = useMemo(() => {
    const es: { a: Pair; b: Pair }[] = [];
    const walk = (node: TNode | null) => {
      if (!node) return;
      if (node.left) { es.push({ a: { key: node.key, group: 1,
        left}); }
      if (node.right) { es.push({ a: { key: node.key, group: 1,
        de.right}); }
    };
    walk(userTree);
    return es;
  }, [userTree]);
  const leftRightEdges = useMemo(() => [...canonEdges].concat(
  useEffect(() => {
    if (!playing || phase === "game") return;
    const t = setTimeout(() => {
      if (phase === "sort") {

```

```

const resetGame = () => {
  setGameEdges([]);
  setSel(null);
  setCursor(null);
  setGameMsg(null);
  setBadEdge(null);
  setHintUsed(false);
};

const startGame = () => {
  resetGame();
  setPhase("game");
  setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair)
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree

const attemptConnect = (x: Pair, y: Pair) => {
  const r = canConnect(gameEdges, x, y);
  if (r.ok && r.edge) {
    setGameEdges((prev) => [...prev, r.edge!]);
    setGameMsg(r.reason ?? null);
    setBadEdge(null);
  } else {
    setGameMsg(r.reason ?? "так нельзя");
    setBadEdge({ a: x, b: y });
    window.setTimeout(() => setBadEdge(null), 900);
  }
};

const pointDown = (p: Pair) => {
  if (phase !== "game") return;
  if (!sel) {
    setSel(p);
    setGameMsg(`Выбрана (${p.key}; ${p.grade}) — теперь
  } else if (sel.key === p.key && sel.grade === p.grade) {
    setSel(null);
  }
};

```

```

        демо-набор
      </button>
    </div>
    <textarea
      value={raw}
      onChange={(e) => { setRaw(e.target.value); set
        rows={2}
        className="w-full bg-slate-950 border border-
          s:border-emerald-500"
        placeholder="3 5, 45 8, 1 -4 ..."
    />
    {parsed.errors.length > 0 && (
      <div className="mt-2 text-xs text-rose-300 bg
        {parsed.errors.map((e, i) => <div key={i}>x
      </div>
    )}
    {parsed.errors.length === 0 && (parsed.equalX |
      <div className="mt-2 text-xs text-amber-300 b
        {parsed.equalY && <div>✂ Есть <b>равные y</b>
        кто <b>раньше</b> в списке</b> (сортировка ста
        {parsed.equalX && <div>✂ Есть <b>равные x</b>
        de.key → left).</div>}
      </div>
    )}
  </div>

  { /* Координатная плоскость – НАВЕРХУ */ }
  <div className="bg-emerald-950/20 rounded-xl p-4
    <div className="flex flex-wrap items-center jus
      <h4 className="text-base font-bold text-emera
        <div className="flex items-center gap-2">
          {phase === "sort" ? (
            <div>
              <span className="text-xs text-slate-400
                {[ "quicksort", "counting" ] as const }.m
              <button
                key={a}
                onClick={() => { setAlgo(a); setSort

```

```

    <Plane
      points={userPairs}
      drawnEdges={phase === "game" ? gameEdges : ph
      pending={null}
      interactive={phase === "game"}
      selected={sel}
      cursor={cursor}
      badEdge={badEdge}
      onPointDown={pointDown}
      onPointUp={pointUp}
      onMove={planeMove}
    />
    <div className="mt-2 text-xs text-slate-300 bg-
      {phase === "sort" ? (
        algo === "none" ? (
          <>Ввод приходит <b>как попало</b> ({n} пар
          ть</b>.</>
        ) : sortIdx < 3 ? (
          algo === "quicksort" ? (
            <>⚡ quicksort: счётчик сравнений: <b>cl
            {comparisons}</b> ( $n \cdot \log_2 n$ ).</>
          ) : (
            <> counting: точки в корзины по значени
            , потому что у – маленькие целые.</>
          )
        ) : (
          <>Отсортировано ({sortCost ?? "способ не
          <b>Игру </b></>
        )
      ) : phase === "game" ? (
        gameWon ? (
          <> Готово! {hintUsed ? "Решение подсказан
          ево: линии вниз по у, обход по х отсортиров
        ) : gameMsg ? (
          <>⚠ {gameMsg}</>
        ) : (
          <>Клигни точку-родителя (выше по «!»), за
        )
      )
    )
  )
}

```

```

        </div>
      })
    </div>

    { /* Победное дерево в игре */
    {phase === "game" && gameTree && {
      <div className="bg-emerald-950/20 rounded-xl p-4">
        <h4 className="text-sm font-bold text-emerald-300">Победное дерево в игре</h4>
        <TreeView root={gameTree} subtitle="Слева направо существует." />
      </div>
    })

    { /* Дерево */
    {order === "y-desc" && userTree && {
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-slate-300">Дерево</h4>
        <TreeView root={userTree} subtitle="Слева направо существует." />
      </div>
    })
  </div>
);
};

```

/* ===== РЕЖИМ SPLIT ===== */

```

const SplitMode: React.FC = () => {
  const [x0, setX0] = useState(7);
  const [stepIdx, setStepIdx] = useState(0);
  const fullTree = useMemo(() => buildTree(POINTS), [])
  const split = useMemo(() => splitTree(buildTree(POINTS)), [fullTree]);
  const steps = split.trace;
  const live = useTreapLive();
  // Компилятор: x – ключ разреза, i – номер шага спуска
  useEffect(() => { if (live.x !== null) setX0(live.x); }, [live.x]);
  useEffect(() => { if (live.i !== null) setStepIdx(Math.min(stepIdx, live.i)); }, [live.i]);
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

```

```

    {done && (
      <div className="grid grid-cols-1 md:grid-cols-2"
        <div className="bg-sky-950/20 rounded-xl p-3"
          <h5 className="text-sm font-bold text-sky-300"
            <TreeView root={lTree} width={480} height={400} />
          </div>
          <div className="bg-rose-950/20 rounded-xl p-3"
            <h5 className="text-sm font-bold text-rose-300"
              <TreeView root={rTree} width={480} height={400} />
            </div>
          </div>
        </div>
      <div className="text-xs text-slate-400 bg-slate-900 p-3"
        <LiveChip live={live} />{' '}
        <b>Как это работает на самом деле:</b> идём с
        деревом), отдаётся <b>вместе с веткой</b> – д
      </div>
    </div>
  );
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
  const [stepIdx, setStepIdx] = useState(0);
  const live = useTreapLive();
  // Компилятор: i – номер шага слияния.
  useEffect(() => { if (live.i !== null) setStepIdx(live.i); }, [live.i]);
  const { l, r } = useMemo(() => splitTree(buildTree(P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, P16, P17, P18, P19, P20, P21, P22, P23, P24, P25, P26, P27, P28, P29, P30, P31, P32, P33, P34, P35, P36, P37, P38, P39, P40, P41, P42, P43, P44, P45, P46, P47, P48, P49, P50, P51, P52, P53, P54, P55, P56, P57, P58, P59, P60, P61, P62, P63, P64, P65, P66, P67, P68, P69, P70, P71, P72, P73, P74, P75, P76, P77, P78, P79, P80, P81, P82, P83, P84, P85, P86, P87, P88, P89, P90, P91, P92, P93, P94, P95, P96, P97, P98, P99, P100, P101, P102, P103, P104, P105, P106, P107, P108, P109, P110, P111, P112, P113, P114, P115, P116, P117, P118, P119, P120, P121, P122, P123, P124, P125, P126, P127, P128, P129, P130, P131, P132, P133, P134, P135, P136, P137, P138, P139, P140, P141, P142, P143, P144, P145, P146, P147, P148, P149, P150, P151, P152, P153, P154, P155, P156, P157, P158, P159, P160, P161, P162, P163, P164, P165, P166, P167, P168, P169, P170, P171, P172, P173, P174, P175, P176, P177, P178, P179, P180, P181, P182, P183, P184, P185, P186, P187, P188, P189, P190, P191, P192, P193, P194, P195, P196, P197, P198, P199, P200, P201, P202, P203, P204, P205, P206, P207, P208, P209, P210, P211, P212, P213, P214, P215, P216, P217, P218, P219, P220, P221, P222, P223, P224, P225, P226, P227, P228, P229, P230, P231, P232, P233, P234, P235, P236, P237, P238, P239, P240, P241, P242, P243, P244, P245, P246, P247, P248, P249, P250, P251, P252, P253, P254, P255, P256, P257, P258, P259, P260, P261, P262, P263, P264, P265, P266, P267, P268, P269, P270, P271, P272, P273, P274, P275, P276, P277, P278, P279, P280, P281, P282, P283, P284, P285, P286, P287, P288, P289, P290, P291, P292, P293, P294, P295, P296, P297, P298, P299, P300, P301, P302, P303, P304, P305, P306, P307, P308, P309, P310, P311, P312, P313, P314, P315, P316, P317, P318, P319, P320, P321, P322, P323, P324, P325, P326, P327, P328, P329, P330, P331, P332, P333, P334, P335, P336, P337, P338, P339, P340, P341, P342, P343, P344, P345, P346, P347, P348, P349, P350, P351, P352, P353, P354, P355, P356, P357, P358, P359, P360, P361, P362, P363, P364, P365, P366, P367, P368, P369, P370, P371, P372, P373, P374, P375, P376, P377, P378, P379, P380, P381, P382, P383, P384, P385, P386, P387, P388, P389, P390, P391, P392, P393, P394, P395, P396, P397, P398, P399, P400, P401, P402, P403, P404, P405, P406, P407, P408, P409, P410, P411, P412, P413, P414, P415, P416, P417, P418, P419, P420, P421, P422, P423, P424, P425, P426, P427, P428, P429, P430, P431, P432, P433, P434, P435, P436, P437, P438, P439, P440, P441, P442, P443, P444, P445, P446, P447, P448, P449, P450, P451, P452, P453, P454, P455, P456, P457, P458, P459, P460, P461, P462, P463, P464, P465, P466, P467, P468, P469, P470, P471, P472, P473, P474, P475, P476, P477, P478, P479, P480, P481, P482, P483, P484, P485, P486, P487, P488, P489, P490, P491, P492, P493, P494, P495, P496, P497, P498, P499, P500, P501, P502, P503, P504, P505, P506, P507, P508, P509, P510, P511, P512, P513, P514, P515, P516, P517, P518, P519, P520, P521, P522, P523, P524, P525, P526, P527, P528, P529, P530, P531, P532, P533, P534, P535, P536, P537, P538, P539, P540, P541, P542, P543, P544, P545, P546, P547, P548, P549, P550, P551, P552, P553, P554, P555, P556, P557, P558, P559, P560, P561, P562, P563, P564, P565, P566, P567, P568, P569, P570, P571, P572, P573, P574, P575, P576, P577, P578, P579, P580, P581, P582, P583, P584, P585, P586, P587, P588, P589, P590, P591, P592, P593, P594, P595, P596, P597, P598, P599, P600, P601, P602, P603, P604, P605, P606, P607, P608, P609, P610, P611, P612, P613, P614, P615, P616, P617, P618, P619, P620, P621, P622, P623, P624, P625, P626, P627, P628, P629, P630, P631, P632, P633, P634, P635, P636, P637, P638, P639, P640, P641, P642, P643, P644, P645, P646, P647, P648, P649, P650, P651, P652, P653, P654, P655, P656, P657, P658, P659, P660, P661, P662, P663, P664, P665, P666, P667, P668, P669, P670, P671, P672, P673, P674, P675, P676, P677, P678, P679, P680, P681, P682, P683, P684, P685, P686, P687, P688, P689, P690, P691, P692, P693, P694, P695, P696, P697, P698, P699, P700, P701, P702, P703, P704, P705, P706, P707, P708, P709, P710, P711, P712, P713, P714, P715, P716, P717, P718, P719, P720, P721, P722, P723, P724, P725, P726, P727, P728, P729, P730, P731, P732, P733, P734, P735, P736, P737, P738, P739, P740, P741, P742, P743, P744, P745, P746, P747, P748, P749, P750, P751, P752, P753, P754, P755, P756, P757, P758, P759, P760, P761, P762, P763, P764, P765, P766, P767, P768, P769, P770, P771, P772, P773, P774, P775, P776, P777, P778, P779, P780, P781, P782, P783, P784, P785, P786, P787, P788, P789, P790, P791, P792, P793, P794, P795, P796, P797, P798, P799, P800, P801, P802, P803, P804, P805, P806, P807, P808, P809, P810, P811, P812, P813, P814, P815, P816, P817, P818, P819, P820, P821, P822, P823, P824, P825, P826, P827, P828, P829, P830, P831, P832, P833, P834, P835, P836, P837, P838, P839, P840, P841, P842, P843, P844, P845, P846, P847, P848, P849, P850, P851, P852, P853, P854, P855, P856, P857, P858, P859, P860, P861, P862, P863, P864, P865, P866, P867, P868, P869, P870, P871, P872, P873, P874, P875, P876, P877, P878, P879, P880, P881, P882, P883, P884, P885, P886, P887, P888, P889, P890, P891, P892, P893, P894, P895, P896, P897, P898, P899, P900, P901, P902, P903, P904, P905, P906, P907, P908, P909, P910, P911, P912, P913, P914, P915, P916, P917, P918, P919, P920, P921, P922, P923, P924, P925, P926, P927, P928, P929, P930, P931, P932, P933, P934, P935, P936, P937, P938, P939, P940, P941, P942, P943, P944, P945, P946, P947, P948, P949, P950, P951, P952, P953, P954, P955, P956, P957, P958, P959, P960, P961, P962, P963, P964, P965, P966, P967, P968, P969, P970, P971, P972, P973, P974, P975, P976, P977, P978, P979, P980, P981, P982, P983, P984, P985, P986, P987, P988, P989, P990, P991, P992, P993, P994, P995, P996, P997, P998, P999, P1000, P1001, P1002, P1003, P1004, P1005, P1006, P1007, P1008, P1009, P1010, P1011, P1012, P1013, P1014, P1015, P1016, P1017, P1018, P1019, P1020, P1021, P1022, P1023, P1024, P1025, P1026, P1027, P1028, P1029, P1030, P1031, P1032, P1033, P1034, P1035, P1036, P1037, P1038, P1039, P1040, P1041, P1042, P1043, P1044, P1045, P1046, P1047, P1048, P1049, P1050, P1051, P1052, P1053, P1054, P1055, P1056, P1057, P1058, P1059, P1060, P1061, P1062, P1063, P1064, P1065, P1066, P1067, P1068, P1069, P1070, P1071, P1072, P1073, P1074, P1075, P1076, P1077, P1078, P1079, P1080, P1081, P1082, P1083, P1084, P1085, P1086, P1087, P1088, P1089, P1090, P1091, P1092, P1093, P1094, P1095, P1096, P1097, P1098, P1099, P1100, P1101, P1102, P1103, P1104, P1105, P1106, P1107, P1108, P1109, P1110, P1111, P1112, P1113, P1114, P1115, P1116, P1117, P1118, P1119, P1120, P1121, P1122, P1123, P1124, P1125, P1126, P1127, P1128, P1129, P1130, P1131, P1132, P1133, P1134, P1135, P1136, P1137, P1138, P1139, P1140, P1141, P1142, P1143, P1144, P1145, P1146, P1147, P1148, P1149, P1150, P1151, P1152, P1153, P1154, P1155, P1156, P1157, P1158, P1159, P1160, P1161, P1162, P1163, P1164, P1165, P1166, P1167, P1168, P1169, P1170, P1171, P1172, P1173, P1174, P1175, P1176, P1177, P1178, P1179, P1180, P1181, P1182, P1183, P1184, P1185, P1186, P1187, P1188, P1189, P1190, P1191, P1192, P1193, P1194, P1195, P1196, P1197, P1198, P1199, P1200, P1201, P1202, P1203, P1204, P1205, P1206, P1207, P1208, P1209, P1210, P1211, P1212, P1213, P1214, P1215, P1216, P1217, P1218, P1219, P1220, P1221, P1222, P1223, P1224, P1225, P1226, P1227, P1228, P1229, P1230, P1231, P1232, P1233, P1234, P1235, P1236, P1237, P1238, P1239, P1240, P1241, P1242, P1243, P1244, P1245, P1246, P1247, P1248, P1249, P1250, P1251, P1252, P1253, P1254, P1255, P1256, P1257, P1258, P1259, P1260, P1261, P1262, P1263, P1264, P1265, P1266, P1267, P1268, P1269, P1270, P1271, P1272, P1273, P1274, P1275, P1276, P1277, P1278, P1279, P1280, P1281, P1282, P1283, P1284, P1285, P1286, P1287, P1288, P1289, P1290, P1291, P1292, P1293, P1294, P1295, P1296, P1297, P1298, P1299, P1300, P1301, P1302, P1303, P1304, P1305, P1306, P1307, P1308, P1309, P1310, P1311, P1312, P1313, P1314, P1315, P1316, P1317, P1318, P1319, P1320, P1321, P1322, P1323, P1324, P1325, P1326, P1327, P1328, P1329, P1330, P1331, P1332, P1333, P1334, P1335, P1336, P1337, P1338, P1339, P1340, P1341, P1342, P1343, P1344, P1345, P1346, P1347, P1348, P1349, P1350, P1351, P1352, P1353, P1354, P1355, P1356, P1357, P1358, P1359, P1360, P1361, P1362, P1363, P1364, P1365, P1366, P1367, P1368, P1369, P1370, P1371, P1372, P1373, P1374, P1375, P1376, P1377, P1378, P1379, P1380, P1381, P1382, P1383, P1384, P1385, P1386, P1387, P1388, P1389, P1390, P1391, P1392, P1393, P1394, P1395, P1396, P1397, P1398, P1399, P1400, P1401, P1402, P1403, P1404, P1405, P1406, P1407, P1408, P1409, P1410, P1411, P1412, P1413, P1414, P1415, P1416, P1417, P1418, P1419, P1420, P1421, P1422, P1423, P1424, P1425, P1426, P1427, P1428, P1429, P1430, P1431, P1432, P1433, P1434, P1435, P1436, P1437, P1438, P1439, P1440, P1441, P1442, P1443, P1444, P1445, P1446, P1447, P1448, P1449, P1450, P1451, P1452, P1453, P1454, P1455, P1456, P1457, P1458, P1459, P1460, P1461, P1462, P1463, P1464, P1465, P1466, P1467, P1468, P1469, P1470, P1471, P1472, P1473, P1474, P1475, P1476, P1477, P1478, P1479, P1480, P1481, P1482, P1483, P1484, P1485, P1486, P1487, P1488, P1489, P1490, P1491, P1492, P1493, P1494, P1495, P1496, P1497, P1498, P1499, P1500, P1501, P1502, P1503, P1504, P1505, P1506, P1507, P1508, P1509, P1510, P1511, P1512, P1513, P1514, P1515, P1516, P1517, P1518, P1519, P1520, P1521, P1522, P1523, P1524, P1525, P1526, P1527, P1528, P1529, P1530, P1531, P1532, P1533, P1534, P1535, P1536, P1537, P1538, P1539, P1540, P1541, P1542, P1543, P1544, P1545, P1546, P1547, P1548, P1549, P1550, P1551, P1552, P1553, P1554, P1555, P1556, P1557, P1558, P1559, P1560, P1561, P1562, P1563, P1564, P1565, P1566, P1567, P1568, P1569, P1570, P1571, P1572, P1573, P1574, P1575, P1576, P1577, P1578, P1579, P1580, P1581, P1582, P1583, P1584, P1585, P1586, P1587, P1588, P1589, P1590, P1591, P1592, P1593, P1594, P1595, P1596, P1597, P1598, P1599, P1600, P1601, P1602, P1603, P1604, P1605, P1606, P1607, P1608, P1609, P1610, P1611, P1612, P1613, P1614, P1615, P1616, P1617, P1618, P1619, P1620, P1621, P1622, P1623, P1624, P1625, P1626, P1627, P1628, P1629, P1630, P1631, P1632, P1633, P1634, P1635, P1636, P1637, P1638, P1639, P1640, P1641, P1642, P1643, P1644, P1645, P1646, P1647, P1648, P1649, P1650, P1651, P1652, P1653, P1654, P1655, P1656, P1657, P1658, P1659, P1660, P1661, P1662, P1663, P1664, P1665, P1666, P1667, P1668, P1669, P1670, P1671, P1672, P1673, P1674, P1675, P1676, P1677, P1678, P1679, P1680, P1681, P1682, P1683, P1684, P1685, P1686, P1687, P1688, P1689, P1690, P1691, P1692, P1693, P1694, P1695, P1696, P1697, P1698, P1699, P1700, P1701, P1702, P1703, P1704, P1705, P1706, P1707, P1708, P1709, P1710, P1711, P1712, P1713, P1714, P1715, P1716, P1717, P1718, P1719, P1720, P1721, P1722, P1723, P1724, P1725, P1726, P1727, P1728, P1729, P1730, P1731, P1732, P1733, P1734, P1735, P1736, P1737, P1738, P1739, P1740, P1741, P1742, P1743, P1744, P1745, P1746, P1747, P1748, P1749, P1750, P1751, P1752, P1753, P1754, P1755, P1756, P1757, P1758, P1759, P1760, P1761, P1762, P1763, P1764, P1765, P1766, P1767, P1768, P1769, P1770, P1771, P1772, P1773, P1774, P1775, P1776, P1777, P1778, P1779, P1780, P1781, P1782, P1783, P1784, P1785, P1786, P1787, P1788, P1789, P1790, P1791, P1792, P1793, P1794, P1795, P1796, P1797, P1798, P1799, P1800, P1801, P1802, P1803, P1804, P1805, P1806, P1807, P1808, P1809, P1810, P1811, P1812, P1813, P1814, P1815, P1816, P1817, P1818, P1819, P1820, P1821, P1822, P1823, P1824, P1825, P1826, P1827, P1828, P1829, P1830, P1831, P1832, P1833, P1834, P1835, P1836, P1837, P1838, P1839, P1840, P1841, P1842, P1843, P1844, P1845, P1846, P1847, P1848, P1849, P1850, P1851, P1852, P1853, P1854, P1855, P1856, P1857, P1858, P1859, P1860, P1861, P1862, P1863, P1864, P1865, P1866, P1867, P1868, P1869, P1870, P1871, P1872, P1873, P1874, P1875, P1876, P1877, P1878, P1879, P1880, P1881, P1882, P1883, P1884, P1885, P1886, P1887, P1888, P1889, P1890, P1891, P1892, P1893, P1894, P1895, P1896, P1897, P1898, P1899, P1900, P1901, P1902, P1903, P1904, P1905, P1906, P1907, P1908, P1909, P1910, P1911, P1912, P1913, P1914, P1915, P1916, P1917, P1918, P1919, P1920, P1921, P1922, P1923, P1924, P1925, P1926, P1927, P1928, P1929, P1930, P1931, P1932, P1933, P1934, P1935, P1936, P1937, P1938, P1939, P1940, P1941, P1942, P1943, P1944, P1945, P1946, P1947, P1948, P1949, P1950, P1951, P1952, P1953, P1954, P1955, P1956, P1957, P1958, P1959, P1960, P1961, P1962, P1963, P1964, P1965, P1966, P1967, P1968, P1969, P1970, P1971, P1972, P1973, P1974, P1975, P1976, P1977, P1978, P1979, P1980, P1981, P1982, P1983, P1984, P1985, P1986, P1987, P1988, P1989, P1990, P1991, P1992, P1993, P1994, P1995, P1996, P1997, P1998, P1999, P2000, P2001, P2002, P2003, P2004, P2005, P2006, P2007, P2008, P2009, P2010, P2011, P2012, P2013, P2014, P2015, P2016, P2017, P2018, P2019, P2020, P2021, P2022, P2023, P2024, P2025, P2026, P2027, P2028, P2029, P2030, P2031, P2032, P2033, P2034, P2035, P2036, P2037, P2038, P2039, P2040, P2041, P2042, P2043, P2044, P2045, P2046, P2047, P2048, P2049, P2050, P2051, P2052, P2053, P2054, P2055, P2056, P2057, P2058, P2059, P2060, P2061, P2062, P2063, P2064, P2065, P2066, P2067, P2068, P2069, P2070, P2071, P2072, P2073, P2074, P2075, P2076, P2077, P2078, P2079, P2080, P2081, P2082, P2083, P2084, P2085, P2086, P2087, P2088, P2089, P2090, P2091, P2092, P2093, P2094, P2095, P2096, P2097, P2098, P2099, P2100, P2101, P2102, P2103, P2104, P2105, P2106, P2107, P2108, P2109, P2110, P2111, P2112, P2113, P2114, P2115, P2116, P2117, P2118, P2119, P2120, P2121, P2122, P2123, P2124, P2125, P2126, P2127, P2128, P2129, P2130, P2131, P2132, P2133, P2134, P2135, P2136, P2137, P2138, P2139, P2140, P2141, P2142, P2143, P2144, P2145, P2146, P2147, P2148, P2149, P2150, P2151, P2152, P2153, P2154, P2155, P2156, P2157, P2158, P2159, P2160, P2161, P2162, P2163, P216
```



```

        в (то, что сохраняет порядок x) отправляется ,
      </div>
    </div>
  );
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
  const [key, setKey] = useState(9);
  const [stepIdx, setStepIdx] = useState(0);
  const fullTree = useMemo(() => buildTree(POINTS), [])
  const res = useMemo(() => eraseNode(buildTree(POINTS)
  const steps = res.trace;
  const live = useTreapLive();
  // Компилятор: x — ключ удаляемой вершины, i — шаг сп
  useEffect(() => { if (live.x !== null && POINTS.some(
  useEffect(() => { if (live.i !== null) setStepIdx(Math
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

  const pathIds = useMemo(() => {
    const ids: NodeId[] = [];
    for (const s of steps.slice(0, idx + 1)) if (s.atId
    return ids;
  }, [steps, idx]);
  const pathSet = new Set(pathIds);

  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center gap-3">
        <span className="text-sm text-slate-300">Удалить
        <LiveChip live={live} />
        <select value={key} onChange={(e) => { setKey(N
          te-700 rounded-lg px-3 py-1.5 text-sm font-mo
          {POINTS.map((p) => <option key={p.key} value=
        </select>
        <button onClick={() => setStepIdx(Math.max(0, i

```

 /* ===== РЕЖИМ LAYERS (2k/2k+1) =====

```
const LayersMode: React.FC = () => {
  const [pick, setPick] = useState<"heap" | "treap">("heap")
  // Полное дерево из 7 узлов (куча): индексы 1..7
  const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
  const full = useMemo(() => buildTree(POINTS), []);
  // Честная слотовая разметка treap: индексы как в куче
  const slots = useMemo(() => {
    const arr: (string | null)[] = Array(16).fill(null)
    const walk = (n: TNode | null, i: number) => {
      if (!n || i >= 16) return;
      arr[i] = n.id;
      walk(n.left, 2 * i);
      walk(n.right, 2 * i + 1);
    };
    walk(full, 1);
    return arr;
  }, [full]);
  const nodeCount = slots.filter(Boolean).length;

  return (
    <div className="space-y-4">
      <div className="flex gap-2">
        <button onClick={() => setPick("heap")} className="text-
          ext-white" : "bg-slate-700 text-slate-300">heap</button>
        <button onClick={() => setPick("treap")} className="text-
          text-white" : "bg-slate-700 text-slate-300">treap</button>
      </div>

      {pick === "heap" ? (
        <div className="bg-slate-900/40 rounded-xl p-4">
          <h4 className="text-sm font-bold text-slate-300">
            4>
          <div className="flex gap-2 overflow-x-auto mb-2">
            {heapArr.slice(1).map((v, i) => (
              <div key={i} className="px-3 py-2 rounded">
                <div className="text-slate-500 text-[10px]>{v}</div>
              </div>
            ))}
          </div>
        </div>
      ) : null}
    </div>
  )
}
```

```
};
```

```
/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) =====
```

```
const SearchMode: React.FC = () => {
```

```
  // Подобрено так, что бинарный поиск ГАРАНТИРОВАННО пр
```

```
  // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
```

```
  const shuffled: Pair[] = [
```

```
    { key: 12, grade: 0 },
```

```
    { key: 3, grade: 6 },
```

```
    { key: 45, grade: 8 },
```

```
    { key: 9, grade: 3 },
```

```
    { key: 6, grade: 5 },
```

```
    { key: 2, grade: 4 },
```

```
    { key: 7, grade: 2 },
```

```
    { key: 1, grade: -4 },
```

```
  ];
```

```
  const target = 7;
```

```
  const sortedByKey = [...shuffled].sort((a, b) => a.key
```

```
  const [stage, setStage] = useState<0 | 1 | 2>(0); //
```

```
  const [bidx, setBidx] = useState(0);
```

```
  // шаги бинарного поиска на отсортированном массиве д
```

```
  const bsSteps = useMemo(() => {
```

```
    const steps: { lo: number; hi: number; mid: number;
```

```
    let lo = 0, hi = sortedByKey.length - 1;
```

```
    while (lo <= hi) {
```

```
      const mid = Math.floor((lo + hi) / 2);
```

```
      if (sortedByKey[mid].key === target) { steps.push
```

```
        нашли за ${steps.length + 1} сравнение(й).` }));
```

```
      if (sortedByKey[mid].key < target) { steps.push({
```

```
        } — вся левая половина выброшена.` })); lo = mid
```

```
      else { steps.push({ lo, hi, mid, found: false, no
```

```
        ` })); hi = mid - 1; }
```

```
    }
```

```
    return steps;
```

```
  }, [sortedByKey]);
```

```

    <div className="text-xs text-slate-300 bg-slate-900" style={bgStyle}
      {stage === 0 && <>Массив пар <b>не отсортирован</b>
        а беспорядок.</>}
      {stage === 1 && <> Бинарный поиск посмотрел на
        ю половину... <b>в которой лежала цель</b>. На
        реставил</b>: массив как был в беспорядке, та
      {stage === 2 && bidx < bsSteps.length - 1 && <>
      {stage === 2 && bidx >= bsSteps.length - 1 && <>
        сравнения. Но чтобы поиск вообще заработал,
        сравнения на <b>всех</b>. «Отсортирую бинарни
    </div>

    <div className="text-xs text-slate-400 bg-slate-900" style={bgStyle}
      <b>Есть ли разница, чем сортировать?</b> Сравн
      ня граница, каждое сравнение даёт 1 бит. Но
      ля маленьких целых у (вкладка «Собрать», кноп
      задать порядок соединения.
    </div>
  </div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
  const [edges, setEdges] = useState<GEdge[]>([]);
  const [sel, setSel] = useState<Pair | null>(null);
  const [hover, setHover] = useState<Pair | null>(null);
  const [msg, setMsg] = useState<string>("Кликни точку-1");

  const v = gameVerdict(edges, POINTS);

  const click = (p: Pair) => {
    if (!sel) {
      setSel(p);
      setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кли

```

```

    </pattern>
  </defs>
  <rect x="34" y="16" width={PL_W - 46} height=
  <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={P
  <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y

  {edges.map((e, i) => {
    <line
      key={`ge-${i}`}
      x1={PX(e.a.key)} y1={PY(e.a.grade)}
      x2={PX(e.b.key)} y2={PY(e.b.grade)}
      stroke={i === edges.length - 1 ? "#34d399"
      strokeWidth={i === edges.length - 1 ? 3.5
      className="transition-all duration-300"
    />
  }}}

  {POINTS.map((p) => {
    const isSel = sel && sameP(sel, p);
    const isHover = hover && sameP(hover, p) &&
    return {
      <g key={`gp-${p.key}:${p.grade}`} onClick=
    l)} className="cursor-pointer">
      <circle cx={PX(p.key)} cy={PY(p.grade)}
      <circle cx={PX(p.key)} cy={PY(p.grade)}
      "#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8"
      transition-all duration-200" />
      <text x={PX(p.key)} y={PY(p.grade) + 3.5
      space">{p.key}</text>
      <text x={PX(p.key)} y={PY(p.grade) + 24
    }</text>
    </g>
  });
  }}}
</svg>
</div>

  /* Судья – условия, как у Ахо–Корасика */

```

```
const runtime = useVizRuntime();
const vars = runtime?.variables;
const x = vizNumber(vars?.x);
const y = vizNumber(vars?.y);
const i = vizNumber(vars?.i);
return { x, y, i, linked: x !== null || y !== null || i !== null };
}
```

```

-----
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { ColoringViz } from "./ColoringViz";
import { ComponentsViz } from "./ComponentsViz";
import { ComplexityClassesViz } from "./ComplexityClassesViz";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import ChapterImage from "./ChapterImage";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, что нужно",
    Component: StackViz,
  },

```

```

    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": {
    title: "Splay: механика → большое дерево → собери с",
    hint: "Три вкладки: покадровая механика поворотов в
      песочница «подними узел сам» с проверкой порядка.
    Component: SplayPlayground,
  },
  "splay-rotations": {
    title: "Splay: механика → большое дерево → собери с",
    hint: "Три вкладки: покадровая механика поворотов в
      песочница «подними узел сам» с проверкой порядка.
    Component: SplayPlayground,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
  "top-sort": { title: "Топологическая сортировка", Component:
  "scc-kosaraju": { title: "Компоненты сильной связности", Component:
  "graph-components": {
    title: "Компоненты связности: BFS-разметка",
    hint: "Каждый запуск обхода красит один «остров»; ч
    Component: ComponentsViz,
  },
  "graph-articulation": { title: "Точки сочленения", Component:
  "bridges-code": { title: "Мосты: DFS + tin/low", Component:
  "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component:
  "planarity-euler-formula": {
    title: "Планарность: K5/K3,3 + раскраска",
    hint: "Слева – непланарные графы с подсветкой пересеч
    Component: () => (
      <div className="space-y-10">
        <PlanarityDemo />
        <ColoringViz />
      </div>
    ),
  },
  },
  dijkstra: {

```



```
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
      <div className="space-y-10">
        <WaterfallAnimationWidget />
        <SalmonAutomatonWidget />
      </div>
    ),
  },
  "complexity-classes": {
    title: "Классы сложности: матрёшка  $P \subseteq NP \subseteq PSPACE$ ",
    hint: "Кликни класс или задачу; стрелки  $\leq$ р показывают вclusions",
    Component: ComplexityClassesViz,
  },
  intro: { title: "Мнемокарточки", Component: MnemonicCards },
};

export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 76/92: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useEffect, useState } from 'react';
import { useVizRuntime, vizNumber, vizString } from '../viz-runtime';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для Waterfall
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
  x: number; // Координаты для SVG водопада
  y: number;
  depth: number;
  fail: number;
```

```
const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');
```

```
// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
```

```
const [textToScan, setTextToScan] = useState<string>('');
const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('');
    'Режим поиска: Введите текст и жмите «Следующий шаг»';
});
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 });
const [foundMatches, setFoundMatches] = useState<{ index: number; text: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);
```

```
// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
```

```
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
```

```
const [trainStep, setTrainStep] = useState<number>(0);
```

```
// — Синхронизация с Python-компилятором: s/текст, i/индекс, v/вершина
```

```
// v — текущая вершина автомата. Влияет на режим поиска
```

```
const runtime = useVizRuntime();
```

```
const liveText = vizString(runtime?.variables?.s) ?? ' ';
```

```
const liveI = vizNumber(runtime?.variables?.i);
```

```
const liveNode = vizNumber(runtime?.variables?.v) ?? 0;
```

```
useEffect(() => {
```

```
    if (liveText) {
```

```
        setTextToScan(liveText.toUpperCase().replace(/[^А-Яа-я0-9]/g, ''));
```

```
        resetSearchSimulation();
```

```
    }
```

```
}, [liveText]);
```

```
useEffect(() => {
```

```
    if (liveI !== null) setCurrentIndex(Math.max(0, Math.min(liveI, textToScan.length)));
```

```
}, [liveI, textToScan.length]);
```

```
useEffect(() => {
```

```
    if (liveNode !== null && WATERFALL_NODES[liveNode])
```

```
}, [liveNode]);
```

```
const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
```

```

    } else {
      setActiveSearchCodeLine(2);
      let jumps = 0;
      while (curr !== 0 && TRIE_TRANSITIONS[curr][char]) {
        curr = WATERFALL_NODES[curr].fail;
        jumps++;
      }
      const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
      if (originalCurr === 0) {
        logMsg += `В корне нет перехода по '${char}'. Логика  

        setJumpPath(null);
      } else {
        logMsg += `Поток по '${char}' у вершины #${originalCurr} не  

        рывок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].fail}).  

        `;
        setJumpPath({ from: originalCurr, to: nextNode, jumps: jumps });
      }
      curr = nextNode;
    }

    const currentMatches: { index: number; word: string }[] = [];
    const matchedWordsForLog: string[] = [];

    let temp = curr;
    while (temp !== 0) {
      if (WATERFALL_NODES[temp].is_terminal) {
        WATERFALL_NODES[temp].words.forEach((word) => {
          currentMatches.push({ index: currentIndex, word });
          matchedWordsForLog.push(word);
        });
      }
      temp = WATERFALL_NODES[temp].term_link;
    }

    if (matchedWordsForLog.length > 0) {
      logMsg += ` БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}  

      setActiveSearchCodeLine(4);
    }
  }

```

```

        codeLine: 5,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 2,
        title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
        desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель: "H".  
а-разведчик плывет в ROOT и ищет ветку по "E".',
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'E',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 5,
        title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
        desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель: "H".  
"H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'I',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 8,
        title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ  
ВАЖНЫМ ВОПРОС! Обрабатываем #8 (SH) на Depth 2.  
(SH) — это важный вопрос. Рыба плывет в ROOT и ищет ветку "H". У корня  
суффикс "H"!',
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'H',
    },

```

```

    desc: 'Финальная вершина #4 (HERS) на Depth 4. По
      #7 (S). fail[HERS] = #7 (S). Автомат полностью
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: 0,
  },
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainStep];

```

```

return (
  <div className="bg-slate-800/95 rounded-2xl p-6 border"
    {/* Шапка и переключатель режимов */}
    <div className="flex flex-wrap items-center justify-between"
      <div className="flex items-center space-x-3">
        <span className="text-3xl"> </span>
        <div>
          <h4 className="text-2xl font-bold text-white">
            Анимация водопада: Полное обучение автомата
          </h4>
          <p className="text-sm text-slate-400">
            Визуализация пошагового построения всех fish
          </p>
        </div>
      </div>
    </div>

```

```

    {/* Переключатель режимов */}
    <div className="bg-slate-900 p-1.5 rounded-xl border"
      <button
        onClick={() => setActiveMode('training')}
        className={`flex items-center space-x-1.5 p-1.5`}
        activeMode === 'training'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      >

```

```

        className={`w-full text-left px-3 py-2`}
      ) {
        idx === trainStep
          ? 'bg-indigo-600 text-white font-medium'
          : 'bg-slate-800 text-slate-400 font-normal'
      }` }
    >
    <span className="line-clamp-1">{stepInfo[idx] || '...'}</span>
  </button>
  )})
</div>
</div>

<div className="flex gap-2">
  <button
    onClick={() => setTrainStep((prev) => Math.max(0, prev - 1))}
    disabled={trainStep === 0}
    className="flex-1 bg-slate-700 hover:bg-slate-600 transition-all"
  >
    Назад
  </button>
  <button
    onClick={() => setTrainStep((prev) => Math.min(
      TRAINING_STEPS_INFO.length,
      prev + 1
    ))}
    disabled={trainStep === TRAINING_STEPS_INFO.length}
    className="flex-1 bg-indigo-600 hover:bg-indigo-500 transition-all shadow-lg shadow-indigo-900/50"
  >
    Далее
  </button>
</div>
</div>

/* Блок с кодом построения fail-ссылки и описания ошибки
<div className="lg:col-span-2 bg-slate-900/90 p-4"
  >
    <div>

```

```

        {currentTrainInfo.codeLine === 5 && <
    root</span>}
    </div>
  </div>
</div>

<div>
  <h5 className="text-slate-200 font-bold mb-2">
    <Lightbulb className="w-4 h-4 text-amber-400"/>
  </h5>
  <div className="bg-slate-800 p-4 rounded-xl w-full min-h-[72px] flex items-center">
    {currentTrainInfo.desc}
  </div>
</div>
</div>
) : (
  /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
  <div className="grid grid-cols-1 lg:grid-cols-3">
    <div className="bg-slate-900/60 p-5 rounded-xl">
      <div>
        <h5 className="text-white font-bold mb-2">
          <span>» </span> Текст для сканирования
        </h5>
        <p className="text-xs text-slate-400 mb-4">
          Введи текст, чтобы лось просканировал
        </p>
        <input
          type="text"
          value={textToScan}
          onChange={handleTextChange}
          maxLength={15}
          placeholder="ВВЕДИТЕ ТЕКСТ..."
          className="w-full bg-slate-800 border border-slate-700 focus:outline-none focus:border-blue-500"
        />
      </div>
    </div>
  </div>

```



```

<div className="w-full overflow-x-auto custom-s
  <svg viewBox="0 0 800 520" className="w-full

```

```

  {/* Горизонтальные линии и подписи уровней */}
  {[
    { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
    { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
    { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
    { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
    { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' },
  ]}.map((level) => (
    <g key={`depth-line-${level.depth}`}>
      <line
        x1="20"
        y1={level.y}
        x2="780"
        y2={level.y}
        stroke="#334155"
        strokeWidth="1"
        strokeDasharray="4,4"
        opacity="0.4"
      />
      <rect x="25" y={level.y - 12} width="14" height="24" />
    </g>
    <text x="32" y={level.y + 4} fill="#94a3b8">
      {level.label}
    </text>
  ))

```

```

  {/* Рендер прямых переходов (потоков воды) */}
  {Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
    const fromNode = WATERFALL_NODES[Number(fromId)];
    return Object.entries(children).map(([childId, childNode]) => {
      const toNode = WATERFALL_NODES[toId];

```

```

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ? true : false;

```

```

        <text x="0" y="5" fontSize="14"
            {isMatchBranch ? ' ' : ' '}
        </text>
    </g>
  })
</g>
);
});
}}

```

```

{/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
  if (node.id === 0) return null;
  const targetNode = WATERFALL_NODES[node.failId];

```

```

  // В режиме обучения показываем fail-ссылку
  // Найдем индекс шага, на котором обработали
  const stepIdxForNode = TRAINING_STEPS_INFO[node.id].stepIdx;
  if (activeMode === 'training' && trainStep === stepIdxForNode) {

```

```

    let isHighlighted = activeMode === 'search' || activeMode === 'fail';
    let isJustEstablished = activeMode === 'training' && trainStep === stepIdxForNode;
    // -ссылку

```

```

    // Вычисляем изогнутую кривую для красоты
    const dx = targetNode.x - node.x;
    const dy = targetNode.y - node.y;
    const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -10 : 10);
    const cy = node.y + dy / 2 - 30;

```

```

    return (
      <g key={`fail-${node.id}`}>
        <path
          d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
          fill="none"
          stroke={isHighlighted ? '#f43f5e' : '#ccc'}
          strokeWidth={isHighlighted ? 2 : 1}
        />
      </g>
    );
  }
}

```


Пока ничего не поймали. Запустите шаги с

```

    </p>
  ) : (
    <div className="flex flex-wrap gap-2">
      {foundMatches.map((match, idx) => (
        <div
          key={idx}
          className="bg-emerald-950 border border-
-sm font-bold shadow animate-[scaleUp_0.3s_
        >
          <CheckCircle2 className="w-4 h-4 text
          <span>{match.word}</span>
          <span className="text-[10px] bg-emera
            Индекс {match.index}
          </span>
        </div>
      )}}
    </div>
  )}
</div>
)}
</div>
);
};

```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 77/92: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 411 | РАЗ

```

import React, { useContext, useEffect, useMemo, useStat
import {
  emitVizDemo,

```

```

return (
  <div className="w-full bg-slate-950 rounded-2xl border-2 border-indigo-600">
    <div className="flex gap-2 mb-4 overflow-x-auto">
      {{
        [
          ["1d", "1. Одномерные суммы"],
          ["2d", "2. Двумерные суммы"],
        ] as const
      }.map(([key, label]) => {
        <button
          key={key}
          onClick={() => setMode(key)}
          className={`px-3 py-2 rounded-lg text-xs sm:text-sm
            mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
          }`>
          >
            {label}
          </button>
        )}}
      </div>
      {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
    </div>
  );
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
  const pref = useMemo(
    () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]], []),
    []
  );
};

```

```

/** Что сейчас под курсором: префикс P[i] или элемент A[i]
const [hover, setHover] = useState<{ kind: "pref" | "el" }>({ kind: "pref" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;

```

```

const inRange = i >= range.l && i <= range.r
const isDrivenAdd = shownDriven !== null
return (
  <div
    key={i}
    onMouseEnter={() => setHover({ kind: 'add' })}
    onMouseLeave={() => setHover(null)}
    onClick={() => setRange((r) => (i < r.l ? { l: i, r: r.r } : { l: r.l, r: i }))}
    className={` ${cellBase} cursor-pointer ${
      isDrivenAdd
        ? "bg-amber-500 text-black ring-2 ring-black"
        : inCover
        ? "bg-indigo-500 text-white ring-2 ring-indigo-500"
        : inRange
        ? "bg-indigo-900/70 text-indigo-300"
        : "bg-slate-800 text-slate-300"
      }`}
    title={`a[${i}] = ${v}`}
  >
    {v}
  </div>
);
}}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
  <div className="w-[46px] shrink-0 text-right">
    {pref.map((v, i) => {
      const active = hover?.kind === "pref" && i === range.l;
      const isL = i === range.l;
      const isR = i === range.r + 1;
      const liveValue = liveP?.[i];
      const blank = hasLiveP && (liveValue === null || liveValue === 0);
      const masked = !hasLiveP && shownDriven !== null;
      const isDrivenCur = shownDriven !== null && i === shownDriven;
      const shownValue = hasLiveP ? liveValue : 0;
      return (

```

```

    </p>
  ) : hover?.kind === "pref" ? {
    hover.i === 0 ? {
      <p className="text-slate-300">
        <b className="text-emerald-400">P[0] = 0</b>
        отрезка, начинающегося с нуля.
      </p>
    } : {
      <p className="text-slate-300">
        <b className="text-emerald-400">
          P[{hover.i}] = {pref[hover.i]}
        </b>{" "}
        – это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
          a[0..{hover.i - 1}] = {A1.slice(0, hove
        </span>
      </p>
    }
  ) : hover?.kind === "a" ? {
    <p className="text-slate-300">
      <b className="text-indigo-400">
        a[{hover.i}] = {A1[hover.i]}
      </b>{" "}
      входит во все префиксы, начиная с{" "}
      <span className="font-mono text-emerald-300">
        подвинуть границу запроса.
      </span>
    </p>
  ) : {
    <p className="text-slate-500">
      Наведите курсор на любое число: у <b classN
      у <b className="text-indigo-400">a</b> – ку
    </p>
  }}
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
  <div className="text-xs text-slate-400 mb-2">

```

```

const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

useEffect(() => {
  const r1 = vizNumber(vars?.r1);
  const c1 = vizNumber(vars?.c1);
  const r2 = vizNumber(vars?.r2);
  const c2 = vizNumber(vars?.c2);
  if (r1 === null || c1 === null || r2 === null || c2 === null) {
    setRect({
      r1: Math.max(0, Math.min(n - 1, Math.trunc(r1))),
      c1: Math.max(0, Math.min(m - 1, Math.trunc(c1))),
      r2: Math.max(0, Math.min(n - 1, Math.trunc(r2))),
      c2: Math.max(0, Math.min(m - 1, Math.trunc(c2))),
    });
  }, [vars, n, m]);

  const D = P[rect.r1][rect.c1];
  const B = P[rect.r1][rect.c2 + 1];
  const Cc = P[rect.r2 + 1][rect.c1];
  const Aa = P[rect.r2 + 1][rect.c2 + 1];
  const total = Aa - B - Cc + D;

  const cornerOf = (i: number, j: number) => {
    if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
    if (i === rect.r1 && j === rect.c2 + 1) return "B";
    if (i === rect.r2 + 1 && j === rect.c1) return "C";
    if (i === rect.r1 && j === rect.c1) return "D";
    return null;
  };

  return (
    <div className="space-y-5">
      <div className="grid gap-5 lg:grid-cols-2">
        {/* Исходная матрица */}
        <div className="overflow-x-auto">
          <div className="text-xs text-slate-400 mb-2">Исходная матрица</div>
          <div className="inline-block">
            {A2.map((row, i) => {

```



```

const cornerColor =
  corner === "A"
    ? "bg-emerald-600 text-white"
    : corner === "D"
      ? "bg-sky-600 text-white"
      : corner
        ? "bg-rose-600 text-white"
        : "bg-slate-900 border border
return (
  <div
    key={j}
    onMouseEnter={() => setHover({ i,
    onMouseLeave={() => setHover(null
    className={`${cellBase} cursor-he
    isCompilerCell
      ? "bg-emerald-600 text-white
      : isHover
        ? "bg-amber-500 text-white
        : blank
          ? "bg-slate-950 text-slat
          : cornerColor
    }`}
    title={hasLiveS ? `S[${i}][${j}]` :
  >
    {blank ? "." : String(shownValue)}
    {corner && !isHover && (
      <span className="absolute -top-
ht">
      {corner}
    </span>
    )}
  </div>
);
}}
</div>
)))
</div>
</div>

```

```

    useVizRuntime,
    vizArray,
    vizNumber,
    vizRecord,
    VizChapterContext,
  } from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j - 1)][j - 1]);
  }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],

```

```
export const SparseTableViz: React.FC = () => {
  // Эта страница посвящена именно двумерной таблице: 1D
  // вкладкой, но больше не подменяет 2D-визуализацию и
  const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">("1d")
  const chapterId = useContext(VizChapterContext);

  // Каждая вкладка = своё демо с собственным скелетом:
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab === "2d_build" ? "2d_build" : "2d_query", [chapterId, tab]);
  }, [chapterId, tab]);

  return (
    <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
      <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
        <button
          onClick={() => setTab("1d")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white : "bg-slate-800 text-slate-400`}
        >
          1. Сворачивание 1D
        </button>
        <button
          onClick={() => setTab("2d_build")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white : "bg-slate-800 text-slate-400`}
        >
          2. Сворачивание 2D (Построение)
        </button>
        <button
          onClick={() => setTab("2d_query")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white : "bg-slate-800 text-slate-400`}
        >
          3. Запрос 2D (Формула)
        </button>
      </div>
      <AnimatePresence mode="wait">
```

```

        computeSteps.push({ i, j });
    }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-600">
          Построение Sparse Table (Min)
        </h3>
        <p className="text-sm text-slate-400">
          Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$ 
        </p>
      </div>
      <div className="flex gap-2">
        <button
          onClick={() => setStep(Math.max(0, step - 1))}
          className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
          disabled={step === 0}
        >
          <ChevronLeft size={20} />
        </button>
        <button
          onClick={() => setStep(Math.min(maxStep, step + 1))}
          className="p-2 bg-indigo-600 hover:bg-indigo-700 disabled:opacity-50"
          disabled={step === maxStep}
        >
          <ChevronRight size={20} />
        </button>
        <button
          onClick={() => setStep(0)}

```

```

        — это минимум на отрезке{" "}
        <span className="font-mono text-sky-300">
            [{covered.from}, {covered.to}]
        </span>{" "}
        длины  $2^{\text{hover.j}} = \{1 \ll \text{hover.j}\} \cdot \{ " \}$ 
        <span className="font-mono text-slate-400">
            min({arr1D.slice(covered.from, covered.to)})
        </span>
    </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор (или тапните) на любое число
        </p>
    )}
</div>
</div>

<div className="overflow-x-auto pb-2">
    <div className="inline-block min-w-full bg-slate-50">
        <div className="grid grid-cols-[auto_repeat(4,1fr)]">
            { /* Headers */ }
            <div className="text-slate-500 font-mono text-sm">
                i \ j
            </div>
            <div className="text-center font-bold text-slate-500">
                 $2^0$  (L=1)
            </div>
            <div className="text-center font-bold text-slate-500">
                 $2^1$  (L=2)
            </div>
            <div className="text-center font-bold text-slate-500">
                 $2^2$  (L=4)
            </div>
            <div className="text-center font-bold text-slate-500">
                 $2^3$  (L=8)
            </div>
        </div>
    </div>
</div>

```

```

// Готовые вычисленные уровни демо не
// уровень j=0 всегда берём из уже сущ
const shownValue = compilerLinked ? r
const displayValue =
  shownValue === undefined || shownVa
    ? "."
    : typeof shownValue === "object"
      ? JSON.stringify(shownValue)
      : String(shownValue);

return (
  <div key={j} className="relative fl
    {!isValid ? (
      <div className="w-[clamp(30px,9
    ) : (
      <motion.div
        onMouseEnter={() => (compiler
        onMouseLeave={() => setHover(
        onClick={() => (compilerLinker
    )}}

    initial={{ opacity: 0, scale:
    animate={{
      // При st = [] каркас уже с
      opacity: compilerLinked ? 1
      scale: compilerLinked || is
    }}
    className={`w-[clamp(30px,9vw
clamp(11px,3vw,17px)] font-bold transition-
hover && hover.i === i && h
    ? "bg-sky-500 text-white
    : isActive
    ? "bg-indigo-500 text-whi
    : isSrc1
    ? "bg-emerald-500 text-v
    : isSrc2
    ? "bg-amber-500 text-v
    : isVisible

```

```

    </div>
  </div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
  {compilerLinked && liveTarget ? (
    <p className="text-base text-slate-300">
      <span className="text-emerald-400 font-bold">
        <span className="font-mono text-white">i =
        – активна ячейка <span className="font-mono
        {(hasLiveTable || hasLiveSource) && <> = <sp
        ?.liveTarget.j] ?? (liveTarget.j === 0 ? l
    </p>
  ) : step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return {
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-l
              ST[{c.i}][{c.j - 1}]
            </span>
            ,
            <span className="text-amber-400 font-bo
              ST[{c.i + half}][{c.j - 1}]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{stl
            <span className="text-amber-400">
              {stlD[c.i + half][c.j - 1]}
            </span>
          ) ={" "}
            <span className="text-indigo-400 font-b

```

```

    setHover(null);
    setLocked(null);
  }, [k]));

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
  <div className="flex flex-col xl:flex-row gap-6">
    <div className="flex-1 min-w-0 bg-slate-900 p-3 scrollable">
      <h3 className="text-xl font-bold text-indigo-400">Задача
      <p className="text-sm text-slate-400 mb-6 text-wrap">
        Для наглядности показываем только <b className="text-rose-300">
          <span className="text-rose-300 animate-pulse">
            иц!</span>
      </p>

      <div className="flex bg-slate-950 p-1 rounded-lg">
        {[0, 1, 2, 3].map(step => (
          <button
            key={step}
            onClick={() => { setK(step); setHover(null); }}
            className={`px-4 py-2 rounded-md text-sm text-slate-400
              hover:text-white hover:bg-slate-900`}
          >
            {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
          </button>
        ))}
      </div>

      <div className="flex flex-col items-center gap-4 overflow-y-auto no-scrollbar">
        {[...Array(shownK + 1)].map((_, i) => shownK
          const stepL = 1 << step;
          const stepSize = 8 - stepL + 1;
          const isCurr = step === shownK;

```



```

        zIndex = 'z-10';
    } else {
        const prevL = 1 << (shownK
        const isTop = r < activeCel
        const isLeft = c < activeCe

        if (isTop && isLeft) {
            highlightClass = 'bg-ros
md:scale-[1.08]';
        } else if (isTop && !isLeft
            highlightClass = 'bg-blur
] md:scale-[1.08]';
        } else if (!isTop && isLeft
            highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
        } else {
            highlightClass = 'bg-amb
3)] md:scale-[1.08]';
        }
        zIndex = 'z-10';
    }
}

return {
    <div
        key={idx}
        onMouseEnter={() => { if(is
        onMouseLeave={() => { if(is
        onClick={() => {
            if(isCurr) {
                if(locked && locked.
                else setLocked({r, c
            }
        }}
        className={`flex items-centr
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
        title={hasLiveTable ? `st2[

```



```

        </div>

        <div className="pt-3 mt-3 border-t
          ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][shownK
        </div>
        <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
        </div>
      ) : {
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
        Наведите курсор на матрицу {L}x{L
        </div>
      )}
    </div>
  )}
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);
  const runtime = useVizRuntime();

  // Координаты запроса – те же захардкоженные имена,
  useEffect(() => {
    const vars = runtime?.variables;
    const nr1 = vizNumber(vars?.r1);
    const nc1 = vizNumber(vars?.c1);
    const nr2 = vizNumber(vars?.r2);
    const nc2 = vizNumber(vars?.c2);
    if (nr1 !== null) setR1(Math.max(0, Math.min(7, M

```

```

        <div
            className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
            style={{
                top: `${(r1 / 8) * 100}%`,
                left: `${(c1 / 8) * 100}%`,
                marginTop: '8px',
                marginLeft: '8px',
                height: 'calc(' + (H/8*100) +
                width: 'calc(' + (W/8*100) +
            }}
        />

```

```

        {/* Показываем 4 угла в самой матрице */}
        <div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px}`, height: `ca
        <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px}`, l
        <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px}`, width: `calc(`${(Ly / 8) * 100}% - 16
        <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px}`, width: `calc(`${(Ly / 8) * 100
        </div>

```

```

        <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
        <label className="flex items-cen
            <span className="text-slate-
                <div className="flex gap-2">
                    <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
                    <input type="number" min={
14 bg-slate-900 border border-slate-700 py-

```

```

    <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

```

```

    <span className="text-slate-500">// 2.
    <span className="text-indigo-400">int<
    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
В запроса)</span><br/>

```

```

    &nbsp;&nbsp;&nbsp;<span className="text-rose-900">
r1</span>][<span className="text-white bg-r

```

```

    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
во, чтобы правый край уперся в c2)</span><b
    &nbsp;&nbsp;&nbsp;<span className="text-blue-500">
r1</span>][<span className="text-white bg-b

```

```

    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
рх, чтобы нижний край уперся в r2)</span><b
    &nbsp;&nbsp;&nbsp;<span className="text-emerald-500">
nded">r2 - Lx + 1</span>][<span className="

```

```

    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
и влево от c2)</span><br/>

```

```

    &nbsp;&nbsp;&nbsp;<span className="text-amber-500">
">r2 - Lx + 1</span>][<span className="text
    {'}}'));
</div>

```

```

<div className="w-full flex justify-cent
<div className="flex flex-col items-ce
    <h4 className="text-slate-400 font-bo
        Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{k
    </h4>

```

```

    <div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>
    {Array.from({length: matRows * ma
        const r = Math.floor(idx / ma
        const c = idx % matCols;

```

```

        n>
            <span className="text-emerald-400"
        span>
            <span className="text-amber-400"
            &nbsp;<span className="text-slate-400"
            <span className="ml-3 text-emerald-400"
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
            </div>
        </div>
    </div>
</div>
)
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 79/92: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```

        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').cla
            ? 'px-3 py-1 rounded text-sm font-bold'
            : 'px-3 py-1 rounded text-sm font-bold'

        document.getElementById('btn-mode-wtf').cla
            ? 'px-3 py-1 rounded text-sm font-bold'
```

```

        if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.remove('text-white transition-colors') : 'px-3 py-1 rounded transition-colors';
        document.getElementById('btn-theme-dark').classList.remove('text-white transition-colors') : 'px-3 py-1 rounded transition-colors';
        document.getElementById('btn-theme-light').classList.remove('text-white transition-colors') : 'px-3 py-1 rounded transition-colors';
        document.getElementById('btn-theme-notebook').classList.remove('text-white transition-colors') : 'px-3 py-1 rounded transition-colors';
    }

    // Initialize theme and mode
    setTheme('dark');

    // Setup intersection observer for TOC highlighting
    document.addEventListener('DOMContentLoaded', () => {
        const mobileMenuBtn = document.getElementById('mobile-menu-btn');
        const mobileDrawer = document.getElementById('mobile-drawer');
        const closeDrawerBtn = document.getElementById('close-drawer-btn');
        const drawerOverlay = document.getElementById('drawer-overlay');

        function toggleDrawer() {
            const isClosed = mobileDrawer.classList.contains('hidden');
            if (isClosed) {
                mobileDrawer.classList.remove('hidden');
                if (drawerOverlay) {
                    drawerOverlay.classList.remove('hidden');
                    // Timeout allows display:block to take effect
                    setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
                }
                document.body.style.overflow = "hidden";
            } else {
                mobileDrawer.classList.add('hidden');
                if (drawerOverlay) {
                    drawerOverlay.classList.add('hidden');
                }
                document.body.style.overflow = "auto";
            }
        }

        mobileMenuBtn.addEventListener('click', toggleDrawer);
        closeDrawerBtn.addEventListener('click', toggleDrawer);
        drawerOverlay.addEventListener('click', toggleDrawer);
    });

```

```
                link.classList.add('bg-
            } else {
                link.classList.remove('
            }
        });
    }
    });
}, observerOptions);

document.querySelectorAll('section[id^="que
    observer.observe(section);
});
});
</script>
</body>
</html>
```

ФАЙЛ 80/92: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕ

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js
  <script id="MathJax-script" async src="https://cdn.
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\(', '\\)']]
        displayMath: [['$$', '$$'], ['\\[', '\\
      }
    }
  }
</script>
```



```

mjsx-container[display="true"] {
  overflow-x: auto;
  overflow-y: hidden;
  max-width: 100%;
  scrollbar-width: none;
  -ms-overflow-style: none;
  -webkit-overflow-scrolling: touch;
}
mjsx-container[display="true"]::-webkit-scrollbar
  display: none;
}

@media (max-width: 640px) {
  html { font-size: 13px; }

  .uno-card { width: 40px; height: 60px; font-size: 12px; }
  .uno-card::before, .uno-card::after { font-size: 12px; }
  .uno-card::before { top: 1px; left: 2px; }
  .uno-card::after { bottom: 1px; right: 2px; }

  .uno-card.mini { width: 28px !important; height: 40px !important; font-size: 4px; }
  .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }

  .uno-equation .text-2xl { font-size: 1.2rem; }
  .uno-equation.gap-4 { gap: 0.5rem !important; }

  /* Allow horizontal scroll for equations in formula scroll
  .formula-scroll { flex-wrap: nowrap !important; }
  .formula-scroll { width: 100%; }
}

.uno-inner {
  position: absolute; width: 110%; height: 75%;
  border-radius: 50%; transform: rotate(-30deg);
  box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
}

.uno-val { position: relative; z-index: 2; }
.card-red { background-color: #e52521; }

```

```

    }
    body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
        background-color: rgba(255, 255, 255, 0.7);
        border-color: #cbd5e1 !important;
        box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
    }
    body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1 !important; }
    body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 { border-color: #94a3b8 !important; }
    body.theme-notebook .font-mono { font-family: "Monospace"; }
    body.theme-notebook #top-controls { background-color: #f0f0f0; }

    /* CREEPY BLINKING */
    @keyframes creepy-blink {
        0%, 95%, 98%, 100% { transform: scaleY(1); }
        96.5% { transform: scaleY(0.1); }
    }
    .creepy-eye {
        transform-origin: center;
        transform-box: fill-box;
        animation: creepy-blink 4s infinite;
    }
    .creepy-eye-alt {
        transform-origin: center;
        transform-box: fill-box;
        animation: creepy-blink 5.2s infinite;
        animation-delay: 1.5s;
    }
    .creepy-emoji {
        display: inline-block;
        transform-origin: center;
        animation: creepy-blink 3.8s infinite;
    }
</style>

```



```
lg:rounded-2xl lg:border lg:border-slate-700
  <div class="flex justify-between items-center">
    <h3 class="font-black text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-indigo-500 text-sm">
      Содержание
    </h3>
    <button id="close-drawer-btn" class="text-slate-500 hover:text-slate-700">
      <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6">
        <path stroke-linecap="round" stroke-width="2" d="M6 18 18 6" />
      </svg>
    </button>
  </div>
  <nav class="space-y-3 text-sm font-medium">
    <details class="group" open>
      <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
        1. Алгебраические структуры
      </summary>
      <div class="pl-6 space-y-2 text-slate-500">
        <a href="#section-1-1" class="block">1.1. Группы
        <a href="#section-1-2" class="block">1.2. Кольца
        <a href="#section-1-3" class="block">1.3. Поля
        <a href="#section-1-4" class="block">1.4. Модули
      </div>
    </details>
    <details class="group">
      <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
        2. Комплексные числа
      </summary>
      <div class="pl-6 space-y-2 text-slate-500">
        <a href="#section-2-1" class="block">2.1. Комплексные числа
        <a href="#section-2-2" class="block">2.2. Комплексные функции
      </div>
    </details>
  </nav>
</div>
```

er-fuchsia-500 pl-3 font-bold text-fuchsia-300">

5. Группа корней из единицы

</summary>

<div class="pl-6 space-y-2 text-fuchsia-300">

5.1. Определение

5.2. Группы корней из единицы

5.3. Простые корни из единицы

5.4. Критерий простоты

5.5. Циклические группы

</div>

</details>

<details class="group">

<summary class="list-none cursor-pointer text-fuchsia-300">

6. Делимость, НОД и Евклид

er-lime-500 pl-3 font-bold text-lime-300">

6. Делимость, НОД и Евклид

</summary>

<div class="pl-6 space-y-2 text-lime-300">

6.1. Делимость

6.2. НОД и НОК

6.3. Алгоритм Евклида

6.4. Простые делители

6.5. Простые делители

6.6. Простые делители

</div>

</details>

<details class="group">

<summary class="list-none cursor-pointer text-pink-300">

7. Взаимно простые числа

er-pink-500 pl-3 font-bold text-pink-300">

7. Взаимно простые числа

</summary>

```

        <a href="#q9-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
        10. Делимость и Схема Г
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-rose-400">
        <a href="#q10-1" class="block">
        <a href="#q10-2" class="block">
        <a href="#q10-3" class="block">
a>
            <a href="#q10-4" class="block">
            <a href="#q10-5" class="block">
        </div>
    </details>

    <details class="group">
        <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
            11. НОД и НОК многочленов
        </a>
        </summary>
        <div class="pl-6 space-y-2 text-cyan-400">
            <a href="#q11-1" class="block">
            <a href="#q11-2" class="block">
нственности НОД</a>
            <a href="#q11-3" class="block">
ПИРАЛЬ)</a>
            <a href="#q11-4" class="block">
вращается!)</a>
        </div>
    </details>
```

```

        <a href="#q14-1" class="block">
        <a href="#q14-2" class="block">
        <a href="#q14-3" class="block">
deg)</a>
        <a href="#q14-4" class="block">
ition-colors border border-rose-500/30"> K
        </div>
    </details>

    <details class="group">
        <summary class="list-none cursor-pointer">
            <a href="#question-15" class="block border-cyan-500 pl-3 font-bold text-cyan-400">
                15. Производная. Критерий
            </a>
        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q15-1" class="block">
            <a href="#q15-2" class="block">
            <a href="#q15-3" class="block">
            <a href="#q15-4" class="block">
        </div>
    </details>

    <details class="group">
        <summary class="list-none cursor-pointer">
            <a href="#question-16" class="block border-red-500 pl-3 font-bold text-red-400">
                16. Неприводимые многочлены
            </a>
        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q16-1" class="block">
            <a href="#q16-2" class="block">
            <a href="#q16-3" class="block">
        </div>
    </details>
```

```

        other.removeAttr('class');
    });
});

// Scroll spy logic
document.addEventListener('DOMContentLoaded', () => {
    const sections = document.querySelectorAll(
        "#q5, #q6, #q7, #q8, #q9, #q10, #q11, #q12, #q13, #q14, #q15, #q16, #q17, #q18, #q19, #q20, #q21, #q22, #q23, #q24, #q25, #q26, #q27, #q28, #q29, #q30, #q31, #q32, #q33, #q34, #q35, #q36, #q37, #q38, #q39, #q40, #q41, #q42, #q43, #q44, #q45, #q46, #q47, #q48, #q49, #q50, #q51, #q52, #q53, #q54, #q55, #q56, #q57, #q58, #q59, #q60, #q61, #q62, #q63, #q64, #q65, #q66, #q67, #q68, #q69, #q70, #q71, #q72, #q73, #q74, #q75, #q76, #q77, #q78, #q79, #q80, #q81, #q82, #q83, #q84, #q85, #q86, #q87, #q88, #q89, #q90, #q91, #q92, #q93, #q94, #q95, #q96, #q97, #q98, #q99, #q100"
    );
    let isScrolling = false;

    document.querySelectorAll('.group').forEach((group) => {
        group.addEventListener('click', () => {
            isScrolling = true;
            setTimeout(() => isScrolling = false, 1000);
        });
    });

    const observer = new IntersectionObserver((entries) => {
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersecting) {
                visibleId = entry.target.id;
            }
        });

        // Highlight active section
        document.querySelectorAll('a').forEach((a) => {
            a.style.fontWeight = 'normal';
            if (a.href.endsWith(visibleId)) {
                a.style.fontWeight = 'bold';
            }
        });

        // Open the parent menu
    });
});

```



```
const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в ' + fileName;
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за некорректной работы веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
```

```
function downloadMD() {
    let clone = document.querySelector('div[id="md"]');

    // Smart markdown parsing based on the original text
    // Smart markdown parsing based on the original text
    clone.querySelectorAll('h2').forEach(el => {
    clone.querySelectorAll('h3').forEach(el => {
    clone.querySelectorAll('.analog').forEach(el => {
    clone.querySelectorAll('li').forEach(el => {
    clone.querySelectorAll('img').forEach(el => {
        let caption = '';
        if (el.nextElementSibling && el.nextElementSibling.tagName === 'CAPTION') {
            caption = el.nextElementSibling.textContent;
        }
        el.textContent = '\n\n[ Иллюстрация: ' + caption + ' ]';
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        el.textContent = '\n\n[ Иллюстрация: ' + el.textContent + ' ]';
    });

    let text = clone.innerHTML;
    text = text.replace(/\n{3,}/g, '\n\n');

    if (window !== window.top) {
        fallbackModal(text, 'Markdown');
        return;
    }
    const blob = new Blob([text], { type: 'text/markdown' });
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.href = url;
    link.download = 'vyisshaya_algebra.md';
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
}
```

```
function setTheme(theme) {
```

```

body.bg-white .analogy-l
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-l
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

```

```

        <span> </span> PDF
      </a>
      <button onclick="downloadMD()"
- bold flex items-center justify-center gap-2
        <span> </span> Markdown
      </button>
      <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
        <span> </span> HTML
      </button>
    </div>

    <!-- Смена темы -->
    <div>
      <div class="text-xs text-slate-500">
        <div class="flex items-center gap-2">
          <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
          <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
          <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-white
терминала"> </button>
        </div>
      </div>
    </div>
  </div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12"
- base min-w-0">
  <header class="text-center mb-12 border-b border-slate-700">
    <h1 class="text-4xl md:text-5xl font-bl
4">
```

```

<div class="overflow-x-auto w-full pb-4"
  <div class="grid grid-cols-3 gap-4"
    <!-- Column headers -->
    <div class="bg-yellow-900/40 border-1"
оля и кольца, целые числа</div>
    <div class="bg-orange-900/40 border-1"
ольцо полиномов</div>
    <div class="bg-cyan-900/40 border-1"
сные числа</div>

    <!-- Row 1: База -->
    <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
    </a>
    <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
    <div class="text-xs text-orange
    <div class="font-bold text-slate
    </a>
    <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
    <div class="text-xs text-cyan-5
    <div class="font-bold text-slate
    </a>

    <!-- Row 2: Базовые операции / Евклид -->
    <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
    </a>
    <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange
    <div class="font-bold text-slate
    </a>

```

```

<div class="hidden md:block"></div>
<a href="#question-13" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange
    <div class="font-bold text-slate
</a>
<a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
    <div class="text-xs text-cyan-5
    <div class="font-bold text-slate
</a>

<!-- Row 6: Кратность -->
<div class="hidden md:block"></div>
<a href="#question-14" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange
    <div class="font-bold text-slate
</a>
<div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange
    <div class="font-bold text-slate
</a>
<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

    <a href="#question-17" onclick="setM
-l-4 border-teal-500 hover:bg-slate-700 tra
    <div class="text-xs text-teal-5
    <div class="font-bold text-slate

```

```
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
  <div class="flex items-center mb-6">
    <div class="bg-red-500/20 p-3 r
      <svg class="w-8 h-8 text-re
rg/2000/svg">
        <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
      </svg>
    </div>
    <h2 class="text-3xl pr-4 border
  </div>

  <div class="prose prose-invert max-w
    <p class="text-lg">
      Хватит тупить. Доказательств
есть для того, чтобы жать на кнопки. Вот жес
    </p>

    <h2 class="text-2xl font-bold b
(Меметика)</h2>

    <!-- Прямое -->
    <div class="bg-slate-800/80 p-6">
      <h3 class="text-xl text-blue
      <p class="text-sm text-slate
      <ul class="list-disc pl-5 s
        <li><span class="text-w
k$).</li>

        <li><span class="text-w
        <li><span class="text-w
ово.</li>

      </ul>
    </div>

    <!-- От противного -->
```

```
</div>
```

```
<!-- Двойная делимость -->
```

```
<div class="bg-slate-800/80 p-6">
```

```
<h3 class="text-xl text-orange-400">
```

```
<p class="text-sm text-slate-400">
```

```
<ul class="list-disc pl-5">
```

```
<li><span class="text-white">
```

```
вверх по уравнениям).</li>
```

```
<li><span class="text-white">
```

```
рху вниз, показывая, что  $\Delta r_i$ 
```

```
<li><span class="text-white">
```

```
х. Твой  $d$  больше.</li>
```

```
</ul>
```

```
</div>
```

```
<h2 class="text-2xl font-bold">
```

```
оритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md">
```

```
<div class="bg-slate-800 p-6">
```

```
<h4 class="text-red-400">
```

```
<p class="text-sm text-slate-400">
```

```
<p class="text-xs font-sans">
```

```
со следующим. Если в конце 0 – пациент мертв
```

```
</div>
```

```
<div class="bg-slate-800 p-6">
```

```
<h4 class="text-red-400">
```

```
<p class="text-sm text-slate-400">
```

```
<p class="text-xs font-sans">
```

```
 $c$ . Пока получается 0 – корень жив. Как то
```

```
корня минус один.</p>
```

```
</div>
```

```
<div class="bg-slate-800 p-6">
```

```
<h4 class="text-red-400">
```



```

        </div>
    </li>

    <li class="bg-slate-800/50"
        <span class="text-2xl">
        <div>
            <h4 class="text-cyan"
            <p class="text-sm t
есть сложение — есть ноль. Если есть умноже
            <code class="text-x
        </div>
    </li>

    <li class="bg-slate-800/50"
        <span class="text-2xl">
        <div>
            <h4 class="text-cyan"
            <p class="text-sm t
авь их драться. Умножение всегда врывается
            <code class="text-x
        </div>
    </li>
</ul>
</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====

```

```
    } catch {
      const href = (sheet as CSSStyleSheet).href;
      if (!href) continue;
      try {
        const res = await fetch(href);
        if (res.ok) parts.push(await res.text());
      } catch {
        /* чужой домен – пропускаем */
      }
    }
  }
  return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid
@media print {
  body { background: #fff; color: #0f172a; }
  .export-note, details { break-inside: avoid; }
  details { display: block; }
  details > div, details > p { display: block !important
  pre { white-space: pre-wrap; }
```



```
const base = id.replace(/[a-z0-9-]+/gi, "-").toLowerCase()
return `${num ? String(num).padStart(2, "0") + "-": ""}${base}`
}
```

```
/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chap
  const css = await collectCss();
  const { body, termIds } = prepareBody(chapter.content
  const html = wrap({
    title: chapter.title,
    subtitle: chapter.category || "Универсальное пособи
    css,
    body,
    gloss: glossarySection(termIds),
    origin: window.location.origin,
  });
  triggerDownload(html, safeName(chapter.title, chapter
}
```

```

/** Вывести всё пособие одним файлом со сквозным оглавлением
export async function downloadBookHtml(chapters: Chapter[]) {
  const css = await collectCss();
  const allTerms: string[] = [];
  const bodies: string[] = [];

  const toc = chapters
    .map((c) => `<li style="margin:.2rem 0"><a href="#ch-${c.id}">${c.title}</a>`
    .join("\n");

  chapters.forEach((c) => {
    const { body, termIds } = prepareBody(c.content);
    termIds.forEach((t) => {
      if (!allTerms.includes(t)) allTerms.push(t);
    });
    bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="border: 1px solid #ccc; padding: 10px; margin-bottom: 10px;">
<p style="text-align:right;font-size:.72rem;margin:0 0 10px 0;">
    `);
  });
}

```

```

-----
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"));
const registryBody = registry.slice(registry.indexOf("Vizualizatsiya"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\\s{2}[a-zA-Z0-9_]+: /g)].map(s => s.slice(2)));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
    interactive: vizIds.has(c.id),
    chars: html.length,
  };
});

```

ФАЙЛ 84/92: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossa
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m
  external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*+?^${}()|[\]\$\s]/g, "\\
const re = new RegExp(
  `(?!<![\\p{L}\\p{N}_-])`($${GLOSSARY_LABELS.map((l) => e
  "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
```

```
/**
 * Оркестратор: бандлит TSX-генератор и запускает его.
 * Использование: node scripts/export/build-guide-pdf.mjs
 */
import { build } from "esbuild";
import { pathToFileURL } from "node:url";
import path from "node:path";
import fs from "node:fs";

const root = path.resolve(import.meta.dirname, "../..");
const outfile = path.join(root, "tmp/guide-pdf-bundle.cjs");

await build({
  entryPoints: [path.join(root, "scripts/export/guide-pdf.mjs"),
    outfile,
  ],
  bundle: true,
  platform: "node",
  format: "cjs",
  target: "node20",
  jsx: "automatic",
  loader: { ".jpg": "dataurl", ".png": "dataurl", ".css": "text", ".mjs": "jsx" },
  external: ["pdfkit", "svg-to-pdfkit", "pdfkit/js/pdfkit"],
  logLevel: "warning",
});

const outArg = process.argv[2];
await import(pathToFileURL(outfile).href);
```

ФАЙЛ 86/92: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

```
/**
 * ШАГ 3: PNG -> PDF
```

```
const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
  doc.addPage({ size: [A4.width, A4.height], margin: 0 });
  doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
  stream.on("finish", resolve);
  stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
```

ФАЙЛ 87/92: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1.1 КБ

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть)
 * с фоллбэком на обход файловой системы, если git недоступен
```



```

    "Интерактивные компоненты и симуляторы",
    "Визуализаторы (вложенные)",
    "HTML-разметка (layout)",
    "Утилиты",
    "Скрипты сборки и экспорта",
    "CI / автоматизация",
    "Прочее",
];

function listFiles() {
  let files;
  try {
    files = execFileSync("git", ["ls-files", "--cached"], {
      cwd: ROOT,
      encoding: "utf8",
    })
      .split("\n")
      .filter(Boolean);
  } catch {
    files = [];
    const walk = (dir) => {
      for (const e of fs.readdirSync(dir, { withFileType: true })) {
        const abs = path.join(dir, e.name);
        const rel = path.relative(ROOT, abs).split(path.sep);
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
        if (e.isDirectory()) walk(abs);
        else files.push(rel);
      }
    };
    walk(ROOT);
  }

  return files
    .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d + path.sep)))
    .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
    .filter((rel) => CODE_EXT.has(path.extname(rel)))
    .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
    .sort((a, b) => {

```

```
push(HR);
push("          УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
push("          ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
push(HR);
push();
push(`Дата генерации:           ${stamp}`);
push(`Файлов исходного кода:    ${files.length}`);
push(`Суммарный объём кода:      ${humanSize(totalByte
push(`Глав/билетов в пособии:    ${chapters.length}`)
push();
```

```
push(HR);
push("          ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
push(HR);
chapters.forEach((c, i) => {
  push(`  ${String(i + 1).padStart(3, " ")} . ${c.titl
});
push();
```

```
push(HR);
push("          ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
  push(`  ${String(i + 1).padStart(3, " ")} . ${rel}`)
});
push();
push();
```

```
current = null;
files.forEach((rel, i) => {
```

ФАЙЛ 88/92: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt      (collect-code.mjs)
 * txt  -> page-XXXX.png               (render-pages.mjs)
 * png  -> full_code_all_pages.pdf     (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся в CI) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
  density: Number(process.env.EXPORT_DENSITY ?? 150),
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=0 */
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",
  size: "A4",
  font: "DejaVu-Sans-Mono",
  fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
  pointsize: 8,
  /** Максимум символов в строке (далее – жёсткий перебор) */
  cols: 138,
}
```

```
/**
 * Автосборка PDF-конспекта `guide_all_topics.pdf`.
 *
 * Что внутри:
 * * весь ТЕКСТ страниц пособия (больше текста, чем HTML)
 * * картинки визуализаций – React-компоненты рендерятся
 *   (renderToStaticMarkup), из разметки берутся SVG-снимки
 * * НИКАКОГО кода компилятора и интерфейсных кнопок.
 *
 * Результат: public/export/guide_all_topics.pdf, ~40–70 МБ
 */
```

```
import * as fs from "node:fs";
import * as path from "node:path";
import { createElement } from "react";
import { renderToStaticMarkup } from "react-dom/server";
import PDFDocument from "pdfkit";
// @ts-ignore - у пакета нет типов
import SVGtoPDF from "svg-to-pdfkit/source.js";
import { chapters } from "../../src/data/content";
import { VizChapterContext } from "../../src/data/vizState";
import { VIZ_REGISTRY, getViz } from "../../src/components";
```

```
// ————— шрифты —————
```

```
const FONT_DIRS = ["/usr/share/fonts/truetype/dejavu",
function findFont(file: string): string | null {
  for (const dir of FONT_DIRS) {
    const p = path.join(dir, file);
    if (fs.existsSync(p)) return p;
  }
  return null;
}
const F_REG = findFont("DejaVuSans.ttf");
const F_BOLD = findFont("DejaVuSans-Bold.ttf");
const F_MONO = findFont("DejaVuSansMono.ttf");
```

```
// ————— HTML → блоки текста —————
```

```

}

/** Найти закрывающий тег с учётом вложенности однотипных
function findClose(html: string, openEnd: number, tag: string): number {
  const re = new RegExp(`<\\/?${tag}(\\s[^>]*)?>`, "gi");
  re.lastIndex = openEnd;
  let depth = 1;
  let m: RegExpExecArray | null;
  while ((m = re.exec(html))) {
    depth += m[0][1] === "/" ? -1 : 1;
    if (depth === 0) return m.index;
  }
  return html.length;
}

function findCloseAny(html: string, openEnd: number, tag: string): { end: number; inner: string } {
  const close = html.toLowerCase().indexOf(`</${tag}`, openEnd);
  if (close < 0) return { end: html.length, inner: html.slice(openEnd, html.length) };
  const inner = html.slice(openEnd, close);
  const end = html.indexOf(">", close) + 1;
  return { end, inner };
}

/** Разобрать HTML главы на учебные блоки (текст → 90%,
export function htmlToBlocks(html: string): Block[] {
  const blocks: Block[] = [];
  let i = 0;
  const skipTo = ["<pre", "<h2", "<h3", "<h4", "<summary"];
  while (i < html.length) {
    const lower = html.toLowerCase();
    const nexts = skipTo.map((t) => lower.indexOf(t, i));
    const liRe = /<li(\\s[^>]*)?>/gi;
    liRe.lastIndex = i;
    const liM = liRe.exec(lower);
    if (liM) nexts.push(liM.index);
    const trRe = /<tr(\\s[^>]*)?>/gi;
    trRe.lastIndex = i;
    const trM = trRe.exec(lower);
  }
}

```

```

    if (lower.startsWith("<li", at) || (liM && at === 1)) {
      const openEnd = html.indexOf(">", at) + 1;
      const close = html.toLowerCase().indexOf("</li>", at);
      const text = stripTags(html.slice(openEnd, close < 0 ? html.length : close));
      if (text) blocks.push({ kind: "li", text });
      i = close < 0 ? html.length : close + 5;
      continue;
    }
    if (lower.startsWith("<tr", at)) {
      const openEnd = html.indexOf(">", at) + 1;
      const close = html.toLowerCase().indexOf("</tr>", at);
      const rowHtml = html.slice(openEnd, close < 0 ? html.length : close);
      const cells: string[] = [];
      const tdRe = /<(t[dh])(\s[^\s]*)?>([\s\S]*?)<\/\1>/g;
      let tm: RegExpExecArray | null;
      while ((tm = tdRe.exec(rowHtml))) cells.push(stripTags(tm[3]));
      if (cells.some(Boolean)) blocks.push({ kind: "table", cells });
      i = close < 0 ? html.length : close + 5;
      continue;
    }
    // <p>
    const openEnd = html.indexOf(">", at) + 1;
    const close = html.toLowerCase().indexOf("</p>", at);
    const text = stripTags(html.slice(openEnd, close < 0 ? html.length : close));
    if (text) blocks.push({ kind: "p", text });
    i = close < 0 ? html.length : close + 4;
  }
  return blocks;
}

```

// ————— SSR визуализаций → SVG —————

```

interface VizArt {
  svgs: string[];
  images: string[]; // dataURL
  grids: string[][]; // текстовые матрицы (div-таблицы)
  error?: string;
}

```

```

    return { svgs: [], images: [], grids: [], error: St
  }
}

```

```
/**
```

```

 * Визуализации бывают и без SVG: матрицы из div-ячеек
 * префиксные суммы). Собираем такие «таблицы» в текст
 */

```

```

function extractDivGrids(html: string): string[][] {
  const grids: string[][] = [];
  const rows: string[] = [];
  const rowMark = /<div class="[^"]*(?:flex gap-1\.5|gr
  let m: RegExpExecArray | null;
  while ((m = rowMark.exec(html))) {
    // сбалансированно находим конец строки-контейнера
    let depth = 1;
    let pos = m.index + m[0].length;
    while (depth > 0 && pos < html.length) {
      const no = html.indexOf("<div", pos);
      const nc = html.indexOf("</div>", pos);
      if (nc < 0) break;
      if (no >= 0 && no < nc) { depth++; pos = no + 4;
      else { depth--; pos = nc + 6; }
    }
    const inner = html.slice(m.index + m[0].length, pos);
    // теги → разделители: порядок значений сохраняется
    const cells = decodeEntities(inner.replace(/<[>]+>
      .split(/\s+/)
      .filter((v) => v.length > 0 && v.length <= 6);
    if (cells.length >= 2 && cells.length <= 24) rows.p
  }
  // подряд идущие строки одинаковой длины = матрица; о
  let cur: string[] = [];
  const flush = () => {
    if (cur.length >= 2) grids.push(cur);
    else if (cur.length === 1 && cur[0].split("\u2009")
      cur = [];
  };
}

```

```

-----
monoFont() { this.doc.font(F_MONO ? "Mono" : "Courier"); }

get y() { return this.doc.y; }
set y(v: number) { this.doc.y = v; }
get pageBottom() { return A4[1] - M.bottom; }
ensureSpace(need: number) {
  if (this.y + need > this.pageBottom) this.doc.addPage();
}
footerAll(skipCover = true) {
  const range = this.doc.bufferedPageRange();
  const oldBottom = this.doc.page.margins.bottom;
  for (let p = range.start; p < range.start + range.count; p++) {
    if (skipCover && p === range.start) continue;
    this.doc.switchToPage(p);
    this.doc.page.margins.bottom = 0;
    this.doc.font(F_REG ? "Body" : "Helvetica").fontSize(12)
      .text(String(p + 1), 0, A4[1] - 30, { width: A4[1] - 30 });
    this.doc.page.margins.bottom = oldBottom;
  }
}
}
}

```

```

const CH_LINE = 13.4;      // межстрочный в теле
const BODY_SIZE = 9.6;

```

```

function drawTextBlock(d: Doc, text: string, opts: { size: number, color: string, indent: number, gap: number, align: string, lineGap: number }) {
  const size = opts.size ?? BODY_SIZE;
  const lead = size * 1.42;
  d.ensureSpace(lead * 2);
  d.doc.fontSize(size)
    .fillColor(opts.color ?? "#1f2937")
    .text(text, M.left + (opts.indent ?? 0), d.y, {
      width: CONTENT_W - (opts.indent ?? 0),
      lineGap: lead - size,
      align: "left",
    });
  d.doc.moveDown((opts.gap ?? 2) / lead);
}

```



```
function drawTableBlock(d: Doc, rows: string[]) {
  const size = 8.4;
  d.ensureSpace(size * 3);
  const text = rows.map((c) => c.replace(/\s*\n\s*/g, " "));
  d.doc.fontSize(size).fillColor("#374151").text(text, 0, 0);
  d.doc.moveDown(2.2);
}
```

```
function drawGrid(d: Doc, rows: string[]) {
  const size = 8;
  const lead = size * 1.6;
  d.ensureSpace(rows.length * lead + 16);
  const top = d.y;
  d.monoFont().fontSize(size).fillColor("#111827");
  const cellW = Math.min(
    34,
    (CONTENT_W - 16) / Math.max(...rows.map((r) => r.split(" ").length))
  );
  rows.forEach((row, ri) => {
    const cells = row.split(" ");
    cells.forEach((cval, ci) => {
      const x = M.left + 6 + ci * cellW;
      const y = top + 4 + ri * lead;
      d.doc.save().rect(x - 2, y - 2, cellW - 4, lead - 2);
      d.doc.text(cval, x + 2, y + lead * 0.12, { width: cellW });
    });
  });
  d.y = top + 8 + rows.length * lead + 8;
  d.doc.moveDown(2);
}
```

```
function drawSvg(d: Doc, svg: string) {
  const wAttr = Number(/width="([\d.]+)"/.exec(svg)?.[1]);
  const hAttr = Number(/height="([\d.]+)"/.exec(svg)?.[1]);
  const vb = /viewBox="([\d.\-\s]+)"/.exec(svg)?.[1]??.split(" ");
  const natW = wAttr || (vb ? vb[2] : 480);
  const natH = hAttr || (vb ? vb[3] : 320);
```

```
// ----- сборка -----

const FRONT_TOC_LINES = 44; // строк содержания на странице

function tocEntries() {
  return chapters.map((c) => ({ id: c.id, title: c.title }));
}

function tocPagesNeeded(): number {
  return Math.max(1, Math.ceil(tocEntries().length / FRONT_TOC_LINES));
}

interface BuildOpts {
  outputPath: string;
  record: boolean;
  /** id → стартовая страница (для второго прохода) */
  pageMap?: Map<string, number>;
  frontPages: number;
}

function buildOne(opts: BuildOpts): Stats {
  return new Promise<Stats>((resolve, reject) => {
    const stream = opts.record ? fs.createWriteStream("out.pdf") : null;
    const d = new Doc(stream as unknown as fs.WriteStream);
    const doc = d.doc;

    const finish = (stats: Stats) => {
      d.footerAll(!opts.record);
      doc.end();
      stream.on("close", () => resolve(stats));
      stream.on("error", reject);
    };

    if (!opts.record) {
      // — обложка (первая страница уже создана) —
      d.bodyFont();
      doc.fillColor("#312e81").fontSize(26).font(F.BOLD);
    }
  });
}
```

```

        const tw = doc.widthOfString(title);
        const pw = doc.widthOfString(String(pageNo2));
        doc.text(title, M.left, d.y, { width: w, lineBreak: true });
        const dots = ".".repeat(Math.max(3, Math.floor((w - tw - pw) / 2)));
        doc.fillColor("#d1d5db").text(dots, M.left + tw + pw, d.y);
        doc.fillColor("#111827").text(String(pageNo2), M.left + tw + pw, d.y);
        d.y += 16.2;
        line++;
    });
    // добиваем до расчётного числа страниц содержания
    const used = Math.ceil(entries.length / FRONT_TOC_ENTRIES);
    for (let p = used; p < opts.frontPages - 1; p++) {
        doc.addPage();
        if (!opts.record) console.log(`TOC: ${entries.length} pages`);
    }

    // — главы —
    const stats: Stats = { chapterStart: new Map(), totalPage: 0 };

    chapters.forEach((ch, ci) => {
        const startPageOf = () => {
            // текущая страница = pages (0-based) + 1
            return doc.bufferedPageRange().start + doc.bufferedPageRange().end;
        };
        if (ci > 0) doc.addPage();
        const pageNow = opts.record ? doc.bufferedPageRange().end : startPageOf();
        stats.chapterStart.set(ch.id, pageNow);
        if (!opts.record && ci < 3) console.log(`глава ${ci + 1}: ${ch.title}`);
        d.bodyFont();
        doc.y = M.top;
        d.y = M.top;

        // заголовок главы
        doc.fontSize(16).font(F_BOLD ? "BodyBold" : "Helvetica");
        doc.text(ch.title, M.left, M.top, { width: CONTENT_WIDTH });
        if (ch.description) {
            doc.moveDown(0.2);
            doc.fontSize(9).font(F_REG ? "Body" : "Helvetica");
            doc.text(ch.description, M.left, M.top + 10, { width: CONTENT_WIDTH });
        }
    });

```

```

        doc.fontSize(10).font(F_BOLD ? "BodyBold" : "Body")
        .text("■ Визуализация: " + vizEntry.title, d.x, d.y);
        d.y = doc.y + 6;
        let drawn = 0;
        for (const img of art.images) {
            if (drawn >= 4) break;
            drawImage(d, img);
            drawn++;
        }
        for (const svg of art.svggs) {
            if (drawn >= 5) break;
            drawSvg(d, svg);
            drawn++;
        }
        for (const grid of art.grids) {
            if (drawn >= 7) break;
            drawGrid(d, grid);
            drawn++;
        }
    } else if (art?.error) {
        // SSR упал – рисуем текстовую заглушку, PDF
    }
}
});

stats.chapterStart = d.chapterStart;
stats.total = doc.bufferedPageRange().count;
finish(stats);
});
}

async function main() {
    const outPath = process.argv[2] || path.join(process.cwd(), "out");
    if (!F_REG) throw new Error("Не найден шрифт DejaVuSans");
    const frontPages = 1 + tocPagesNeeded(); // обложка + оглавление

    // проход 1: считаем, на какой странице начинается каждая глава
    const stats = await buildOne({ outPath: "/dev/null",

```

Универсальное пособие • полный исходный код

```
-----
* Разбивает full_code_all_pages.txt на страницы формата
* каждую страницу в PNG через ImageMagick (шрифт DejaVu)
*
* Результат: tmp/export-pages/page-0001.png, page-0002
*/
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
  const expanded = line.replace(/\t/g, "    ").replace(
    if (expanded.length <= PAGE.cols) return [expanded];
  const parts = [];
  const indent = " ".repeat(Math.min((expanded.match(/^
let rest = expanded;
let first = true;
while (rest.length > 0) {
  const width = first ? PAGE.cols : PAGE.cols - indent;
  parts.push((first ? "" : indent) + rest.slice(0, width));
  rest = rest.slice(width);
  first = false;
}
return parts;
}

function paginate(txt) {
  const source = txt.replace(/\r\n/g, "\n").split("\n");
  const pages = [];
  for (let i = 0; i < source.length; i += PAGE.rows) {
    pages.push(source.slice(i, i + PAGE.rows));
  }
}
```

```

    "-strip",
    "-define", "png:compression-level=9",
    job.pngFile,
  ];

  const concurrency = Math.max(2, Math.min(os.cpus().length, 10));
  let done = 0;
  let cursor = 0;

  async function worker() {
    while (cursor < jobs.length) {
      const job = jobs[cursor++];
      await execFileAsync(im, args(job));
      fs.rmSync(job.txtFile, { force: true });
      done += 1;
      if (done % 25 === 0 || done === total) {
        process.stdout.write(`      отрендерено ${done} страниц\n`);
      }
    }
  }

  await Promise.all(Array.from({ length: concurrency }, () => worker()));

  process.stdout.write("\n");

  const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
  if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);

  const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
  console.log("[2/3] txt -> png");
  console.log(`      страниц: ${total}`);
  console.log(`      выход:   ${path.relative(ROOT, PAGE_PATH)}`);
}

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});

```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist

  deploy:
    name: Publish to GitHub Pages
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
```

Универсальное пособие • полный исходный код

промежуточные PNG сохраняются как artifacts запуска.

on:

 push:

 branches:

 - "**"

 paths-ignore:

 # чтобы коммит самого workflow не запускал бесконечный цикл

 - "public/export/**"

 - "**/*.md"

 workflow_dispatch:

 inputs:

 density:

 description: "DPI растеризации страниц (по умолчанию 300)"

 required: false

 default: "150"

concurrency:

 group: rebuild-pdf-\${{ github.ref }}

 cancel-in-progress: true

permissions:

 contents: write

jobs:

 rebuild-pdf:

 name: код → txt → png → pdf

 runs-on: ubuntu-latest

 # не реагируем на собственный коммит бота

 if: "!contains(github.event.head_commit.message, '[bot])"

 steps:

 - name: Checkout

 uses: actions/checkout@v6

 with:

 fetch-depth: 0

 persist-credentials: true


```
      continue-on-error: true

- name: Summary
  run: |
    {
      echo "###   Экспорт пересобран"
      echo ""
      echo "| Артефакт | Размер |"
      echo "| --- | --- |"
      echo "| \`public/export/full_code_all_pages"
      echo "| \`public/export/full_code_all_pages"
      echo "| \`public/export/guide_all_topics.pdf"
      echo "Страниц в конспекте: $(node -e "conso"
      echo "| PNG-страниц | $(ls tmp/export-pages."
    } >> "$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-export
    path: |
      public/export/full_code_all_pages.pdf
      public/export/full_code_all_pages.txt
      public/export/guide_all_topics.pdf
    if-no-files-found: error
    retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
```